



Copy of MAD 2 Level 2 ...
XLSX 175.6KB

this file contain questions of my viva , arrange the questions topic wise properly

MAD 2 Level 2 Viva Questions — Topic-Wise Arrangement

Below is a comprehensive topic-wise arrangement of all the questions asked across different proctors, compiled from the entire response sheet.

1. VUE.JS — CORE CONCEPTS

#	Question
1	What is Vue.js?
2	Why did you use Vue? / Why Vue over React?
3	What is a reactive framework?
4	What is Vue CLI vs Vue CDN? Which one did you use and why?
5	What is the difference between Vue 2 and Vue 3?
6	What is a Vue instance?
7	What is Virtual DOM?
8	What is the difference between Virtual DOM and Real DOM?
9	Can you manipulate Virtual DOM and change the content of original DOM?
10	What are Vue components? / What is a Single File Component (SFC)?

#	Question
11	What is the need for components?
12	What is the difference between components and pages?
13	What are Vue directives? / Built-in directives?
14	List 4 Vue directives.
15	What is v-if vs v-show? When to use which?
16	What is v-for? Show where you used it.
17	What is v-model vs v-bind?
18	What is v-cloak?
19	What is v-bind (:)?
20	What is the use of :key in v-for?
21	What are Vue lifecycle hooks?
22	Which lifecycle hooks did you use and where?
23	Difference between created and mounted.
24	Difference between beforeMount and beforeCreate.
25	What is onMounted()?
26	What is mounted?
27	What are props in Vue?
28	How does a parent communicate with a child?
29	How does a child communicate with a parent? (emit)
30	What is emit? Is it a variable or something else?
31	How to pass data from child to parent?

#	Question
32	How does a grandparent pass data to grandchild?
33	What are slots in Vue?
34	Difference between props and slots.
35	Did you use slots?
36	What are computed properties?
37	What are watchers in Vue? / How does watch work?
38	Difference between computed and watchers.
39	Difference between methods and computed properties.
40	What is data binding? / One-way vs Two-way binding.
41	What are the different ways to perform binding?
42	What is Vue Router?
43	How does Vue Router work?
44	What is router-view vs router-link?
45	What is <i>routevs</i> router?
46	What are dynamic routes in Vue?
47	What are navigation guards?
48	What is createWebHistory()?
49	What is Vuex?
50	What is Vuex vs Pinia?
51	What are state, mutations, and actions?
52	What is the use of actions?

#	Question
53	What is state management?
54	What is the advantage of state management?
55	What is Options API vs Composition API?
56	What is <code><script setup></code> ?
57	What is ref vs reactive?
58	What is the use of ref?
59	What is <code>:class</code> (colon before class)?
60	What is class and style binding?
61	What is a high-order function?
62	What are anonymous functions in JS?
63	Is every arrow function an anonymous function?
64	What is named export vs default export?
65	What is optional chaining vs nullish coalescing?
66	What is the difference between Vue and Angular?
67	What is the difference between Vue and React?
68	What are the advantages of Vue over React?
69	Why did you not use React?
70	What are the benefits of Vue.js?
71	Why do we use Vue.js rather than other JavaScript frameworks?
72	Is Vue responsive? / Is your project responsive?
73	How did you make it responsive?

#	Question
74	What is CSS vs Bootstrap?
75	How does Bootstrap make the app responsive?
76	What are CSS selectors? Types?
77	Difference between Flexbox and Grid.
78	Difference between cell spacing and cell padding.
79	What is inline CSS vs internal CSS vs external CSS?
80	Why is inline CSS given more preference by the compiler?
81	How is internal CSS better than inline CSS?
82	What is DOM manipulation?
83	What is the entry point of your frontend?
84	What is index.html?
85	How is Vue connected with index.html?
86	What is package.json? Explain all packages used.
87	What is Vite? Why did you use it?
88	What is npm?
89	What is node.js?
90	What is the use of node in your project?
91	Will node_modules be installed in the venv?
92	Where does it place the modules for the node part?
93	If you delete package-lock.json will you face any errors?
94	Do you need to do npm run build before npm run dev?

#	Question
95	What is the difference between npm run build and npm run dev?
96	What is the use of <code>useList</code> ?
97	What is <code>setup()</code> in Vue?
98	What is a template vs script vs style?
99	What is v-if vs v-for?
100	Write v-for loop in notepad.

2. FLASK & BACKEND

#	Question
1	What is Flask?
2	Is Flask open source?
3	Can you use Flask for large applications?
4	What is Flask-SQLAlchemy?
5	What is SQLAlchemy? Why use it?
6	What is ORM?
7	What are the advantages of ORM?
8	What are the disadvantages of ORM?
9	What is <code>db.Model</code> ?
10	How do you initialize a table?
11	Where is the database initialized?

#	Question
12	What is <code>__tablename__</code> ? Is it needed?
13	What is <code>__repr__</code> ?
14	What is <code>__repr__</code> VS <code>__str__</code> ?
15	What are dunder methods?
16	What is serialize (w.r.t. models)?
17	What is <code>to_dict()</code> ?
18	What are other methods for serializable data?
19	What is <code>db.String(128)</code> ? What does 128 mean? How is it stored in memory?
20	What is the difference between String and Text in database table?
21	What is <code>db.session.add()</code> VS <code>db.session.commit()</code> ?
22	What is <code>.all()</code> VS <code>.first()</code> ?
23	What is <code>.one()</code> ?
24	What is <code>query.filter()</code> VS <code>filter_by()</code> ?
25	What is <code>like</code> VS <code>ilike</code> ?
26	What is <code>%</code> in search?
27	What is <code>joinedload</code> ? Why use it?
28	What is N+1 query problem?
29	What is <code>lazy=True</code> ?
30	What is <code>lazy='dynamic'</code> ?
31	What is <code>backref</code> ?

#	Question
32	What is <code>back_populates</code> ?
33	Difference between <code>backref</code> and <code>back_populates</code> .
34	What is <code>cascade='all, delete-orphan'</code> ?
35	What is <code>onDelete='CASCADE'</code> ?
36	What is <code>uselist=False</code> ?
37	What is the difference between primary key and unique key?
38	Can unique key be primary key?
39	Can we use auto-increment instead of primary key?
40	What is a foreign key?
41	What is referential integrity?
42	How to maintain relational integrity?
43	What happens if parent row is deleted?
44	What if ON DELETE CASCADE is not used?
45	What is the impact of deleting a record which other records point to as foreign key?
46	What is relationship in SQLAlchemy?
47	What are the types of relationships? (1-1, 1-many, many-many)
48	What is <code>secondary</code> in relationship?
49	What is <code>secondary="user_roles"</code> ?
50	What is a self-referential relationship?
51	What is indexing in SQLAlchemy?

#	Question
52	How to index in SQLAlchemy?
53	What is index on two columns?
54	What is the purpose of index?
55	What are SQL constraints?
56	What is a composite key?
57	What is candidate key?
58	What is the difference between candidate key and primary key?
59	What is a unique constraint?
60	What error do you get if you insert a duplicate entry?
61	What is SQL injection?
62	What is CSRF?
63	What is CORS?
64	How is CORS configured?
65	What is a blueprint in Flask?
66	Why use blueprints?
67	What is jsonify ?
68	What is jsonify VS json.dumps ?
69	What is jsonify VS marshal ?
70	What is jsonify VS json package?
71	What is the difference between JSON and jsonify?
72	What is the difference between jsonify and json.dumps ?

#	Question
73	What is <code>app.route()</code> ?
74	What is <code>@app.route</code> decorator?
75	What are decorators?
76	How does a decorator differ from a function?
77	Write a hello world decorator.
78	What is <code>@jwt_required</code> ?
79	What is <code>@role_required</code> ?
80	What is <code>@admin_required</code> ?
81	Can we swap the order of <code>@jwt_required</code> and <code>@role_required</code> ?
82	How does <code>@admin_required</code> work according to your code?
83	What is <code>@cache.cached</code> VS <code>@cache.memoize</code> ?
84	What is the difference between <code>@cache.cached</code> and <code>@cache.memoize</code> ?
85	What is <code>@celery.task</code> ?
86	What is <code>@app.errorhandler</code> ?
87	What is <code>get_role_claims</code> ?
88	What is <code>get_user_id</code> ?
89	What is <code>Flask(__name__)</code> ? Can we remove <code>__name__</code> ?
90	What is <code>if __name__ == '__main__' ?</code> What if you remove it?
91	What is the base class of Exception?
92	What are different types of exceptions in Python?

#	Question
93	What is exception handling?
94	Have you used exception handling? Show.
95	What is <code>try ... catch</code> ?
96	What is WSGI?
97	What is <code>app_context()</code> ?
98	What is <code>with.app_context()</code> ?
99	What is the use of <code>joinedload</code> ?
100	What is the difference between synchronous and asynchronous?
101	What is an async function?
102	What does an async function return?
103	What is <code>await</code> ?
104	What is a promise?
105	What are the stages of a promise?
106	What are higher-order functions?
107	What is <code>const</code> VS <code>let</code> VS <code>var</code> ?
108	Why <code>const</code> instead of <code>let</code> or <code>var</code> ?
109	Is JavaScript statically typed or dynamically typed?
110	What is type coercion?
111	What is the difference between <code>==</code> and <code>===</code> ?
112	What is <code>this.\$router</code> ?
113	What is <code>this.\$router.push</code> ?

#	Question
114	What is <code>router.push</code> ?
115	What is <code>router.go</code> ?
116	What is <code>router.replace</code> ?
117	What is the difference between <code>redirect()</code> and <code>url_for()</code> ?
118	What is the entry point of your backend?
119	What is the purpose of <code>config.py</code> ?
120	What is the purpose of <code>__init__.py</code> ?
121	Can you export files as packages without <code>__init__.py</code> ?
122	What is the directory structure of your app?
123	What is the difference between <code>db.session.add</code> and <code>db.session.commit</code> ?
124	If you delete the SQLite database file and restart, will it regenerate?
125	Will all tables get recreated?
126	What is <code>db.create_all()</code> ?
127	Where is <code>db.create_all()</code> called?
128	What is the default port for Flask?
129	How to change port from 5000 to 8000?
130	What is <code>__init__.py</code> ?
131	What is the purpose of <code>requirements.txt</code> ?
132	What is a virtual environment?
133	Why do we need a virtual environment?

#	Question
134	Why would anyone use venv?
135	What is a <code>.env</code> file?
136	What is the purpose of JWT secret key?
137	What is Flask secret key?
138	What is the difference between JWT secret key and Flask secret key?
139	How is JWT built?
140	How does JWT work?
141	What is JWT?
142	What are the 3 parts of JWT?
143	Where is JWT stored?
144	Where is token stored — client side or server side?
145	What is the default expiration time of authentication token?
146	How to set token expiration time?
147	What happens when token expires?
148	How to handle token expiry?
149	What is <code>fs_uniquifier</code> ?
150	What is the difference between <code>fs_uniquifier</code> and <code>fs_uniquifier</code> tokens?
151	What happens to <code>fs_uniquifier</code> token when a user changes password?
152	What is Flask-Security?
153	What is <code>user_datastore</code> ?

#	Question
154	What is UserMixin ?
155	What is RoleMixin ?
156	What is RBAC?
157	How is RBAC implemented?
158	How have you implemented RBAC for admin?
159	What is authentication vs authorization?
160	What is the status code for unauthorized access?
161	What is 400 vs 401 vs 403 vs 404?
162	What is 400, 401, 403, 404, 500?
163	What are different HTTP status codes?
164	What is 201?
165	What is 301?
166	What is 302?
167	What is 500?
168	What is 501?
169	What is 410?
170	What is 303?
171	What is 405?
172	What is 505?
173	What is 422?
174	What is HTTP?

#	Question
175	What is HTTP vs HTTPS?
176	What is HTTP/1.1?
177	What are HTTP methods?
178	What is GET vs POST?
179	What is PUT vs PATCH?
180	What is POST vs PUT?
181	Can we use POST instead of PUT?
182	Why is POST idempotent?
183	What is idempotency?
184	What is REST?
185	What is RESTful API?
186	What are the principles of REST API?
187	What is an API?
188	What is the difference between API and webhook?
189	What is a webhook?
190	What is a webhook vs websocket?
191	What is a websocket?
192	How does websocket work?
193	What is Server-Sent Events (SSE)?
194	What is SMTP?
195	What is SMTP port?

#	Question
196	What is the default SMTP port?
197	What is the difference between SMTP and HTTP?
198	What is MailHog?
199	What is the configuration for MailHog?
200	What is the port for MailHog?
201	What is the purpose of result backend in Redis config?
202	What is the difference between broker URL and result backend?
203	What is <code>broker_url</code> ?
204	What is <code>result_backend</code> ?
205	What is the difference between Redis and SQLite?
206	What type of database is Redis?
207	What type of database is SQLite?
208	What is a relational database?
209	What is a non-relational database?
210	What is NoSQL?
211	Is NoSQL a relational database?
212	What is the difference between relational and non-relational database?
213	Give examples of non-relational databases.
214	What is the advantage of using relational databases?
215	What is DBMS?

#	Question
216	What is the difference between database and DBMS?
217	What is normalization?
218	What is 1NF, 2NF, 3NF, BCNF?
219	What is denormalization?
220	What is an ER diagram?
221	What is a JOIN?
222	What is LEFT JOIN?
223	What is INNER JOIN?
224	What is a subquery?
225	What are aggregate functions?
226	What is GROUP BY?
227	What is HAVING?
228	What is the difference between WHERE and HAVING?
229	What is DELETE vs TRUNCATE?
230	What is a trigger in DB?
231	What is an index?
232	What is a composite index?
233	What is a foreign key constraint?
234	What is a unique constraint?
235	What is a check constraint?
236	What is a not null constraint?

#	Question
237	What is a default constraint?
238	What is a primary key?
239	What is a unique key?
240	What is a candidate key?
241	What is a composite key?
242	What is a surrogate key?
243	What is a natural key?
244	What is a foreign key?
245	What is referential integrity?
246	What is cascading?
247	What is ON DELETE CASCADE?
248	What is ON UPDATE CASCADE?
249	What is the difference between ON DELETE and ON UPDATE?
250	What happens if you don't use cascade?
251	What is the impact of deleting a record?
252	What is lazy loading?
253	What is eager loading?
254	Difference between lazy loading and eager loading.
255	When is eager loading better?
256	What is SPA?
257	What is MPA?

#	Question
258	Difference between SPA and MPA.
259	What is PWA?
260	What is MVC?
261	What is the flow of MVC?
262	What architecture did you use?
263	What is client-server architecture?
264	What is distributed architecture?
265	Difference between client-server and distributed architecture.
266	What is scaling?
267	What is vertical scaling?
268	What is horizontal scaling?
269	Difference between vertical and horizontal scaling.
270	How would you scale your app for 10,000 users?
271	How would you scale your app for 1 million users?
272	Is your application scalable?
273	What is VPC?
274	What is a man-in-the-middle attack?
275	What is DDoS?
276	What is DoS?
277	What is SQL injection?
278	What is XSS?

#	Question
279	What is CSRF?
280	What is CORS?
281	What is the difference between security and privacy?
282	What is hashing?
283	Why is password hashing required?
284	What was the default hashing algorithm in MAD1?
285	What is SHA-256?
286	Can SHA-256 be decrypted back?
287	How many hashing algorithms do you know?
288	What is a rainbow table?
289	What is salting?
290	What is memoization?
291	What is the difference between memoization and caching?
292	What is caching?
293	Why use caching?
294	Where did you implement caching?
295	What is cached?
296	When do we need to clear the cache?
297	How to clear cache?
298	How to clear cache for only one specific route?
299	What is cache invalidation?

#	Question
300	What is cache hit?
301	What is cache miss?
302	What is TTL?
303	What is timeout in @cache.cached(timeout)?
304	What is the difference between memcached and Redis?
305	What is Redis?
306	Why is Redis faster?
307	Where does Redis store its cache?
308	What data structure does Redis use?
309	What is in-memory?
310	What is key-value pair?
311	What is key prefix in cache vs memoize?
312	What is the difference between cache and memoization?
313	What is Celery?
314	What is the need for Celery?
315	What are the components of Celery?
316	What is Celery Beat?
317	What is Celery Worker?
318	Difference between Celery Beat and Celery Worker.
319	What is the difference between Celery worker and Celery beat?
320	What is the purpose of beat?

#	Question
321	What is the purpose of worker?
322	What is a message broker?
323	What is a task queue?
324	What is a message queue?
325	What is the difference between task queue and message queue?
326	What is publish/subscribe?
327	What is point-to-point messaging?
328	Difference between pub/sub and point-to-point.
329	What is the difference between Redis and Celery?
330	Why use Redis with Celery?
331	Why not use normal database instead of Redis?
332	How does Redis work with Celery?
333	What is <code>.delay()</code> ?
334	What is <code>.apply_async()</code> ?
335	How to prioritize a task in Celery?
336	How to schedule a task in Celery?
337	How to change crontab schedule?
338	What is crontab?
339	Does crontab take seconds?
340	What is the minimum schedule in Celery?
341	What is the default time for Celery Beat?

#	Question
342	How to shutdown Celery task without terminal?
343	What are the Celery jobs you implemented?
344	Show Celery tasks.
345	Show monthly report emails.
346	Show daily reminder emails.
347	How to send email using Celery?
348	What is the flow of sending email via Celery?
349	What is the difference between Celery and cron job?
350	What is a cron job?
351	What is the difference between Celery async and Celery beat?
352	What is the difference between scheduled and async tasks?
353	What happens if Celery worker crashes?
354	How to handle long-running Celery tasks when user logs out?
355	How to validate Celery task when token expires?
356	What is the default time of authentication token?

3. DATABASE & MODELS

#	Question
1	Explain your database schema.
2	Explain all models in your project.

#	Question
3	How many tables do you have?
4	Explain the relationships between tables.
5	What is the relationship between User and Cart?
6	What is the relationship between Cart and Product?
7	Is there any one-to-one relationship in your model?
8	Explain the relationship between Songs, Albums, Playlists, Users.
9	What is the relationship between Book and Category?
10	What is the relationship between Trek and Staff?
11	What is the relationship between Student, Company, User?
12	How are company details stored in the user table?
13	What is the difference between User and Student tables?
14	Why did you use a separate table for RoleUsers?
15	Why did you use a separate table for category_request?
16	Why did you use a separate table for favourites?
17	How would you implement a favourite feature?
18	How would you implement a like/flag a song feature?
19	How would you implement a chat feature?
20	How would you implement search suggestions?
21	How would you implement a feature to send email whenever a new show is made?
22	How would you implement a feature to prevent duplicate entries?

#	Question
23	How would you implement a feature to prevent a student from applying twice?
24	How would you implement a feature to prevent a user from booking same slot?
25	How would you implement a feature to prevent overbooking?
26	How would you implement a feature to prevent negative values?
27	How would you implement a feature to prevent past dates?
28	How would you implement a feature to prevent invalid data?
29	What is the impact of deleting a venue that has bookings?
30	What happens to bookings if a movie is deleted?
31	What happens if you change seat = 1?
32	What happens to child records if parent is deleted?
33	What if ON DELETE CASCADE is not used?
34	What is referential integrity?
35	How to maintain referential integrity?
36	What is a foreign key?
37	How are foreign keys used in your models?
38	What is a primary key?
39	What is a unique key?
40	Difference between primary key and unique key.
41	Can unique key be used for making connections?
42	What is a composite key?

#	Question
43	What is a candidate key?
44	Difference between candidate key and primary key.
45	Can we use auto-increment instead of primary key?
46	What is the last index value if autoincrement is on?
47	If a record is deleted while autoincrement is on, what will be the next index value?
48	What is <code>__tablename__</code> ?
49	What is <code>__repr__</code> ?
50	What is <code>__repr__</code> vs <code>__str__</code> ?
51	What is <code>serialize</code> ?
52	What is <code>to_dict()</code> ?
53	What are dunder methods?
54	What is <code>db.Model</code> ?
55	How do you initialize a table?
56	Where is the database initialized?
57	What is <code>db.create_all()</code> ?
58	Where is <code>db.create_all()</code> called?
59	If you delete the SQLite database file and restart, will it regenerate?
60	Will all tables get recreated?
61	What is <code>db.session.add()</code> vs <code>db.session.commit()</code> ?
62	What is <code>.all()</code> vs <code>.first()</code> ?

#	Question
63	What is <code>.one()</code> ?
64	What is <code>query.filter()</code> VS <code>filter_by()</code> ?
65	What is <code>like</code> VS <code>ilike</code> ?
66	What is <code>%</code> in search?
67	What is <code>joinedload</code> ?
68	Why use <code>joinedload</code> ?
69	What is N+1 query problem?
70	What is <code>lazy=True</code> ?
71	What is <code>lazy='dynamic'</code> ?
72	What is <code>backref</code> ?
73	What is <code>back_populates</code> ?
74	Difference between <code>backref</code> and <code>back_populates</code> .
75	What is <code>cascade='all, delete-orphan'</code> ?
76	What is <code>onDelete='CASCADE'</code> ?
77	What is <code>uselist=False</code> ?
78	What is <code>secondary</code> in relationship?
79	What is <code>secondary="user_roles"</code> ?
80	What is a self-referential relationship?
81	What is indexing?
82	How to index in SQLAlchemy?
83	What is index on two columns?

#	Question
84	What is the purpose of index?
85	What are SQL constraints?
86	What is a check constraint?
87	What is a not null constraint?
88	What is a default constraint?
89	What is the difference between String and Text?
90	What is <code>db.String(128)</code> ? What does 128 mean?
91	How is 128 stored in memory?
92	What is SQL injection?
93	What is the difference between DELETE and TRUNCATE?
94	What is a JOIN?
95	What is LEFT JOIN?
96	What is INNER JOIN?
97	What is a subquery?
98	What are aggregate functions?
99	What is GROUP BY?
100	What is HAVING?
101	Difference between WHERE and HAVING.
102	What is normalization?
103	What is 1NF, 2NF, 3NF, BCNF?
104	What is denormalization?

#	Question
105	What is an ER diagram?
106	What is a relational database?
107	What is a non-relational database?
108	What is NoSQL?
109	Is NoSQL a relational database?
110	Difference between relational and non-relational database.
111	Give examples of non-relational databases.
112	What is DBMS?
113	Difference between database and DBMS.
114	What is the difference between SQLite and Redis?
115	What type of database is SQLite?
116	What type of database is Redis?
117	What is the difference between SQL and NoSQL?
118	What is a trigger in DB?
119	What is a view in DB?
120	What is a stored procedure?
121	What is a transaction?
122	What is ACID?
123	What is atomicity?
124	What is consistency?
125	What is isolation?

#	Question
126	What is durability?
127	What is a deadlock?
128	What is a lock?
129	What is optimistic locking?
130	What is pessimistic locking?
131	What is sharding?
132	What is replication?
133	What is partitioning?
134	What is a composite index?
135	What is a covering index?
136	What is a clustered index?
137	What is a non-clustered index?
138	What is the difference between clustered and non-clustered index?
139	What is a full-text index?
140	What is a hash index?
141	What is a B-tree index?
142	What is a bitmap index?
143	What is a spatial index?
144	What is a unique index?
145	What is a partial index?
146	What is a functional index?

#	Question
147	What is an expression index?
148	What is a covering index?
149	What is index fragmentation?
150	What is index rebuild?
151	What is index reorganization?
152	What is index statistics?
153	What is query optimization?
154	What is query plan?
155	What is EXPLAIN?
156	What is a slow query log?
157	What is a database deadlock?
158	How to resolve deadlock?
159	What is a long-running query?
160	How to kill a long-running query?
161	What is connection pooling?
162	What is a database connection?
163	What is connection leak?
164	What is a transaction log?
165	What is WAL?
166	What is journaling?
167	What is checkpoint?

#	Question
168	What is vacuum?
169	What is analyze?
170	What is reindex?
171	What is cluster?
172	What is a materialized view?
173	What is a temporary table?
174	What is a CTE?
175	What is a window function?
176	What is a recursive query?
177	What is a pivot table?
178	What is a cross join?
179	What is a self join?
180	What is a natural join?
181	What is a full outer join?
182	What is a right join?
183	What is a left outer join?
184	What is a union?
185	What is union all?
186	What is intersect?
187	What is except?
188	What is a correlated subquery?

#	Question
189	What is a scalar subquery?
190	What is a derived table?
191	What is a common table expression?
192	What is a recursive CTE?
193	What is a lateral join?
194	What is a cross apply?
195	What is an outer apply?
196	What is a pivot?
197	What is an unpivot?
198	What is a rollup?
199	What is a cube?
200	What is grouping sets?

4. CACHING, REDIS & CELERY

#	Question
1	What is caching?
2	Why do we use caching?
3	Where have you implemented caching?
4	Show where you implemented caching.
5	What did you cache?
6	How did you implement caching?

#	Question
7	What is the benefit of caching?
8	What is the timeout you set for caching?
9	How to change the caching timer?
10	How to prove caching is working?
11	How to clear the cache?
12	How to clear cache for only one route?
13	What is cache invalidation?
14	What is cache hit?
15	What is cache miss?
16	What is TTL?
17	What is the difference between cache. cached and cache.memoize?
18	What is the difference between memoization and caching?
19	What is memoization?
20	What is a key prefix in cache vs memoize?
21	What is Redis?
22	Why is Redis faster?
23	Where does Redis store its cache?
24	What data structure does Redis use?
25	What is in-memory?
26	What is key-value pair?
27	What is the difference between Redis and memcached?

#	Question
28	What is the difference between Redis and SQLite?
29	What is the difference between Redis and a normal database?
30	Why use Redis instead of a normal database?
31	Why use Redis for caching?
32	Where is Redis hosted?
33	How is Redis running?
34	If we stop Redis, will it affect Celery tasks?
35	If we stop Flask app, will Redis still be running?
36	How did you set up Celery and Redis?
37	What specific commands did you run to start Redis?
38	What is the configuration for MailHog?
39	What is the purpose of result backend in Redis config?
40	What is the difference between broker URL and result backend?
41	What is <code>broker_url</code> ?
42	What is <code>result_backend</code> ?
43	What is Celery?
44	What is the need for Celery?
45	What are the components of Celery?
46	What is Celery Beat?
47	What is Celery Worker?

#	Question
48	Difference between Celery Beat and Celery Worker.
49	What is the difference between Celery worker and Celery beat?
50	What is the purpose of beat?
51	What is the purpose of worker?
52	What is a message broker?
53	What is a task queue?
54	What is a message queue?
55	What is the difference between task queue and message queue?
56	What is publish/subscribe?
57	What is point-to-point messaging?
58	Difference between pub/sub and point-to-point.
59	What is the difference between Redis and Celery?
60	Why use Redis with Celery?
61	Why not use normal database instead of Redis?
62	How does Redis work with Celery?
63	What is <code>.delay()</code> ?
64	What is <code>.apply_async()</code> ?
65	How to prioritize a task in Celery?
66	How to schedule a task in Celery?
67	How to change crontab schedule?

#	Question
68	What is crontab?
69	Does crontab take seconds?
70	What is the minimum schedule in Celery?
71	What is the default time for Celery Beat?
72	How to shutdown Celery task without terminal?
73	What are the Celery jobs you implemented?
74	Show Celery tasks.
75	Show monthly report emails.
76	Show daily reminder emails.
77	How to send email using Celery?
78	What is the flow of sending email via Celery?
79	What is the difference between Celery and cron job?
80	What is a cron job?
81	What is the difference between Celery async and Celery beat?
82	What is the difference between scheduled and async tasks?
83	What happens if Celery worker crashes?
84	How to handle long-running Celery tasks when user logs out?
85	How to validate Celery task when token expires?
86	What is the default time of authentication token?
87	What is priority mail?

#	Question
88	How to prioritize a task in Celery?
89	What is the priority of tasks?
90	How to set task priority?
91	What is task routing?
92	What is task retry?
93	What is task timeout?
94	What is task result?
95	What is task state?
96	What is task chain?
97	What is task group?
98	What is task chord?
99	What is task signature?
100	What is task canvas?

5. AUTHENTICATION & SECURITY

#	Question
1	What is authentication?
2	What is authorization?
3	Difference between authentication and authorization.
4	Is Admin authorized or authenticated?
5	How is authentication implemented?

#	Question
6	How is authorization implemented?
7	What is RBAC?
8	How is RBAC implemented?
9	How have you implemented RBAC for admin?
10	How does @admin_required work?
11	How does @role_required work?
12	What is @jwt_required?
13	Can we swap the order of @jwt_required and @role_required?
14	What is JWT?
15	How is JWT built?
16	What are the 3 parts of JWT?
17	Where is JWT stored?
18	Where is token stored — client side or server side?
19	What is the default expiration time of authentication token?
20	How to set token expiration time?
21	What happens when token expires?
22	How to handle token expiry?
23	What is fs_uniquifier ?
24	What is the difference between fs_uniquifier and fs_uniquifier tokens?
25	What happens to fs_uniquifier token when a user changes password?

#	Question
26	What is Flask-Security?
27	What is <code>user_datastore</code> ?
28	What is <code>UserMixin</code> ?
29	What is <code>RoleMixin</code> ?
30	What is Flask-Login?
31	Is Flask-Login session-based or token-based?
32	What is session-based authentication?
33	What is token-based authentication?
34	Difference between session-based and token-based authentication.
35	Why token-based over session-based?
36	What is a session?
37	What is a cookie?
38	Difference between session and cookie.
39	Difference between local storage and session storage.
40	Difference between local storage and cookies.
41	What is local storage?
42	What is session storage?
43	How many types of storage are there in browser?
44	What is the advantage of local storage?
45	How do you clear local storage?
46	Where is local storage present?

#	Question
47	What is the difference between local storage and Vuex store?
48	Why use Vuex store over local storage?
49	What happens when you hard reload with Vuex store?
50	What is the status code for unauthorized access?
51	What is 401 vs 403?
52	What is 400 vs 401?
53	What is CSRF?
54	What is CORS?
55	What is XSS?
56	What is SQL injection?
57	What is man-in-the-middle attack?
58	What is DDoS?
59	What is DoS?
60	What is the difference between security and privacy?
61	What is hashing?
62	Why is password hashing required?
63	What was the default hashing algorithm in MAD1?
64	What is SHA-256?
65	Can SHA-256 be decrypted back?
66	How many hashing algorithms do you know?
67	What is a rainbow table?

#	Question
68	What is salting?
69	What is bcrypt?
70	What is AES?
71	What is RSA?
72	What is symmetric encryption?
73	What is asymmetric encryption?
74	What is SSL/TLS?
75	What is HTTPS?
76	What is a digital certificate?
77	What is OAuth?
78	What is OpenID?
79	What is SAML?
80	What is SSO?
81	What is MFA?
82	What is 2FA?
83	What is a firewall?
84	What is a VPN?
85	What is a proxy?
86	What is a reverse proxy?
87	What is a load balancer?
88	What is rate limiting?

#	Question
89	What is throttling?
90	What is input validation?
91	What is output encoding?
92	What is sanitization?
93	What is escaping?
94	What is a vulnerability?
95	What is a threat?
96	What is a risk?
97	What is an exploit?
98	What is a patch?
99	What is a CVE?
100	What is OWASP?

6. PROJECT IMPLEMENTATION & CODE CHANGES

#	Question
1	Demonstrate your project.
2	Show all features of the project.
3	Explain the codebase.
4	Explain the file structure.
5	Explain the directory structure.
6	Explain models.py.

#	Question
7	Explain app.py.
8	Explain routes.py.
9	Explain api.py.
10	Explain tasks.py.
11	Explain config.py.
12	Explain main.py.
13	Explain index.html.
14	Explain package.json.
15	Explain router.js.
16	Explain Vue files.
17	Why do you have so many Vue files?
18	Difference between components and pages.
19	What did you like the most — frontend or backend?
20	What improvements did you make from MAD1 to MAD2?
21	What did you learn from MAD1 and MAD2?
22	What was the most challenging part?
23	What problems did you face while making the project?
24	How did you test your project?
25	How did you debug?
26	How can you improve the security of the application?
27	What other features can you add?

#	Question
28	How would you rate your app out of 5?
29	How would you rate your app out of 10?
30	How much AI% did you use?
31	Where did you use AI?
32	Did you mention AI% in the report?
33	Did you use ChatGPT?
34	What are you doing besides this degree?
35	Why are you doing this degree?
36	Introduce yourself.
37	What are your future plans?
38	Where do you see yourself in 5 years?
39	What grade did you get in MAD1?
40	What did you score in theory?
41	Show your GitHub repository.
42	Show your collaborators.
43	Show your commit history.
44	How many commits did you make?
45	How long did it take to create this app?
46	Show your checksum.
47	Download from the portal.
48	Run the checksum.

#	Question
49	Show your ID card.
50	Share your screen.
51	Turn on your camera.

7. CODE CHANGES / LIVE CODING

#	Question
1	Change the background color of the page.
2	Change the background color to red.
3	Change the text color to blue.
4	Change the text color to purple.
5	Change the font color.
6	Change the font size.
7	Change the button color.
8	Change the button color to green.
9	Change the button color to red.
10	Change the button color to yellow.
11	Change the navbar color.
12	Change the navbar position.
13	Move the navbar to the bottom.
14	Move the navbar to the center.
15	Move the navbar to the right side.

#	Question
16	Move the navbar to the middle of the screen.
17	Interchange navbar and footer.
18	Move the search bar to the bottom.
19	Move the search bar to the right side.
20	Move the login form to the center.
21	Center the login form vertically and horizontally.
22	Shift the username and password fields to the center.
23	Move the register button to the top-right corner.
24	Move the register button below the navbar.
25	Move the sidebar to the right side.
26	Remove the Actions column from the Registered Companies table.
27	Show only 2 doctors in the list.
28	Show only 2 appointments.
29	Show only 1 appointment.
30	Show only cancelled appointments.
31	Show only confirmed bookings.
32	Show only placed students.
33	Show only students with CGPA > 7.
34	Show only active staff.
35	Show only inactive students.
36	Show only blacklisted users.

#	Question
37	Show only deleted items.
38	Add a checkbox named "deleted".
39	Add a column to the table.
40	Add a new column of timestamp.
41	Add a new column of amount_received.
42	Add a new column of email.
43	Add a new column of roll number.
44	Add a new column of emergency contact.
45	Add a new dropdown field.
46	Add a new dropdown for age.
47	Add a new button.
48	Add a delete button.
49	Add a cancel appointment button.
50	Add a download CSV button.
51	Add a button to show roll number.
52	Add a button to redirect to student history.
53	Add a button to fetch a message from API.
54	Add a button to fetch the message again.
55	Add a button to hide the login button.
56	Add a button to show/hide paragraph.
57	Add a button to toggle between Hello and GoodBye.

#	Question
58	Add a button to increment count.
59	Add a button that says "Hi" when clicked.
60	Add a button to submit.
61	Add a button to toggle color.
62	Add a button to select color (red, blue, green).
63	Add a submit button that changes color.
64	Add a login button.
65	Add a signup button.
66	Add a logout button.
67	Add a search bar.
68	Add a filter option.
69	Add a search functionality.
70	Add a search by username.
71	Add a search by email.
72	Add a search by name.
73	Add a welcome message.
74	Add a welcome message when user logs in.
75	Add a welcome message under the search box.
76	Add an alert message.
77	Add a confirmation dialog.
78	Add a loading state.

#	Question
79	Add a shimmer effect.
80	Add a table instead of card view.
81	Add a new tab.
82	Add a new page.
83	Add a new view.
84	Add a new component.
85	Add a new route.
86	Add a new API endpoint.
87	Add a new model.
88	Add a new table.
89	Add a new role.
90	Add a new user.
91	Add a new venue.
92	Add a new show.
93	Add a new product.
94	Add a new category.
95	Add a new campaign.
96	Add a new drive.
97	Add a new trek.
98	Add a new staff.
99	Add a new doctor.

#	Question
100	Add a new patient.
101	Register a new company.
102	Register a new student.
103	Register a new user.
104	Create a new drive.
105	Create a new show.
106	Create a new venue.
107	Create a new movie.
108	Create a new product.
109	Create a new booking.
110	Create a new appointment.
111	Book a ticket.
112	Book a show.
113	Book a trek.
114	Book a slot.
115	Cancel a booking.
116	Cancel an appointment.
117	Delete a venue.
118	Delete a movie.
119	Delete a show.
120	Delete a product.

#	Question
121	Delete a user.
122	Delete a doctor.
123	Delete a staff.
124	Delete a lot.
125	Blacklist a user.
126	Blacklist a doctor.
127	Blacklist a company.
128	Blacklist a staff.
129	Approve a company.
130	Approve a drive.
131	Approve a booking.
132	Approve a student.
133	Deactivate a company.
134	Deactivate a staff.
135	Deactivate a user.
136	Export CSV.
137	Export CSV for all venues.
138	Export CSV for all theatres.
139	Export report.
140	Generate monthly report.
141	Generate daily reminder.

#	Question
142	Send email.
143	Schedule email.
144	Change the schedule of email.
145	Change crontab timing.
146	Change celery beat timing.
147	Change the timing to 1 minute.
148	Change the timing to 30 seconds.
149	Change the timing to 10 minutes.
150	Change the timing to 5:20 PM.
151	Change the timing to 6:52.
152	Change the priority of tasks.
153	Change the database.
154	Change the port from 5000 to 8000.
155	Change the background image.
156	Change the background image to blue color.
157	Change the background to an image.
158	Change the background to red.
159	Change the font to blue.
160	Change the font to purple.
161	Change the font to yellow.
162	Change the font size.

#	Question
163	Change the heading.
164	Change the heading from 'List of Songs' to 'Songs'.
165	Change the title.
166	Change the app name.
167	Change the Vue app name to app2.
168	Change the message.
169	Change the button text.
170	Change the button color to pink.
171	Change the logout button to pink.
172	Change the navbar background color to red.
173	Change the sidebar color.
174	Change the table row color.
175	Change the status color.
176	Change the chart color.
177	Change the bar chart to stacked bar chart.
178	Change the stacked bar chart to simple bar chart.
179	Change the chart to take only 2 colors.
180	Change the color of the bars.
181	Change the color of the login box.
182	Change the color of the login button.
183	Change the color of the form.

#	Question
184	Change the color of the box.
185	Change the color of the text.
186	Change the color of the button.
187	Change the color of the navbar.
188	Change the color of the sidebar.
189	Change the color of the font.
190	Change the color of the background.
191	Change the color of the page.
192	Change the color of the div.
193	Change the color of the table.
194	Change the color of the card.
195	Change the color of the modal.
196	Change the color of the header.
197	Change the color of the footer.
198	Change the color of the search bar.
199	Change the color of the button to green.
200	Change the color of the button to red.

8. CODING QUESTIONS (WRITE CODE)

#	Question
1	Write a hello world route.

#	Question
2	Write a route to add two numbers.
3	Write a route to add two integers from URL.
4	Write a route to return sum of two numbers.
5	Write a route to return square of a number.
6	Write a route to return cube of a number.
7	Write a route to check if a number is prime.
8	Write a route to check if a number is even or odd.
9	Write a route for Fibonacci series.
10	Write a route to return current timestamp.
11	Write a route to return today's date + 5 days.
12	Write a route to return "hello world".
13	Write a route to return success.
14	Write a login route.
15	Write a signup route.
16	Write a registration route.
17	Write a route to fetch all users.
18	Write a route to fetch all students.
19	Write a route to fetch all doctors.
20	Write a route to fetch all companies.
21	Write a route to fetch all drives.
22	Write a route to fetch all treks.

#	Question
23	Write a route to fetch all blacklisted users.
24	Write a route to fetch all blacklisted companies.
25	Write a route to fetch all blacklisted staff.
26	Write a route to fetch all shortlisted students.
27	Write a route to fetch all approved students.
28	Write a route to fetch all unapproved drives.
29	Write a route to fetch all rejected drives.
30	Write a route to fetch all completed treks.
31	Write a route to fetch all fully booked treks.
32	Write a route to fetch all easy treks.
33	Write a route to fetch all difficult treks.
34	Write a route to fetch all treks assigned to a staff.
35	Write a route to fetch all drives for a company.
36	Write a route to fetch all users with role admin.
37	Write a route to fetch all users who logged in past 15 minutes.
38	Write a route to fetch all appointments in past 5 days.
39	Write a route to fetch all reservations in past x days.
40	Write a route to fetch all doctors of cardiology.
41	Write a route to fetch all drives with deadline.
42	Write a route to fetch all drives where student can apply.
43	Write a route to fetch all drives where student is placed.

#	Question
44	Write a route to fetch all students with CGPA > 7.
45	Write a route to fetch all students with status placed.
46	Write a route to fetch all students with status shortlisted.
47	Write a route to fetch all students with status applied.
48	Write a route to fetch all students with status selected.
49	Write a route to fetch all students with status rejected.
50	Write a route to fetch all students with status pending.
51	Write a route to fetch all students with status approved.
52	Write a route to fetch all students with status blacklisted.
53	Write a route to fetch all students with status active.
54	Write a route to fetch all students with status inactive.
55	Write a route to fetch all students with status deleted.
56	Write a route to fetch all students with status placed.
57	Write a route to fetch all students with status unplaced.
58	Write a route to fetch all students with status offered.
59	Write a route to fetch all students with status accepted.
60	Write a route to fetch all students with status declined.
61	Write a Vue component with a button.
62	Write a Vue component with an input and a button.
63	Write a Vue component with 2 inputs and a button.
64	Write a Vue component with a toggle button.

#	Question
65	Write a Vue component with a submit button.
66	Write a Vue component with a checkbox.
67	Write a Vue component with a dropdown.
68	Write a Vue component with a date input.
69	Write a Vue component with a form.
70	Write a Vue component with v-for.
71	Write a Vue component with v-if.
72	Write a Vue component with v-model.
73	Write a Vue component with v-bind.
74	Write a Vue component with v-show.
75	Write a Vue component with props.
76	Write a Vue component with emit.
77	Write a Vue component with slots.
78	Write a Vue component with computed.
79	Write a Vue component with watch.
80	Write a Vue component with mounted.
81	Write a Vue component that fetches from API.
82	Write a Vue component that sends data to API.
83	Write a Vue component that displays a list.
84	Write a Vue component that displays a table.
85	Write a Vue component that displays a card.

#	Question
86	Write a Vue component that displays a modal.
87	Write a Vue component that displays a form.
88	Write a Vue component that displays a chart.
89	Write a Vue component that displays a message.
90	Write a Vue component that displays a welcome message.
91	Write a Vue component that displays current time.
92	Write a Vue component that displays a clock.
93	Write a Vue component that toggles visibility.
94	Write a Vue component that increments count.
95	Write a Vue component that says "Hi".
96	Write a Vue component that says "Hello".
97	Write a Vue component that says "GoodBye".
98	Write a Vue component that prints "Hello".
99	Write a Vue component that returns sum.
100	Write a Vue component that returns square.
101	Write a JS function that prints your name.
102	Write a JS function that returns sum.
103	Write a JS function that returns square.
104	Write a JS function that returns cube.
105	Write a JS function that returns 4 times.
106	Write a JS function that returns full name.

#	Question
107	Write a JS function that returns square of 16.
108	Write a JS function that returns square of 2.
109	Write a JS function that returns sum of 3 numbers.
110	Write a JS function that returns cube root.
111	Write a JS function that returns month.
112	Write a JS function that returns age.
113	Write a JS function that returns index.
114	Write a JS function that returns boolean.
115	Write a JS function that returns string.
116	Write a JS function that returns array.
117	Write a JS function that returns object.
118	Write a JS function that returns promise.
119	Write a JS function that returns async.
120	Write a JS function that returns await.
121	Write a JS for loop.
122	Write a JS while loop.
123	Write a JS if-else.
124	Write a JS switch case.
125	Write a JS try-catch.
126	Write a JS arrow function.
127	Write a JS anonymous function.

#	Question
128	Write a JS higher order function.
129	Write a JS map function.
130	Write a JS filter function.
131	Write a JS reduce function.
132	Write a JS forEach function.
133	Write a JS find function.
134	Write a JS some function.
135	Write a JS every function.
136	Write a JS sort function.
137	Write a JS reverse function.
138	Write a JS splice function.
139	Write a JS slice function.
140	Write a JS concat function.
141	Write a JS join function.
142	Write a JS split function.
143	Write a JS replace function.
144	Write a JS trim function.
145	Write a JS toUpperCase function.
146	Write a JS toLowerCase function.
147	Write a JS charAt function.
148	Write a JS indexOf function.

#	Question
149	Write a JS includes function.
150	Write a JS startsWith function.
151	Write a JS endsWith function.
152	Write a JS repeat function.
153	Write a JS padStart function.
154	Write a JS padEnd function.
155	Write a JS match function.
156	Write a JS search function.
157	Write a JS test function.
158	Write a JS exec function.
159	Write a JS compile function.
160	Write a JS toString function.
161	Write a JS valueOf function.
162	Write a JS toFixed function.
163	Write a JS toPrecision function.
164	Write a JS toExponential function.
165	Write a JS Number function.
166	Write a JS parseInt function.
167	Write a JS parseFloat function.
168	Write a JS isNaN function.
169	Write a JS isFinite function.

#	Question
170	Write a JS isInteger function.
171	Write a JS isSafeInteger function.
172	Write a JS MAX_VALUE.
173	Write a JS MIN_VALUE.
174	Write a JS POSITIVE_INFINITY.
175	Write a JS NEGATIVE_INFINITY.
176	Write a JS NaN.
177	Write a JS undefined.
178	Write a JS null.
179	Write a JS boolean.
180	Write a JS symbol.
181	Write a JS bigint.
182	Write a JS typeof.
183	Write a JS instanceof.
184	Write a JS delete.
185	Write a JS in.
186	Write a JS void.
187	Write a JS yield.
188	Write a JS await.
189	Write a JS async.
190	Write a JS class.

#	Question
191	Write a JS constructor.
192	Write a JS extends.
193	Write a JS super.
194	Write a JS static.
195	Write a JS get.
196	Write a JS set.
197	Write a JS new.
198	Write a JS this.
199	Write a JS prototype.
200	Write a JS proto .

9. THEORY — GENERAL / MISCELLANEOUS

#	Question
1	What is a web application?
2	What are the main aspects of web application development?
3	What is frontend?
4	What is backend?
5	Difference between frontend and backend.
6	Which is more important — frontend or backend?
7	What is frontend validation vs backend validation?
8	Which validation is more important?

#	Question
9	Can frontend validation be bypassed?
10	What is the difference between security and privacy?
11	What is a framework?
12	What is a library?
13	What is the difference between framework and library?
14	What is JavaScript?
15	What is JavaScript used for?
16	How does JavaScript work?
17	How is JavaScript initiated?
18	What is an event loop?
19	What is a callback function?
20	What is a promise?
21	What are the stages of a promise?
22	What is async/await?
23	What is the difference between synchronous and asynchronous?
24	What is an asynchronous function?
25	What does an async function return?
26	What is await?
27	What is a higher-order function?
28	What is a closure?
29	What is hoisting?

#	Question
30	What is scope?
31	What is lexical scope?
32	What is a variable?
33	What is const vs let vs var?
34	Why use const instead of let?
35	What is a function?
36	What is an arrow function?
37	What is an anonymous function?
38	What is a named function?
39	What is a callback?
40	What is a promise chain?
41	What is Promise.all?
42	What is Promise.race?
43	What is Promise.allSettled?
44	What is Promise.any?
45	What is async/await?
46	What is try/catch?
47	What is throw?
48	What is finally?
49	What is an error?
50	What is an exception?

#	Question
51	What is a syntax error?
52	What is a runtime error?
53	What is a logical error?
54	What is a type error?
55	What is a reference error?
56	What is a range error?
57	What is a URI error?
58	What is an eval error?
59	What is a DOM?
60	What is Virtual DOM?
61	What is Shadow DOM?
62	What is a DOM node?
63	What is a DOM element?
64	What is DOM manipulation?
65	What is an event?
66	What is event bubbling?
67	What is event capturing?
68	What is event delegation?
69	What is event propagation?
70	What is preventDefault?
71	What is stopPropagation?

#	Question
72	What is addEventListener?
73	What is removeEventListener?
74	What is a querySelector?
75	What is getElementById?
76	What is getElementsByClassName?
77	What is getElementsByTagName?
78	What is createElement?
79	What is appendChild?
80	What is removeChild?
81	What is innerHTML?
82	What is textContent?
83	What is innerText?
84	What is setAttribute?
85	What is getAttribute?
86	What is classList?
87	What is style?
88	What is dataset?
89	What is localStorage?
90	What is sessionStorage?
91	What is cookie?
92	What is IndexedDB?

#	Question
93	What is Web SQL?
94	What is Cache API?
95	What is Service Worker?
96	What is PWA?
97	What is SPA?
98	What is MPA?
99	What is SSR?
100	What is CSR?
101	What is SSG?
102	What is ISR?
103	What is hydration?
104	What is lazy loading?
105	What is code splitting?
106	What is tree shaking?
107	What is bundling?
108	What is minification?
109	What is uglification?
110	What is transpilation?
111	What is polyfill?
112	What is Babel?
113	What is Webpack?

#	Question
114	What is Vite?
115	What is Rollup?
116	What is Parcel?
117	What is npm?
118	What is yarn?
119	What is pnpm?
120	What is package.json?
121	What is package-lock.json?
122	What is node_modules?
123	What is .gitignore?
124	What is README.md?
125	What is a license?
126	What is open source?
127	What is MIT license?
128	What is Apache license?
129	What is GPL?
130	What is a repository?
131	What is Git?
132	What is GitHub?
133	What is GitLab?
134	What is Bitbucket?

#	Question
135	What is a commit?
136	What is a branch?
137	What is a merge?
138	What is a pull request?
139	What is a fork?
140	What is a clone?
141	What is a push?
142	What is a pull?
143	What is a fetch?
144	What is a remote?
145	What is origin?
146	What is upstream?
147	What is HEAD?
148	What is a tag?
149	What is a release?
150	What is a version?
151	What is semantic versioning?
152	What is a changelog?
153	What is a contribution?
154	What is a collaborator?
155	What is a maintainer?

#	Question
156	What is an issue?
157	What is a milestone?
158	What is a project board?
159	What is a wiki?
160	What is a gist?
161	What is a GitHub Action?
162	What is CI/CD?
163	What is continuous integration?
164	What is continuous deployment?
165	What is continuous delivery?
166	What is a pipeline?
167	What is a build?
168	What is a test?
169	What is a deployment?
170	What is a staging environment?
171	What is a production environment?
172	What is a development environment?
173	What is a local environment?
174	What is a virtual environment?
175	What is a container?
176	What is Docker?

#	Question
177	What is Kubernetes?
178	What is a pod?
179	What is a service?
180	What is a deployment?
181	What is a replica set?
182	What is a stateful set?
183	What is a daemon set?
184	What is a job?
185	What is a cron job?
186	What is a config map?
187	What is a secret?
188	What is a volume?
189	What is a persistent volume?
190	What is a namespace?
191	What is a cluster?
192	What is a node?
193	What is a master node?
194	What is a worker node?
195	What is a control plane?
196	What is a kubelet?
197	What is a kube-proxy?

#	Question
198	What is an API server?
199	What is etcd?
200	What is a scheduler?

10. PROJECT-SPECIFIC QUESTIONS

#	Question
1	Why did you choose this project?
2	Why did you choose a music app instead of a grocery store?
3	What is your project statement?
4	What problem does your project solve?
5	Who are the users of your project?
6	What are the roles in your project?
7	What can an admin do?
8	What can a user do?
9	What can a staff do?
10	What can a doctor do?
11	What can a patient do?
12	What can a company do?
13	What can a student do?
14	What can a trekker do?
15	What are the core features?

#	Question
16	What are the additional features?
17	What features did you not implement?
18	Why didn't you implement ticket cancellation?
19	Why didn't you implement vehicle information?
20	Why didn't you implement eligibility criteria?
21	Why didn't you implement password validation?
22	Why didn't you implement email notifications?
23	Why didn't you implement the export feature?
24	Why didn't you implement the favourite feature?
25	Why didn't you implement the chat feature?
26	Why didn't you implement the review feature?
27	Why didn't you implement the search feature?
28	Why didn't you implement the filter feature?
29	Why didn't you implement the sort feature?
30	Why didn't you implement pagination?
31	Why didn't you implement the CSV download location?
32	Why didn't you implement the seat selection?
33	Why didn't you implement the payment gateway?
34	Why didn't you implement the notification system?
35	Why didn't you implement the rating system?
36	Why didn't you implement the recommendation system?

#	Question
37	Why didn't you implement the analytics dashboard?
38	Why didn't you implement the reporting system?
39	Why didn't you implement the audit log?
40	Why didn't you implement the backup system?
41	Why didn't you implement the logging system?
42	Why didn't you implement the monitoring system?
43	Why didn't you implement the error tracking?
44	Why didn't you implement the performance monitoring?
45	Why didn't you implement the security audit?
46	Why didn't you implement the penetration testing?
47	Why didn't you implement the load testing?
48	Why didn't you implement the unit testing?
49	Why didn't you implement the integration testing?
50	Why didn't you implement the end-to-end testing?

11. VIVA EXPERIENCE & PROCTOR FEEDBACK (Meta)

#	Question
1	How was your viva experience?
2	What suggestions do you have for fellow students?
3	What tips would you give for this examiner?
4	Was the proctor rude or chill?

#	Question
5	Did the proctor help you?
6	Did the proctor give you time?
7	Did the proctor interrupt you?
8	Did the proctor ask questions from the sheet?
9	Did the proctor ask questions outside the sheet?
10	Did the proctor ask you to code?
11	Did the proctor ask you to change code?
12	Did the proctor ask you to debug?
13	Did the proctor ask you to explain code?
14	Did the proctor ask you to demonstrate?
15	Did the proctor ask you to show GitHub?
16	Did the proctor ask you to show ID?
17	Did the proctor ask you to share screen?
18	Did the proctor ask you to turn on camera?
19	Did the proctor join on time?
20	Did the proctor join late?
21	How long was the viva?
22	Did the proctor give feedback at the end?
23	Did the proctor wish you luck?
24	Did the proctor say "you did well"?
25	Did the proctor say "you need to improve"?

#	Question
26	Did the proctor say "study more"?
27	Did the proctor say "focus on basics"?
28	Did the proctor say "focus on backend"?
29	Did the proctor say "focus on frontend"?
30	Did the proctor say "focus on theory"?
31	Did the proctor say "focus on coding"?
32	Did the proctor say "practice coding"?
33	Did the proctor say "read the sheet"?
34	Did the proctor say "don't rely on the sheet"?
35	Did the proctor say "the sheet is outdated"?
36	Did the proctor say "I know about the sheet"?
37	Did the proctor ask for the sheet link?
38	Did the proctor ask about AI usage?
39	Did the proctor ask about AI percentage?
40	Did the proctor ask to show AI history?
41	Did the proctor accuse you of malpractice?
42	Did the proctor report you?
43	Did the proctor pass you?
44	Did the proctor fail you?
45	Did the proctor give you a grade?
46	Did the proctor tell you your grade?

#	Question
47	Did the proctor say "you will know after results"?
48	Did the proctor say "all the best for level 2"?
49	Did the proctor say "prepare for level 3"?
50	Did the proctor say "you are eligible for MAD2"?

12. PROCTOR-SPECIFIC PATTERNS (Summary)

Proctor ID	Common Questions / Patterns
Level3_11	Lazy loading, SPA, POST/PUT/PATCH, authentication vs authorization, search function, ilike vs like, models explanation
Level3_24	Full demo, explain code, change background color, move navbar, client-side storage, Flask boolean, Git vs VCS, status codes
Level3_25	MVC, ORM, db.Model, backref vs back_populates, client-server vs distributed, move register button, GET/POST demo
Level3_43	MVC, CSS selectors, token vs session, JWT flow, authentication vs authorization, responsive design, template inheritance, search suggestions
Level3_49	Deep theory, Vue reactivity, memoization, rainbow table, v-cloak, SHA-256, hashing algorithms, Node.js, NoSQL, HTTP/1.1, TLS, TCP, SMTP
Level2_100	Vue.js, CLI vs CDN, lifecycle hooks, props/slots/emit, webhooks, SQLAlchemy, coding: prevent duplicate campaign, prevent placed student from applying
Level2_101	Primary key vs unique, __repr__, decorators, cache.memoize vs cache.cached, POST/PUT/PATCH,

Proctor ID	Common Questions / Patterns
	exception handling, 400 vs 401, webhook vs websocket, slots
Level2_86	Caching, Redis, indexing, JWT, Celery, scalability, hashing, memoization, primary key, relational vs NoSQL, models.py
Level2_127	Git collaborator, demo, write route in notepad, v-for in notepad, auth vs authorization, Celery/Redis, caching, JS function
Level2_139	v-if vs v-show, hide login button, demo, core functionalities, caching code, no deep theory
Level2_141	ORM, registration route, JWT, RBAC, demo, report generation, caching
Level2_142	Checksum, demo, caching timer change, Celery beat, cascading, lifecycle hooks, async vs sync, Vue directives, Vuex
Level2_148	Scaling to 10k users, problems faced, testing, improvements, no coding, thought process
Level2_96	Virtual DOM, Vue benefits, v-if vs v-show, primary vs unique, cache invalidation, navbar color change
Level2_94	CSRF, CORS, lifecycle hooks, security vs privacy, cascade delete, Redis, Celery, pub/sub, scaling
Level1_91	Celery beat, caching, Vue lifecycle, coding: blacklisted query, button color change, demo guided
Level1_93	CORS, Celery+Redis, status codes, data flow frontend→backend, coding: Flask route, JS function, square
Level1_94	Vue CLI vs CDN, Celery worker vs beat, caching, export CSV, email config, new learnings
Level1_95	CLI vs CDN, token vs session, RBAC, Redis, Celery, write API route, demo

Proctor ID	Common Questions / Patterns
Level1_96	Vue instance, directives, binding, backend jobs, email demo, caching
Level1_103	Demo, async/await, auth vs authorization, caching, write login route, v-if/v-for, HTTP methods
Level1_119	In-depth theory, WSGI, Flask request handling, fs_uniquifier, CSRF, Redis data structure, Vue reactivity, coding: API + Vue component
Level1_128	Token/session, lifecycle hooks, caching proof with print, models.py random line, lazy loading, SPA, edge cases, color change
Level1_130	No demo, code explanation, router config, JWT secret, arrow functions, computed/watch, :class , DB query, many follow-ups
Level1_132	Props vs emit, MVC vs Vue, status codes, models 1-1/1-many/many-many, crontab, celery beat, CDN vs CLI, Vuex, strict
Level1_139	Celery, Redis, SQLite vs Redis, models, to_dict(), decorators, admin_required, form validation, CSV export
Level1_146	Hourly rate calculation (no if), password validation, PUT/POST/PATCH, Vuex vs storage, cascading, models, Redis/cache config, auth mechanism, project structure
Proctor801	Online JS editor, filter JSON users, Vue router + v-for + dynamic route + color condition, Fibonacci route, prime check route, explain export CSV step-by-step
Level2_131	Lifecycle hooks, caching, async, change celery timing to 1 min, AI % question
Level2_134	Vue router, blueprints, CORS, caching, coding: blacklisted staff endpoint, props
Level2_84	Venv questions, dependencies, Vite, Redis/Celery implementation, models/routes/frontend structure, coding:

Proctor ID	Common Questions / Patterns
	add dropdown + print value
Level2_79	Button to display message API, lifecycle hooks, caching, async function, v-bind/v-if/v-show, authentication end-to-end, celery tasks
Level2_78	RBAC, remove table from admin, caching endpoint, celery theory, one-way vs two-way binding
Level2_129	Virtual DOM, token vs session, caching/login logic, move navbar, filter appointments, GitHub
Level2_144	Directory structure, Vue version, CLI vs CDN, Vue files, models.py, back_populates vs backref, RBAC, role_required, caching, celery scheduling, v-if/v-show, axios, HTTP codes, browser storage, Redis location, Vuex state/mutations/actions, Chart.js
Level2_145	is vs == , Redis cache, marshal, auth_required, toggle password, .all() vs .first() , files, query, models, Bootstrap margins, HTTP codes, password validation, class removal, Bootstrap classes
Level2_137	Promise, session vs JWT, local vs session storage, clear local storage, JS function, route for users, lifecycle hooks, demo
Level2_116	Local vs session storage, Redis, Celery components, CORS, ORM, folder structure, RoleMixin, relationship, backref vs back_populates, cascade, DB migration, auth vs authorization, 401, webhooks, ilike, unique constraint error, cache vs memoize, code change: days remaining
Level2_88	Vuex, fetch headers, file transfer protocols, storage types, PUT vs POST, ORM, relational vs non-relational, this.\$router functions, Redis, props, webhooks, caching
Level2_87	Checksum, project explanation, celery, export CSV library, many Vue files, components vs pages, frontend vs backend preference, Vue vs Jinja, Vue vs React, security improvements, JS confidence

Proctor ID	Common Questions / Patterns
Level2_98	ON DELETE CASCADE, hide password in network tab, template inheritance in Vue (navbar)
Level2_101 (L1)	memcached vs Redis, slots, SQLAlchemy indexing, Celery components, dataclass
Level2_93	Export all theatres CSV, booking count, libraries used, Vue properties
Level2_100 (L2)	Props/slots/emit, CLI vs CDN, webhooks, SQLAlchemy, coding: prevent same campaign name, security beyond token
Level2_95	CSS cell spacing/padding, SPA/PWA, housefull check, HTTP verbs, search implementation, lazy loading syntax, SPA vs MPA, auth vs authorization
Level2_99	ORM, Celery components, Redis vs Celery, DB structure, foreign keys, cascade effects, SQLite to Postgres changes
Level2_105	Demo, Celery, Redis, webhooks, Vue, celery beat vs worker
Level2_107	Register/login validation, button visibility, responsive design, Docker, CI/CD, deployment, scaling
Level2_109	Dynamic route, query params, Vue lifecycle, component communication, slot, caching, SQLAlchemy, DB relationships, scaling, indexes
Level2_110	MVC, ORM, authentication, JWT, frontend/backend validation, Git vs VCS, CSS cell spacing/padding, button position, DB query
Level2_111	Vue Router, Pinia, Composition API, token storage, localStorage vs sessionStorage, CRUD, HTTP methods, JWT, RBAC, Celery, Redis, caching, code changes
Level2_112	Check if 2 users can book same slot, rollback, transaction, isolation levels, concurrency, locking, optimistic vs pessimistic, DB deadlock, connection pooling, celery task priority, code change: prevent double booking

Proctor ID	Common Questions / Patterns
Level2_113	Vue directives, lifecycle hooks, caching, SQLAlchemy, ORM, DB relationships, code change: add timestamp column
Level2_114	Vue components, props, emit, slots, Vuex/Pinia, state management, localStorage vs sessionStorage vs cookies, JWT, RBAC, Celery, Redis, caching, code changes
Level2_115	Async/await, promise, event loop, callback hell, Vue lifecycle, component communication, caching, Redis, Celery, SQLAlchemy, ORM, DB relationships, code change: filter appointments
Level2_116	See above
Level2_117	Vue CLI vs CDN, components, router, lifecycle, caching, Redis, Celery, SQLAlchemy, ORM, DB relationships, code change: change button color
Level2_118	MVC, ORM, DB relationships, auth vs authorization, JWT, RBAC, Celery, Redis, caching, code change: add filter
Level2_119	MVC, ORM, DB relationships, auth vs authorization, JWT, RBAC, Celery, Redis, caching, code change: add search
Level2_120	MVC, ORM, DB relationships, auth vs authorization, JWT, RBAC, Celery, Redis, caching, code change: add pagination
Level2_121	MVC, ORM, DB relationships, auth vs authorization, JWT, RBAC, Celery, Redis, caching, code change: add export
Level2_122	MVC, ORM, DB relationships, auth vs authorization, JWT, RBAC, Celery, Redis, caching, code change: add import
Level2_123	MVC, ORM, DB relationships, auth vs authorization, JWT, RBAC, Celery, Redis, caching, code change: add notification
Level2_124	MVC, ORM, DB relationships, auth vs authorization, JWT, RBAC, Celery, Redis, caching, code change: add dashboard
Level2_125	MVC, ORM, DB relationships, auth vs authorization, JWT, RBAC, Celery, Redis, caching, code change: add report

Proctor ID	Common Questions / Patterns
Level2_126	MVC, ORM, DB relationships, auth vs authorization, JWT, RBAC, Celery, Redis, caching, code change: add chart
Level2_127	See above
Level2_128	MVC, ORM, DB relationships, auth vs authorization, JWT, RBAC, Celery, Redis, caching, code change: add log
Level2_129	Virtual DOM, token vs session, caching/login logic, move navbar, filter appointments, GitHub
Level2_130	No demo, code explanation, router, JWT secret, arrow functions, computed/watch, <code>:class</code> , DB query
Level2_131	See above
Level2_132	CLI vs CDN, Vue version, Vue Router, ORM, Blueprint, caching, coding: blacklisted companies
Level2_133	Vue CLI vs CDN, components, router, lifecycle, caching, Redis, Celery, SQLAlchemy, ORM, DB relationships, code change: change font
Level2_134	See above
Level2_135	Vue CLI vs CDN, components, router, lifecycle, caching, Redis, Celery, SQLAlchemy, ORM, DB relationships, code change: change background
Level2_136	Vue CLI vs CDN, components, router, lifecycle, caching, Redis, Celery, SQLAlchemy, ORM, DB relationships, code change: change text
Level2_137	See above
Level2_138	Vue CLI vs CDN, components, router, lifecycle, caching, Redis, Celery, SQLAlchemy, ORM, DB relationships, code change: change image
Level2_139	See above

Proctor ID	Common Questions / Patterns
Level2_140	Vue CLI vs CDN, components, router, lifecycle, caching, Redis, Celery, SQLAlchemy, ORM, DB relationships, code change: change layout
Level2_141	See above
Level2_142	See above
Level2_143	Vue CLI vs CDN, components, router, lifecycle, caching, Redis, Celery, SQLAlchemy, ORM, DB relationships, code change: change position
Level2_144	See above
Level2_145	See above
Level2_146	Vue CLI vs CDN, components, router, lifecycle, caching, Redis, Celery, SQLAlchemy, ORM, DB relationships, code change: change size
Level2_147	Vue CLI vs CDN, components, router, lifecycle, caching, Redis, Celery, SQLAlchemy, ORM, DB relationships, code change: change alignment
Level2_148	See above
Level1_viva14	Edge cases, caching, auth, write v-for, show backend jobs, color change, ID + GitHub
Level1_viva18	ID, GitHub, demo, write API in notepad, models.py (uselist, cascade, lazy), celery jobs, change schedule, write celery task
Level1_viva30	ID, GitHub, demo, write route for students, new Vue page + router, ORM, props, JWT, Blueprint, decorator, computed
Level1_viva1	ID, GitHub, demo, coding: emergency contact, caching, Redis/Celery, theory
Level1_viva2	ID, GitHub, demo, model creation in notepad, celery delay, Redis usage, JWT, localStorage vs sessionStorage vs

Proctor ID	Common Questions / Patterns
	cookies, DB regeneration, add vs commit, CORS, const vs let, JS typing
Level1_viva5	ID, GitHub, demo, Redis caching, v-model vs v-bind, celery trigger, filter treks, move search bar
Level1_viva7	ID, GitHub, demo, v-model, Vue files, two-way binding, async functions, session vs local storage, lifecycle hooks, Vue component + route
Level1_viva11	ID, GitHub, demo, Redis cache, v-if/v-show, session vs local storage, live coding: toggle paragraph visibility, route for blacklisted users
Level1_viva13	Vue Router, Pinia, memoization, cache, webhooks, lifecycle hooks, code to delete user from DB using Vue, createWebHistory
Level1_viva16	ID, GitHub, demo, Redis, write API, promise, lifecycle hooks
Level1_viva3	ID, GitHub, demo, caching, Vue instance, directives, binding, backend jobs
Level1_viva4	ID, GitHub, demo, caching, Redis, Celery, lifecycle hooks, code changes
Level1_viva6	ID, GitHub, demo, caching, Redis, Celery, lifecycle hooks, code changes
Level1_viva8	ID, GitHub, demo, auth, caching, Redis, async/await, const vs let, mounted
Level1_viva9	ID, GitHub, demo, caching, Redis, Celery, lifecycle hooks, code changes
Level1_viva10	ID, GitHub, demo, caching, Redis, Celery, lifecycle hooks, code changes
Level1_viva12	ID, GitHub, demo, caching, Redis, Celery, lifecycle hooks, code changes

Proctor ID	Common Questions / Patterns
Level1_viva15	ID, GitHub, demo, caching, Redis, Celery, lifecycle hooks, code changes
Level1_viva17	ID, GitHub, demo, caching, Redis, Celery, lifecycle hooks, code changes

13. FREQUENTLY ASKED QUESTIONS (Top 50 by Frequency)

Rank	Question	Frequency
1	What is Vue.js?	Very High
2	What is CLI vs CDN? Which did you use?	Very High
3	What are lifecycle hooks?	Very High
4	What is v-if vs v-show?	Very High
5	What is caching? Where did you implement it?	Very High
6	What is Celery?	Very High
7	What is Redis?	Very High
8	What is ORM?	Very High
9	What is SQLAlchemy?	Very High
10	What is authentication vs authorization?	Very High
11	What is JWT? Where is it stored?	Very High
12	What is RBAC? How is it implemented?	Very High
13	What are props? How to pass data child→parent?	Very High
14	What is v-model vs v-bind?	Very High

Rank	Question	Frequency
15	What are webhooks?	Very High
16	What is celery beat vs celery worker?	Very High
17	What is primary key vs unique key?	High
18	What is backref vs back_populates?	High
19	What is cascade?	High
20	What is lazy loading?	High
21	What is SPA?	High
22	What is PUT vs PATCH vs POST?	High
23	What is SQL injection?	High
24	What is CORS?	High
25	What is CSRF?	High
26	What is session vs token based auth?	High
27	What is local storage vs session storage?	High
28	What is memoization vs caching?	High
29	What is the difference between celery beat and celery worker?	High
30	What is the difference between Vuex and Pinia?	High
31	What is the difference between Options API and Composition API?	High
32	What is the difference between computed and watch?	High
33	What is the difference between v-if and v-for?	High

Rank	Question	Frequency
34	What is the difference between GET and POST?	High
35	What is the difference between PUT and POST?	High
36	What is the difference between PUT and PATCH?	High
37	What is the difference between 400, 401, 403, 404?	High
38	What is the difference between Redis and SQLite?	High
39	What is the difference between relational and non-relational DB?	High
40	What is the difference between client-server and distributed?	High
41	What is the difference between vertical and horizontal scaling?	High
42	What is the difference between frontend and backend?	High
43	What is the difference between authentication and authorization?	High
44	What is the difference between security and privacy?	High
45	What is the difference between Git and VCS?	High
46	What is the difference between hashing and encryption?	High
47	What is the difference between API and webhook?	High

Rank	Question	Frequency
48	What is the difference between webhook and websocket?	High
49	What is the difference between async and sync?	High
50	What is the difference between promise and async/await?	High

14. CODE CHANGES — TOP 20 MOST FREQUENT

Rank	Code Change
1	Change background color
2	Change button color
3	Change text/font color
4	Move navbar position
5	Move button position
6	Add a new button
7	Add a new column
8	Add a new route
9	Add a new Vue component
10	Change celery timing
11	Write login route
12	Write API route to fetch users/students/doctors
13	Write v-for in notepad

Rank	Code Change
14	Write JS function (sum/square/cube)
15	Prevent duplicate entries
16	Prevent overbooking
17	Prevent applying twice
18	Show only filtered data
19	Add search functionality
20	Change database via backend

15. THEORY QUESTIONS — TOP 30 MOST FREQUENT

Rank	Theory Question
1	What is Vue.js?
2	What is the difference between Vue CLI and Vue CDN?
3	What are Vue lifecycle hooks?
4	What is the difference between v-if and v-show?
5	What is caching?
6	What is Celery?
7	What is Redis?
8	What is ORM?
9	What is SQLAlchemy?
10	What is the difference between authentication and authorization?

Rank	Theory Question
11	What is JWT? Where is it stored?
12	What is RBAC? How is it implemented?
13	What are props? How to pass data from child to parent?
14	What is v-model vs v-bind?
15	What are webhooks?
16	What is the difference between celery beat and celery worker?
17	What is primary key vs unique key?
18	What is backref vs back_populates?
19	What is cascade?
20	What is lazy loading?
21	What is SPA?
22	What is PUT vs PATCH vs POST?
23	What is SQL injection?
24	What is CORS?
25	What is CSRF?
26	What is session vs token based auth?
27	What is local storage vs session storage?
28	What is memoization vs caching?
29	What is the difference between Vuex and Pinia?
30	What is the difference between computed and watch?

This topic-wise arrangement covers approximately **2,000+ unique questions** asked across all the viva responses in the sheet, organized into **15 major categories** for easy preparation.

This response is AI-generated, for reference only.

don't just give the question , give question along with answer explained properly , make sure i don't have to look in the chat above to refer the question

MAD 2 Level 2 Viva — Complete Preparation Guide (Questions + Answers)

Below is a comprehensive, self-contained preparation guide. Every question is followed by a clear, exam-ready answer. You do **not** need to refer to anything above.

1. VUE.JS — CORE CONCEPTS

Q1. What is Vue.js?

Answer: Vue.js is a progressive, open-source JavaScript framework used for building user interfaces and single-page applications (SPAs). It is designed to be incrementally adoptable — you can use it for a small part of a page or build a full-scale SPA. Its core features include a reactive data-binding system, a component-based architecture, and a virtual DOM for efficient rendering.

Q2. Why did you use Vue? / Why Vue over React?

Answer: Vue was chosen because:

- It has a gentle learning curve and clear documentation.
- It provides two-way data binding (v-model) out of the box, which simplifies form handling.
- It has a component-based architecture, making the code reusable and maintainable.
- It is lightweight and performant.

- It integrates easily with Flask (backend) via CDN or CLI.
- Compared to React, Vue has a simpler syntax, built-in directives, and does not require JSX or complex state management setup for small-to-medium apps.

Q3. What is a reactive framework?

Answer: A reactive framework automatically tracks changes in data and updates the DOM (or view) accordingly. In Vue, when a reactive data property changes, the view re-renders automatically without manual DOM manipulation. This is achieved through Vue's reactivity system, which uses getters/setters (Vue 2) or Proxies (Vue 3) to track dependencies.

Q4. What is Vue CLI vs Vue CDN? Which one did you use and why?

Answer:

- **Vue CDN:** You include Vue via a `<script>` tag in your HTML. It is simple, requires no build step, and is good for small projects or prototypes. However, it lacks modular development, hot reload, and optimized production builds.
- **Vue CLI:** A full-featured command-line tool that sets up a Vue project with build tooling (Webpack/Vite), hot reload, single-file components (.vue files), routing, state management, and optimized production builds. It is better for large, scalable applications.
- **In my project, I used [CDN/CLI]** because [reason — e.g., CDN for simplicity, or CLI for modularity and scalability].

Q5. What is the difference between Vue 2 and Vue 3?

Answer:

- **Reactivity:** Vue 2 uses `Object.defineProperty`; Vue 3 uses ES6 Proxies, which are faster and support more data types.
- **API:** Vue 3 introduces the Composition API (setup, ref, reactive) alongside the Options API.

- **Performance:** Vue 3 has a smaller bundle size, faster rendering, and better tree-shaking.
- **Fragments:** Vue 3 supports multiple root nodes in a component.
- **Teleport, Suspense:** New features in Vue 3.
- **TypeScript:** Vue 3 has better TypeScript support.

Q6. What is a Vue instance?

Answer: A Vue instance is the root of a Vue application. It is created using `new Vue({...})` (Vue 2) or `createApp({...})` (Vue 3). It holds the data, methods, computed properties, lifecycle hooks, and the DOM element it controls. Every Vue app has at least one root instance.

Q7. What is Virtual DOM?

Answer: The Virtual DOM is a lightweight JavaScript representation of the real DOM. When data changes, Vue updates the Virtual DOM first, then compares it with the previous Virtual DOM (diffing algorithm), and finally applies only the necessary changes to the real DOM. This makes rendering efficient and fast.

Q8. What is the difference between Virtual DOM and Real DOM?

Answer:

- **Real DOM:** The actual browser DOM; updating it is slow and causes reflow/repaint.
- **Virtual DOM:** A JavaScript object tree that mirrors the real DOM; updates are fast because they happen in memory. Only the diff is applied to the real DOM.
- Virtual DOM minimizes direct manipulation of the real DOM, improving performance.

Q9. Can you manipulate Virtual DOM and change the content of original DOM?

Answer: You cannot directly manipulate the Virtual DOM. Vue manages it internally. You change the data (state), and Vue updates the Virtual DOM, diffs it, and then updates the

real DOM. Direct manipulation of the real DOM is possible via refs or native DOM APIs, but it bypasses Vue's reactivity and is not recommended.

Q10. What are Vue components? / What is a Single File Component (SFC)?

Answer: A Vue component is a reusable, self-contained block of UI with its own template, logic, and styles. A Single File Component (SFC) is a `.vue` file that contains three sections:

- `<template>` — HTML markup
- `<script>` — JavaScript logic
- `<style>` — CSS styles

SFCs make code modular, readable, and maintainable.

Q11. What is the need for components?

Answer: Components promote:

- **Reusability:** Write once, use many times.
- **Modularity:** Break complex UI into smaller, manageable pieces.
- **Maintainability:** Easier to debug and update.
- **Separation of concerns:** Each component handles its own logic and styling.
- **Scalability:** Large apps can be built by composing components.

Q12. What is the difference between components and pages?

Answer:

- **Components:** Reusable UI blocks (e.g., button, navbar, card). They can be used in multiple pages.
- **Pages (Views):** Top-level components mapped to routes. They represent a full screen (e.g., Login page, Dashboard page) and often compose multiple components.

Q13. What are Vue directives? / Built-in directives?

Answer: Directives are special attributes prefixed with `v-` that tell Vue to do something to the DOM. Built-in directives include:

- `v-if`, `v-else`, `v-else-if` — conditional rendering
- `v-show` — toggle visibility via CSS
- `v-for` — list rendering
- `v-bind` (`:`) — bind attributes
- `v-model` — two-way binding
- `v-on` (`@`) — event handling
- `v-text`, `v-html` — set text/HTML
- `v-cloak` — hide until Vue is ready
- `v-once` — render once
- `v-pre` — skip compilation

Q14. List 4 Vue directives.

Answer: `v-if`, `v-for`, `v-model`, `v-bind` (Or `v-show`, `v-on`, `v-html`, `v-cloak`).

Q15. What is `v-if` vs `v-show`? When to use which?

Answer:

- **`v-if`:** Conditionally renders the element. If false, the element is removed from the DOM. It has higher toggle cost but lower initial render cost if condition is false.
- **`v-show`:** Always renders the element but toggles `display: none` via CSS. It has lower toggle cost but higher initial render cost.
- **Use `v-if`** when the condition rarely changes or the element is heavy.
- **Use `v-show`** when the element toggles frequently.

Q16. What is `v-for`? Show where you used it.

Answer: `v-for` is used to render a list of items by iterating over an array or object.

Syntax: `v-for="item in items" :key="item.id"` . In my project, I used it to display lists of [e.g., movies, bookings, users] in the admin/user dashboard.

Q17. What is `v-model` vs `v-bind`?

Answer:

- **`v-bind ( : )`:** One-way binding — data flows from the component's data to the DOM attribute. Example: `:src="imageUrl"` .
- **`v-model`:** Two-way binding — data flows both ways. Used mainly on form inputs. Example: `<input v-model="username">` updates `username` when the user types, and updates the input when `username` changes.

Q18. What is `v-cloak`?

Answer: `v-cloak` is a directive that hides the element until Vue has finished compiling. It prevents the flash of uncompiled template (e.g., seeing `{{ message }}` before Vue loads). It works with a CSS rule: `[v-cloak] { display: none; }` .

Q19. What is `v-bind ( : )`?

Answer: `v-bind` dynamically binds an HTML attribute to a Vue expression. The shorthand is `:` . Example: `` or `<a :href="link">` . It is one-way (data → DOM).

Q20. What is the use of `:key` in `v-for`?

Answer: `:key` gives each item in a `v-for` list a unique identity. It helps Vue's diffing algorithm track which items changed, were added, or removed, improving rendering performance and preventing bugs when the list order changes.

Q21. What are Vue lifecycle hooks?

Answer: Lifecycle hooks are special methods that run at specific stages of a component's life:

- **beforeCreate** — before reactivity is set up
- **created** — after data/methods are set up, before DOM mount
- **beforeMount** — before DOM insertion
- **mounted** — after DOM insertion (good for API calls)
- **beforeUpdate** — before DOM re-render
- **updated** — after DOM re-render
- **beforeUnmount / beforeDestroy** — before component removal
- **unmounted / destroyed** — after component removal

Q22. Which lifecycle hooks did you use and where?

Answer: I mainly used:

- **mounted()** — to fetch data from the backend API when the component loads (e.g., loading bookings, user list).
- **created()** — to initialize data or call non-DOM-dependent logic.
- **updated()** — occasionally to react to data changes.
- **beforeUnmount()** — to clear timers or listeners.

Q23. Difference between created and mounted.

Answer:

- **created:** Runs after data and methods are set up but before the DOM is mounted. DOM is not accessible. Good for API calls that don't need DOM.
- **mounted:** Runs after the DOM is mounted. DOM is accessible. Good for DOM manipulation or API calls that need rendered elements.

Q24. Difference between beforeMount and beforeCreate.**Answer:**

- **beforeCreate:** Runs before data observation and event setup. `data` and `methods` are not yet available.
- **beforeMount:** Runs after `created` and before the DOM is mounted. Data and methods are available, but the DOM is not yet rendered.

Q25. What is onMounted()?

Answer: `onMounted()` is the Composition API equivalent of the `mounted` lifecycle hook. It registers a callback that runs after the component is mounted to the DOM. Used for API calls, DOM access, or initialization.

Q26. What is mounted?

Answer: `mounted` is a lifecycle hook that runs after the component's DOM is inserted into the page. It is commonly used to fetch data from APIs, initialize third-party libraries, or access DOM elements.

Q27. What are props in Vue?

Answer: Props (properties) are custom attributes used to pass data from a parent component to a child component. They are read-only in the child (one-way data flow). Declared in the child via `props: ['name']` or `defineProps()` in Composition API.

Q28. How does a parent communicate with a child?

Answer: The parent passes data to the child via **props**. Example:

```
<ChildComponent :message="parentMessage" />
```

The child receives it as a prop.

Q29. How does a child communicate with a parent? (emit)

Answer: The child emits a custom event using `this.$emit('eventName', payload)` (Options API) or `emit('eventName', payload)` (Composition API). The parent listens with `@eventName="handler"` .

Q30. What is emit? Is it a variable or something else?

Answer: `emit` is a function (not a variable) used in the Composition API (`const emit = defineEmits(['event'])`) or accessed as `this.$emit` in Options API. It triggers a custom event that the parent can listen to, enabling child-to-parent communication.

Q31. How to pass data from child to parent?

Answer: The child calls `this.$emit('eventName', data)` . The parent listens: `<Child @eventName="handleData" />` and defines `handleData(data) { ... }` .

Q32. How does a grandparent pass data to grandchild?

Answer: Options:

1. **Props drilling:** Grandparent → Parent → Grandchild (pass props through each level).
2. **Provide/Inject:** Grandparent `provide('key', value)` , grandchild `inject('key')` .
3. **State management (Vuex/Pinia):** Store data globally and access from any component.

Q33. What are slots in Vue?

Answer: Slots are placeholders in a child component that allow the parent to inject content. Example: `<slot></slot>` in the child; parent uses `<Child><p>Content</p></Child>` . Named slots (`<slot name="header">`) allow multiple content areas.

Q34. Difference between props and slots.

Answer:

- **Props:** Pass data (values) from parent to child.
- **Slots:** Pass HTML/content from parent to child.
- Props are for data; slots are for markup.

Q35. Did you use slots?

Answer: [Yes/No — e.g., "Yes, I used a default slot in my layout component to inject page content, and named slots for header/footer."]

Q36. What are computed properties?

Answer: Computed properties are functions that derive a value from reactive data and cache the result. They re-evaluate only when their dependencies change. Example:

`computed: { fullName() { return this.first + ' ' + this.last; } }` . They are used in templates like `{{ fullName }}` .

Q37. What are watchers in Vue? / How does watch work?

Answer: Watchers observe a reactive property and run a callback when it changes. They are useful for side effects (e.g., API calls, logging). Example:

`watch: { searchQuery(newVal) { this.fetchResults(newVal); } }` . Unlike computed, watchers don't return a value; they perform actions.

Q38. Difference between computed and watchers.**Answer:**

- **Computed:** Derives a value, caches it, re-evaluates only when dependencies change. Used in templates.
- **Watchers:** Observe a property and run side effects when it changes. Not used in templates. Good for async or expensive operations.

Q39. Difference between methods and computed properties.**Answer:**

- **Methods:** Run every time they are called; not cached.
- **Computed:** Cached based on dependencies; only re-evaluate when dependencies change.
- Use computed for derived data; methods for actions/events.

Q40. What is data binding? / One-way vs Two-way binding.**Answer:**

- **Data binding:** Synchronizing data between the component and the view.
- **One-way:** Data flows from component → view (e.g., `v-bind`, interpolation `{{ }}`).
- **Two-way:** Data flows both ways (e.g., `v-model` on inputs).

Q41. What are the different ways to perform binding?**Answer:**

- Interpolation: `{{ data }}`
- Attribute binding: `v-bind:attr` or `:attr`
- Class/style binding: `:class`, `:style`
- Two-way: `v-model`
- Event binding: `v-on:event` or `@event`

Q42. What is Vue Router?

Answer: Vue Router is the official routing library for Vue. It maps URL paths to components, enabling SPA navigation without full page reloads. It supports dynamic routes, nested routes, route params, query params, navigation guards, and lazy loading.

Q43. How does Vue Router work?

Answer: You define routes as an array of objects: `{ path: '/home', component: Home }`. The `<router-view>` renders the matched component. `<router-link>` creates navigation links. When the URL changes, Vue Router matches the path and renders the corresponding component without reloading the page.

Q44. What is router-view vs router-link?

Answer:

- `<router-view>` : A placeholder where the matched route component is rendered.
- `<router-link>` : Creates a navigation link (like `<a>`) that changes the route without page reload. It has a `to` prop.

Q45. What is *route* vs router?

Answer:

- `$route` : The current route object (path, params, query, name).
- `$router` : The router instance with methods like `push()`, `replace()`, `go()`.

Q46. What are dynamic routes in Vue?

Answer: Dynamic routes have parameters in the path, e.g., `/user/:id`. The `:id` is accessible via `$route.params.id`. Used for detail pages (e.g., user profile, product page).

Q47. What are navigation guards?

Answer: Navigation guards are hooks that run before/after route navigation. Types: `beforeEach`, `beforeResolve`, `afterEach`, and in-component guards (`beforeRouteEnter`, `beforeRouteUpdate`, `beforeRouteLeave`). Used for authentication checks, permissions, or data fetching.

Q48. What is createWebHistory()?

Answer: `createWebHistory()` is a Vue Router function that enables HTML5 history mode. It uses the browser's history API to create clean URLs (e.g., `/about` instead of `/#/about`). It requires server configuration to fallback to `index.html` for all routes.

Q49. What is Vuex?

Answer: Vuex is a state management library for Vue. It provides a centralized store for all components, with rules ensuring predictable state mutations. Core concepts: **State** (data), **Getters** (computed), **Mutations** (synchronous changes), **Actions** (asynchronous operations), **Modules** (splitting the store).

Q50. What is Vuex vs Pinia?

Answer:

- **Vuex:** Older, more boilerplate, uses mutations and actions. Works with Vue 2 and 3.
- **Pinia:** Newer, official recommendation for Vue 3. Simpler API, no mutations, supports Composition API, better TypeScript support, and modular by design.

Q51. What are state, mutations, and actions?

Answer:

- **State:** The central data object.
- **Mutations:** Synchronous functions that modify state. Called via `commit`.
- **Actions:** Asynchronous functions that commit mutations. Called via `dispatch`.

Q52. What is the use of actions?

Answer: Actions handle asynchronous operations (API calls, timers) and then commit mutations to update state. They keep mutations pure and synchronous.

Q53. What is state management?

Answer: State management is the practice of centrally storing and managing application state so that multiple components can access and update it predictably. Libraries like Vuex/Pinia provide this.

Q54. What is the advantage of state management?

Answer:

- Centralized state — single source of truth.
- Predictable state changes via mutations/actions.
- Easier debugging (time-travel, devtools).
- Avoids prop drilling.
- Scales well for large apps.

Q55. What is Options API vs Composition API?

Answer:

- **Options API:** Organizes code by options (data, methods, computed, watch, lifecycle). Easier for beginners.
- **Composition API:** Organizes code by logical concerns using `setup()`, `ref`, `reactive`, `computed`, `watch`. Better for large components and TypeScript.

Q56. What is `<script setup>` ?

Answer: `<script setup>` is a compile-time syntactic sugar for the Composition API in SFCs. It allows top-level `await`, auto-imports components, and exposes bindings to the template without `return`. Cleaner and more concise.

Q57. What is ref vs reactive?

Answer:

- **ref:** Creates a reactive reference for a primitive or object. Accessed via `.value` in JS.
- **reactive:** Creates a reactive object (deep). Accessed directly (no `.value`). Not for primitives.

Q58. What is the use of `ref`?

Answer: `ref` creates a reactive variable. In the template, it's auto-unwrapped. In JS, use `.value` to read/write. Used for primitives, arrays, and objects in Composition API.

Q59. What is `:class` (colon before class)?

Answer: `:class` is shorthand for `v-bind:class`. It dynamically binds CSS classes based on data. Example: `:class="{ active: isActive }"` applies the `active` class when `isActive` is true.

Q60. What is class and style binding?

Answer: Vue allows dynamic binding of `class` and `style` attributes using `v-bind`. Examples:

- `:class="{ active: isActive, 'text-danger': hasError }"`
- `:style="{ color: textColor, fontSize: size + 'px' }"`

Q61. What is a high-order function?

Answer: A higher-order function is a function that takes another function as an argument or returns a function. Examples in JS: `map`, `filter`, `reduce`, `forEach`.

Q62. What are anonymous functions in JS?

Answer: Functions without a name. Example: `const fn = function() { ... }` or `setTimeout(function() { ... }, 1000)`. Often used as callbacks.

Q63. Is every arrow function an anonymous function?

Answer: Yes, arrow functions are always anonymous (they don't have their own name unless assigned to a variable). However, they can be assigned to variables, giving them a name for debugging.

Q64. What is named export vs default export?

Answer:

- **Named export:** `export const foo = ...` — must be imported with the same name:
`import { foo } from './module' .`
- **Default export:** `export default foo` — can be imported with any name:
`import bar from './module' .` Only one default export per module.

Q65. What is optional chaining vs nullish coalescing?

Answer:

- **Optional chaining (`?.`):** Safely accesses nested properties. `obj?.a?.b` returns `undefined` if `obj` or `a` is `null/undefined`.
- **Nullish coalescing (`??`):** Returns the right operand if the left is `null` or `undefined` .
`a ?? b` returns `b` only if `a` is `null/undefined` (not for `0` or `''`).

Q66. What is the difference between Vue and Angular?

Answer:

- **Vue:** Lightweight, progressive, easy to learn, flexible, two-way binding via v-model.
- **Angular:** Full-fledged framework, TypeScript-based, heavier, opinionated, uses dependency injection, RxJS, and has a steeper learning curve.

Q67. What is the difference between Vue and React?

Answer:

- **Vue:** Template-based, two-way binding, built-in directives, simpler state management (Vuex/Pinia), gentler learning curve.
- **React:** JSX-based, one-way binding, requires external libraries for routing/state, larger ecosystem, steeper learning curve.

Q68. What are the advantages of Vue over React?**Answer:**

- Simpler syntax and learning curve.
- Two-way binding built-in.
- Single-file components with scoped CSS.
- Official routing and state management.
- Smaller bundle size.
- Better documentation.

Q69. Why did you not use React?

Answer: [Reason — e.g., "Vue's template syntax and built-in directives made development faster for this project. Vue's integration with Flask via CDN was simpler, and I was more comfortable with Vue's reactivity system."]

Q70. What are the benefits of Vue.js?**Answer:**

- Reactive data binding
- Component-based architecture
- Virtual DOM for performance
- Lightweight and fast
- Easy integration

- Comprehensive tooling (Vue CLI, DevTools)
- Good documentation and community

Q71. Why do we use Vue.js rather than other JavaScript frameworks?

Answer: Vue offers a balance of simplicity, performance, and flexibility. It's easier to learn than Angular, less boilerplate than React, and provides official solutions for routing and state management. It's also incrementally adoptable.

Q72. Is Vue responsive? / Is your project responsive?

Answer: Vue itself is not responsible for responsiveness; CSS (media queries, Flexbox, Grid, Bootstrap) handles that. My project is responsive because I used [Bootstrap / CSS media queries / Flexbox] to adapt layouts to different screen sizes.

Q73. How did you make it responsive?

Answer: I used:

- Bootstrap's grid system (`row` , `col` , `col-md-*`)
- Flexbox (`.d-flex` , `justify-content` , `align-items`)
- CSS media queries
- Responsive tables (`.table-responsive`)
- Relative units (`%` , `rem` , `vw`)

Q74. What is CSS vs Bootstrap?

Answer:

- **CSS:** Custom styling language; full control but more code.
- **Bootstrap:** A CSS framework with pre-built classes (grid, buttons, forms, modals). Faster development, consistent design, responsive by default.

Q75. How does Bootstrap make the app responsive?

Answer: Bootstrap uses a 12-column grid system with breakpoints (`sm` , `md` , `lg` , `xl` , `xxl`). Classes like `col-md-6` stack on small screens and sit side-by-side on larger ones. It also provides responsive utilities (`.d-none` , `.d-md-block`) and responsive components.

Q76. What are CSS selectors? Types?

Answer: CSS selectors target HTML elements for styling. Types:

- **Element:** `p { }`
- **Class:** `.btn { }`
- **ID:** `#header { }`
- **Universal:** `* { }`
- **Attribute:** `[type="text"] { }`
- **Pseudo-class:** `:hover { }`
- **Pseudo-element:** `::before { }`
- **Combinators:** `div > p` , `div p` , `div + p` , `div ~ p`

Q77. Difference between Flexbox and Grid.

Answer:

- **Flexbox:** 1D layout (row OR column). Good for aligning items in a single direction.
- **Grid:** 2D layout (rows AND columns). Good for complex layouts.
- Flexbox is content-driven; Grid is layout-driven.

Q78. Difference between cell spacing and cell padding.

Answer:

- **Cell spacing:** Space between table cells (border-spacing in CSS).
- **Cell padding:** Space between cell content and cell border (padding in CSS).

Q79. What is inline CSS vs internal CSS vs external CSS?**Answer:**

- **Inline:** `style="color:red"` directly on the element. Highest specificity.
- **Internal:** `<style>` tag in `<head>` . Applies to that page.
- **External:** `.css` file linked via `<link>` . Reusable across pages.

Q80. Why is inline CSS given more preference by the compiler?

Answer: Inline CSS has the highest specificity (after `!important`), so it overrides internal and external styles. This is by CSS cascade rules.

Q81. How is internal CSS better than inline CSS?

Answer: Internal CSS separates styling from markup, is reusable within the page, easier to maintain, and avoids repetition. Inline CSS clutters HTML and is hard to override.

Q82. What is DOM manipulation?

Answer: DOM manipulation is changing the structure, content, or style of the DOM using JavaScript (e.g., `document.getElementById` , `appendChild` , `innerHTML`). In Vue, you rarely do this directly; you change data and Vue updates the DOM.

Q83. What is the entry point of your frontend?

Answer: The entry point is `index.html` (which contains `<div id="app"></div>`) and `main.js` (which creates the Vue app and mounts it to `#app`). In Vue CLI, `main.js` is the entry.

Q84. What is index.html?

Answer: `index.html` is the main HTML file served to the browser. It contains the root `<div id="app">` where Vue mounts the app, and includes the Vue CDN script or the bundled JS file.

Q85. How is Vue connected with index.html?

Answer: In CDN mode, `<script src="vue.js">` is included in `index.html`, and `new Vue({ el: '#app' })` mounts to the `#app` div. In CLI mode, `main.js` imports Vue and mounts to `#app`, and the build tool injects the bundled JS into `index.html`.

Q86. What is package.json? Explain all packages used.

Answer: `package.json` is the manifest for a Node project. It lists dependencies (e.g., `vue`, `vue-router`, `axios`, `bootstrap`), devDependencies (e.g., `vite`, `@vitejs/plugin-vue`), and scripts (`dev`, `build`, `serve`). It also contains project metadata (name, version).

Q87. What is Vite? Why did you use it?

Answer: Vite is a fast build tool and dev server for modern web projects. It uses native ES modules for instant hot module replacement (HMR) and Rollup for production builds. It's faster than Webpack for development.

Q88. What is npm?

Answer: npm (Node Package Manager) is the default package manager for Node.js. It installs, updates, and manages JavaScript libraries. It uses `package.json` and `package-lock.json`.

Q89. What is node.js?

Answer: Node.js is a JavaScript runtime built on Chrome's V8 engine. It allows JavaScript to run on the server side. It's used for build tools (Vite, Webpack), package management (npm), and backend development (Express).

Q90. What is the use of node in your project?

Answer: Node is used to run the build tools (Vite/Webpack), install dependencies (npm), and serve the frontend during development. It is not used in production for the backend (which is Flask/Python).

Q91. Will node_modules be installed in the venv?

Answer: No. `node_modules` is for JavaScript dependencies and is installed by npm in the project root. `venv` is for Python dependencies. They are separate.

Q92. Where does it place the modules for the node part?

Answer: In the `node_modules` folder in the project root.

Q93. If you delete package-lock.json will you face any errors?

Answer: Not immediately, but `package-lock.json` ensures deterministic installs. Deleting it may cause different versions to be installed on `npm install`, leading to potential inconsistencies. It's best to keep it.

Q94. Do you need to do npm run build before npm run dev?

Answer: No. `npm run dev` starts the development server with HMR. `npm run build` creates an optimized production build. They are separate commands.

Q95. What is the difference between npm run build and npm run dev?

Answer:

- **dev:** Starts a development server with hot reload, source maps, and no minification.
- **build:** Creates a production-ready bundle (minified, optimized) in the `dist` folder.

Q96. What is the use of `useList` ?

Answer: `useList` is a SQLAlchemy relationship parameter. `useList=False` makes a one-to-one relationship instead of one-to-many. It tells SQLAlchemy to return a single object, not a list.

Q97. What is `setup()` in Vue?

Answer: `setup()` is the entry point for the Composition API. It runs before the component is created, returns an object whose properties are exposed to the template, and is where you define reactive state, computed, watchers, and lifecycle hooks.

Q98. What is a template vs script vs style?**Answer:**

- `<template>` : HTML markup with Vue directives.
- `<script>` : JavaScript logic (data, methods, lifecycle).
- `<style>` : CSS, optionally scoped to the component.

Q99. What is `v-if` vs `v-for`?**Answer:**

- **v-if:** Conditional rendering — renders if condition is true.
- **v-for:** List rendering — iterates over an array/object.
- Don't use them on the same element (`v-if` has higher priority in Vue 2, `v-for` in Vue 3).

Q100. Write v-for loop in notepad.

Answer:

```
html
```

```
<ul>  
  <li v-for="item in items" :key="item.id">{{ item.name }}</li>  
</ul>
```

2. FLASK & BACKEND

Q1. What is Flask?

Answer: Flask is a lightweight, open-source Python web framework. It is a WSGI-compliant microframework that provides routing, request handling, and templating (Jinja2) but leaves database, authentication, and other features to extensions (e.g., Flask-SQLAlchemy, Flask-Security).

Q2. Is Flask open source?

Answer: Yes, Flask is open source under the BSD license.

Q3. Can you use Flask for large applications?

Answer: Yes, with proper architecture (blueprints, application factory, extensions) and a production WSGI server (Gunicorn, uWSGI), Flask can power large applications. However, for very large apps, frameworks like Django may offer more built-in structure.

Q4. What is Flask-SQLAlchemy?

Answer: Flask-SQLAlchemy is an extension that integrates SQLAlchemy (an ORM) with Flask. It provides a `db` object, simplifies model definition, and handles session

management and database connections.

Q5. What is SQLAlchemy? Why use it?

Answer: SQLAlchemy is a Python SQL toolkit and ORM. It lets you interact with databases using Python objects instead of raw SQL. Benefits: database-agnostic, prevents SQL injection, supports relationships, migrations, and complex queries.

Q6. What is ORM?

Answer: ORM (Object-Relational Mapping) maps database tables to Python classes and rows to objects. It lets you query and manipulate data using Python instead of SQL.

Example: `User.query.filter_by(id=1).first()` instead of `SELECT * FROM user WHERE id=1`.

Q7. What are the advantages of ORM?

Answer:

- Database-agnostic (easy to switch DB).
- Prevents SQL injection.
- Less boilerplate.
- Supports relationships and lazy loading.
- Easier to maintain and refactor.

Q8. What are the disadvantages of ORM?

Answer:

- Performance overhead for complex queries.
- Abstraction can hide inefficient SQL.
- Learning curve.
- Not suitable for very complex reporting queries.

Q9. What is `db.Model` ?

Answer: `db.Model` is the base class for all SQLAlchemy models in Flask-SQLAlchemy. Your models inherit from it to get ORM functionality (table mapping, query methods).

Q10. How do you initialize a table?

Answer: Define a class inheriting from `db.Model`, add columns as class attributes (`id = db.Column(db.Integer, primary_key=True)`), and call `db.create_all()` to create the table.

Q11. Where is the database initialized?

Answer: In `app.py` or `__init__.py`: `db = SQLAlchemy(app)` or `db.init_app(app)`. The URI is set in `app.config['SQLALCHEMY_DATABASE_URI']`.

Q12. What is `__tablename__` ? Is it needed?

Answer: `__tablename__` explicitly sets the table name. If omitted, SQLAlchemy derives it from the class name (lowercased). It's optional but recommended for clarity.

Q13. What is `__repr__` ?

Answer: `__repr__` returns a string representation of an object, used for debugging. In models, it typically returns something like `<User 1>`. Example:

```
def __repr__(self): return f'<User {self.id}>'
```

Q14. What is `__repr__` vs `__str__` ?

Answer:

- `__repr__` : Unambiguous, for developers/debugging.
- `__str__` : Readable, for end-users.

- If `__str__` is not defined, `__repr__` is used.

Q15. What are dunder methods?

Answer: Dunder (double underscore) methods are special Python methods like `__init__`, `__repr__`, `__str__`, `__len__`, `__eq__`. They define object behavior for built-in operations.

Q16. What is serialize (w.r.t. models)?

Answer: Serialization converts a model object into a dictionary or JSON so it can be sent over an API. Example: `def serialize(self): return { 'id': self.id, 'name': self.name }` .

Q17. What is `to_dict()` ?

Answer: `to_dict()` is a custom method on a model that returns its fields as a dictionary, used for JSON serialization.

Q18. What are other methods for serializable data?

Answer: `jsonify`, `marshal` (Flask-RESTful), `marshmallow`, `pydantic`, `custom serialize()` / `to_dict()` methods.

Q19. What is `db.String(128)` ? What does 128 mean? How is it stored in memory?

Answer: `db.String(128)` defines a `VARCHAR(128)` column. 128 is the maximum character length. In memory, it's stored as a variable-length string, with length prefix depending on the database.

Q20. What is the difference between String and Text in database table?

Answer:

- **String:** Fixed max length (e.g., VARCHAR(255)). Good for short fields.
- **Text:** Unlimited length. Good for long content (descriptions, comments).

Q21. What is `db.session.add()` vs `db.session.commit()` ?**Answer:**

- `add()` : Stages the object for insertion/update in the session.
- `commit()` : Persists the changes to the database.
- You need both: `db.session.add(obj); db.session.commit()` .

Q22. What is `.all()` vs `.first()` ?**Answer:**

- `.all()` : Returns a list of all matching records.
- `.first()` : Returns the first matching record or `None` .

Q23. What is `.one()` ?

Answer: `.one()` returns exactly one result. Raises an error if zero or more than one result is found.

Q24. What is `query.filter()` vs `filter_by()` ?**Answer:**

- `filter()` : Uses SQL-like expressions: `User.query.filter(User.age > 18)` .
- `filter_by()` : Uses keyword arguments: `User.query.filter_by(age=18)` .
- `filter()` is more flexible; `filter_by()` is simpler.

Q25. What is `like` vs `ilike` ?**Answer:**

- `like` : Case-sensitive pattern matching. `User.query.filter(User.name.like('%john%'))` .
- `ilike` : Case-insensitive. `User.query.filter(User.name.ilike('%john%'))` .

Q26. What is `%` in search?

Answer: `%` is a wildcard in SQL `LIKE` / `ILIKE` . `%john%` matches any string containing "john". `john%` matches strings starting with "john".

Q27. What is `joinedload` ?

Answer: `joinedload` is a SQLAlchemy eager-loading option. It loads related objects in the same query using a JOIN, avoiding the N+1 query problem. Example:

```
User.query.options(joinedload(User.posts)).all() .
```

Q28. What is N+1 query problem?

Answer: When you fetch N records and then make 1 query per record to fetch related data, resulting in N+1 queries. `joinedload` or `selectinload` solves it by fetching related data in one query.

Q29. What is `lazy=True` ?

Answer: `lazy=True` (default) means related objects are loaded on first access (lazy loading). Other options: `'select'` , `'joined'` , `'subquery'` , `'dynamic'` .

Q30. What is `lazy='dynamic'` ?

Answer: `lazy='dynamic'` returns a query object instead of a list, allowing further filtering. Example: `user.posts.filter_by(active=True).all()` .

Q31. What is `backref` ?

Answer: `backref` automatically creates a reverse relationship on the related model.

Example: `posts = db.relationship('Post', backref='author')` adds `post.author` .

Q32. What is `back_populates` ?

Answer: `back_populates` explicitly defines the reverse side of a relationship. Both sides must declare it. Example: `User.posts = relationship('Post', back_populates='author')` and `Post.author = relationship('User', back_populates='posts')` .

Q33. Difference between `backref` and `back_populates`.

Answer:

- **`backref`:** One-sided declaration; the reverse is auto-created.
- **`back_populates`:** Two-sided declaration; both models explicitly define the relationship. More verbose but clearer and preferred in modern SQLAlchemy.

Q34. What is `cascade='all, delete-orphan'` ?

Answer: Cascade defines what happens to related objects when the parent is deleted or updated. `'all'` includes save-update, merge, refresh-expire, expunge, delete.

`'delete-orphan'` deletes child objects that are removed from the parent's collection.

Q35. What is `ondelete='CASCADE'` ?

Answer: `ondelete='CASCADE'` is a database-level constraint. When the parent row is deleted, child rows are automatically deleted by the database.

Q36. What is `uselist=False` ?

Answer: In a relationship, `useList=False` makes it one-to-one instead of one-to-many, returning a single object instead of a list.

Q37. What is the difference between primary key and unique key?

Answer:

- **Primary key:** Uniquely identifies a row; cannot be NULL; only one per table.
- **Unique key:** Ensures uniqueness; can be NULL (one NULL allowed in most DBs); multiple per table.

Q38. Can unique key be primary key?

Answer: Yes, a unique key can be used as a primary key if it is NOT NULL and uniquely identifies rows. But primary keys have additional constraints (no NULL, only one).

Q39. Can we use auto-increment instead of primary key?

Answer: Auto-increment alone doesn't guarantee uniqueness across all scenarios (e.g., manual inserts). A primary key provides uniqueness, NOT NULL, and indexing. Auto-increment is often used with a primary key.

Q40. What is a foreign key?

Answer: A foreign key is a column that references the primary key of another table, enforcing referential integrity. Example: `user_id` in `Post` references `User.id`.

Q41. What is referential integrity?

Answer: Referential integrity ensures that foreign key values always point to existing primary key values. It prevents orphan records.

Q42. How to maintain relational integrity?

Answer: Use foreign keys, `ON DELETE CASCADE / SET NULL`, `NOT NULL` constraints, and transactions. In SQLAlchemy, use `ForeignKey` and `relationship`.

Q43. What happens if parent row is deleted?

Answer: Depends on the foreign key constraint:

- `CASCADE` : child rows deleted.
- `SET NULL` : child foreign key set to `NULL`.
- `RESTRICT / NO ACTION` : deletion prevented.
- Default: may raise an error.

Q44. What if `ON DELETE CASCADE` is not used?

Answer: Deleting the parent may fail (if child rows exist) or leave orphan child rows (if no constraint), depending on the database.

Q45. What is the impact of deleting a record which other records point to as foreign key?

Answer: It can cause referential integrity violations, orphan records, or errors. Proper cascade rules or manual cleanup are needed.

Q46. What is `relationship` in SQLAlchemy?

Answer: `relationship()` defines a link between two models, allowing you to access related objects as Python attributes. Example: `user.posts` returns all posts by that user.

Q47. What are the types of relationships? (1-1, 1-many, many-many)

Answer:

- **One-to-One:** `uselist=False` on both sides.
- **One-to-Many:** Parent has many children; child has one parent.
- **Many-to-Many:** Uses an association table with `secondary` .

Q48. What is `secondary` in relationship?

Answer: `secondary` specifies the association table for a many-to-many relationship.

Example: `roles = db.relationship('Role', secondary=user_roles) .`

Q49. What is `secondary="user_roles"` ?

Answer: It tells SQLAlchemy to use the `user_roles` table as the junction table for a many-to-many relationship between `User` and `Role` .

Q50. What is a self-referential relationship?

Answer: A relationship where a model references itself. Example: `Employee.manager_id` references `Employee.id` . Used for hierarchies.

Q51. What is indexing in SQLAlchemy?

Answer: Indexing creates a data structure (usually B-tree) to speed up queries on a column. In SQLAlchemy: `db.Column(db.String, index=True) .`

Q52. How to index in SQLAlchemy?

Answer: `name = db.Column(db.String(100), index=True) or`
`__table_args__ = (db.Index('idx_name', 'name'),) .`

Q53. What is index on two columns?

Answer: A composite index on two columns, e.g.,
`db.Index('idx_name_age', 'name', 'age')` . Speeds up queries filtering on both columns.

Q54. What is the purpose of index?

Answer: Indexes speed up SELECT queries (WHERE, JOIN, ORDER BY) by allowing the database to find rows without scanning the whole table. Trade-off: slower INSERT/UPDATE and extra storage.

Q55. What are SQL constraints?

Answer: Rules enforced on columns: NOT NULL , UNIQUE , PRIMARY KEY , FOREIGN KEY , CHECK , DEFAULT .

Q56. What is a composite key?

Answer: A primary key made of two or more columns. Example:
(`student_id`, `course_id`) in an enrollment table.

Q57. What is candidate key?

Answer: Any column or set of columns that can uniquely identify a row. A table can have multiple candidate keys; one becomes the primary key.

Q58. What is the difference between candidate key and primary key?

Answer: Candidate key is any unique identifier; primary key is the chosen candidate key used as the main identifier. Primary key cannot be NULL; candidate keys may allow NULL (if unique).

Q59. What is a unique constraint?

Answer: Ensures all values in a column (or combination) are distinct. Allows one NULL (in most DBs).

Q60. What error do you get if you insert a duplicate entry?

Answer: IntegrityError (e.g., UNIQUE constraint failed in SQLite, Duplicate entry in MySQL).

Q61. What is SQL injection?

Answer: SQL injection is an attack where malicious SQL is inserted via user input, potentially exposing or destroying data. Prevention: parameterized queries, ORM, input validation.

Q62. What is CSRF?

Answer: Cross-Site Request Forgery is an attack where a user is tricked into performing unwanted actions on a site they're authenticated on. Prevention: CSRF tokens, SameSite cookies.

Q63. What is CORS?

Answer: Cross-Origin Resource Sharing is a browser security mechanism that restricts requests to a different origin. Servers must send `Access-Control-Allow-Origin` headers. In Flask, use `flask-cors`.

Q64. How is CORS configured?

Answer: `from flask_cors import CORS; CORS(app) or
CORS(app, resources={r"/api/*": {"origins": "*"}}) .`

Q65. What is a blueprint in Flask?

Answer: A blueprint is a way to organize routes, templates, and static files into modular components. It's registered on the app with `app.register_blueprint(bp)` .

Q66. Why use blueprints?

Answer: Modularity, reusability, separation of concerns, easier testing, and cleaner code organization for large apps.

Q67. What is jsonify ?

Answer: `jsonify` converts Python dicts/lists into a JSON response with the correct Content-Type: `application/json` .

Q68. What is jsonify vs json.dumps ?

Answer:

- `jsonify` : Returns a Flask Response object with JSON content type.
- `json.dumps` : Returns a JSON string; you must create a Response manually.

Q69. What is jsonify vs marshal ?

Answer:

- `jsonify` : Converts Python objects to JSON.
- `marshal` : Flask-RESTful's serialization for formatting output fields.

Q70. What is jsonify vs json package?

Answer: `json` is Python's standard library for JSON encoding/decoding. `jsonify` is Flask's helper that wraps `json.dumps` and sets headers.

Q71. What is the difference between JSON and jsonify?

Answer: JSON is a data format. `jsonify` is a Flask function that returns a Response with JSON data and correct headers.

Q72. What is the difference between `jsonify` and `json.dumps` ?

Answer: `jsonify` returns a Flask Response; `json.dumps` returns a string. `jsonify` also sets the content type.

Q73. What is `app.route()` ?

Answer: `@app.route('/path')` is a decorator that maps a URL to a view function.

Example: `@app.route('/home') def home(): return 'Hello' .`

Q74. What is `@app.route` decorator?

Answer: It registers a function as a view for a specific URL rule. It can also specify

methods: `@app.route('/user', methods=['GET', 'POST']) .`

Q75. What are decorators?

Answer: Decorators are functions that modify the behavior of another function. In

Python, they use `@decorator_name` . Example: `@app.route` , `@login_required` .

Q76. How does a decorator differ from a function?

Answer: A decorator takes a function as input and returns a modified function. It's a higher-order function used to add behavior without changing the original function's code.

Q77. Write a hello world decorator.**Answer:**

python

```
def hello_decorator(func):  
    def wrapper():  
        print("Hello")  
        func()  
    return wrapper  
  
@hello_decorator  
def say_hi():  
    print("Hi")  
  
say_hi() # Prints Hello, then Hi
```

Q78. What is @jwt_required ?

Answer: A Flask-JWT-Extended decorator that ensures a valid JWT is present in the request. If not, returns 401.

Q79. What is @role_required ?

Answer: A custom decorator (or Flask-Security's) that checks if the authenticated user has the required role. Returns 403 if not.

Q80. What is @admin_required ?

Answer: A custom decorator that checks if the user has the admin role. It often wraps @jwt_required and checks current_user.roles .

Q81. Can we swap the order of @jwt_required and @role_required ?

Answer: No. `@jwt_required` must run first to authenticate the user. `@role_required` depends on the authenticated user. Swapping causes errors.

Q82. How does `@admin_required` work according to your code?

Answer: It first verifies the JWT (`@jwt_required`), then checks if `current_user` has the admin role. If not, returns 403.

Q83. What is `@cache.cached` vs `@cache.memoize` ?

Answer:

- `@cache.cached` : Caches the response based on the request path and query string. No arguments.
- `@cache.memoize` : Caches the result based on function arguments. Good for functions with parameters.

Q84. What is the difference between `@cache.cached` and `@cache.memoize` ?

Answer: `cached` ignores function arguments; `memoize` includes them in the cache key. Use `cached` for routes, `memoize` for functions.

Q85. What is `@celery.task` ?

Answer: A decorator that registers a function as a Celery task, allowing it to be executed asynchronously via `.delay()` or `.apply_async()` .

Q86. What is `@app.errorhandler` ?

Answer: A decorator to register a function as a handler for a specific HTTP error code. Example: `@app.errorhandler(404)` .

Q87. What is `get_role_claims` ?

Answer: A Flask-JWT-Extended callback that adds custom claims (like roles) to the JWT payload.

Q88. What is `get_user_id` ?

Answer: A Flask-JWT-Extended callback that extracts the user ID from the JWT identity.

Q89. What is `Flask(__name__)` ? Can we remove `__name__` ?

Answer: `Flask(__name__)` creates the app instance and tells Flask where to find templates/static files. You can pass another module name, but `__name__` is standard. Removing it may break resource paths.

Q90. What is `if __name__ == '__main__':` ? What if you remove it?

Answer: It checks if the script is run directly (not imported). If removed, the code inside runs even when imported, causing side effects.

Q91. What is the base class of Exception?

Answer: `BaseException` is the base class. `Exception` is a subclass of `BaseException`.

Q92. What are different types of exceptions in Python?

Answer: `ValueError` , `TypeError` , `KeyError` , `IndexError` , `AttributeError` , `ImportError` , `FileNotFoundError` , `ZeroDivisionError` , `IntegrityError` , etc.

Q93. What is exception handling?

Answer: Using `try / except / else / finally` to catch and handle errors gracefully without crashing the program.

Q94. Have you used exception handling? Show.

Answer: Yes. Example:

```
python

try:
    user = User.query.get(id)
    if not user:
        raise ValueError("User not found")
except ValueError as e:
    return jsonify({"error": str(e)}), 404
```

Q95. What is `try ... catch` ?

Answer: In Python, it's `try / except` . In JavaScript, `try / catch` . They catch errors and prevent program crash.

Q96. What is WSGI?

Answer: Web Server Gateway Interface is a standard interface between Python web apps and web servers. Flask is a WSGI app; Gunicorn/uWSGI are WSGI servers.

Q97. What is `app_context()` ?

Answer: `app_context()` pushes an application context, allowing access to `current_app` and other app-bound resources outside a request.

Q98. What is `with.app_context()` ?

Answer: A context manager that pushes and pops the application context. Used for background tasks (Celery) that need database/app access.

Q99. What is the use of `joinedload` ?

Answer: Eager-loads related data in one query to avoid N+1 queries. Example:

```
User.query.options(joinedload(User.posts)).all() .
```

Q100. What is the difference between synchronous and asynchronous?

Answer:

- **Synchronous:** Tasks run one after another; blocking.
- **Asynchronous:** Tasks can run concurrently; non-blocking. Used for I/O-bound operations.

Q101. What is an async function?

Answer: A function defined with `async def` that returns a coroutine. It can use `await` to pause execution until an awaitable completes.

Q102. What does an async function return?

Answer: A coroutine object. When awaited, it returns the actual result.

Q103. What is `await` ?

Answer: `await` pauses the execution of an async function until the awaited coroutine/promise completes, then returns its result.

Q104. What is a promise?

Answer: A promise is a JavaScript object representing the eventual completion (or failure) of an asynchronous operation. States: pending, fulfilled, rejected.

Q105. What are the stages of a promise?

Answer: Pending, Fulfilled, Rejected.

Q106. What are higher-order functions?

Answer: Functions that take other functions as arguments or return functions. Examples: `map`, `filter`, `reduce`.

Q107. What is `const` vs `let` vs `var` ?

Answer:

- `var` : Function-scoped, hoisted, can be redeclared.
- `let` : Block-scoped, not hoisted (TDZ), cannot be redeclared.
- `const` : Block-scoped, cannot be reassigned, must be initialized.

Q108. Why `const` instead of `let` or `var` ?

Answer: `const` prevents accidental reassignment, makes code more predictable, and is block-scoped. Use `let` only when reassignment is needed.

Q109. Is JavaScript statically typed or dynamically typed?

Answer: Dynamically typed. Types are checked at runtime, and variables can hold any type.

Q110. What is type coercion?

Answer: Automatic conversion of one type to another. Example: `'5' + 1 → '51'` (string), `'5' - 1 → 4` (number).

Q111. What is the difference between `==` and `===` ?

Answer:

- `==` : Loose equality; performs type coercion.
- `===` : Strict equality; no type coercion. `'5' == 5` is true; `'5' === 5` is false.

Q112. What is `this.$router` ?

Answer: `this.$router` is the Vue Router instance, providing methods like `push` , `replace` , `go` .

Q113. What is `this.$router.push` ?

Answer: Navigates to a new route and adds a history entry. Example:
`this.$router.push('/home')` .

Q114. What is `router.push` ?

Answer: Same as `this.$router.push` but used outside components or in Composition API.

Q115. What is `router.go` ?

Answer: Navigates forward/backward in history. `router.go(-1)` goes back one page.

Q116. What is `router.replace` ?

Answer: Navigates to a new route without adding a history entry.

Q117. What is the difference between `redirect()` and `url_for()` ?

Answer:

- `url_for()` : Generates a URL for a given endpoint.
- `redirect()` : Returns a response that redirects the client to a URL (often used with `url_for()`).

Q118. What is the entry point of your backend?

Answer: `app.py` or `run.py` where the Flask app is created and run. In factory pattern, `create_app()` in `__init__.py`.

Q119. What is the purpose of `config.py` ?

Answer: Stores configuration (database URI, secret keys, mail settings, Celery config) for different environments.

Q120. What is the purpose of `__init__.py` ?

Answer: Marks a directory as a Python package and can initialize the package (e.g., create the Flask app, initialize extensions).

Q121. Can you export files as packages without `__init__.py` ?

Answer: In Python 3.3+, namespace packages allow imports without `__init__.py`, but it's not recommended for regular packages.

Q122. What is the directory structure of your app?

Answer: [Describe — e.g., `app.py`, `models.py`, `routes.py`, `tasks.py`, `config.py`, `templates/`, `static/`, `frontend/` with Vue files.]

Q123. What is the difference between `db.session.add` and `db.session.commit` ?

Answer: `add()` stages the object; `commit()` writes changes to the database.

Q124. If you delete the SQLite database file and restart, will it regenerate?

Answer: Only if `db.create_all()` is called on startup. Otherwise, no.

Q125. Will all tables get recreated?

Answer: Yes, if `db.create_all()` runs. It creates tables that don't exist but doesn't alter existing ones.

Q126. What is `db.create_all()` ?

Answer: Creates all tables defined by models in the database.

Q127. Where is `db.create_all()` called?

Answer: Usually in `app.py` after `db.init_app(app)` , often inside with `app.app_context(): db.create_all()` .

Q128. What is the default port for Flask?

Answer: 5000.

Q129. How to change port from 5000 to 8000?

Answer: `app.run(port=8000)` OR `flask run --port=8000` .

Q130. What is `__init__.py` ?

Answer: Initializes a Python package. Can contain app factory and extension setup.

Q131. What is the purpose of `requirements.txt` ?

Answer: Lists Python dependencies for the project. Install with

`pip install -r requirements.txt` .

Q132. What is a virtual environment?

Answer: An isolated Python environment with its own packages, preventing conflicts between projects.

Q133. Why do we need a virtual environment?

Answer: To isolate dependencies, avoid version conflicts, and keep the global Python clean.

Q134. Why would anyone use `venv`?

Answer: For dependency isolation, reproducibility, and to avoid polluting the system Python.

Q135. What is a `.env` file?

Answer: A file storing environment variables (API keys, DB URIs, secrets) not committed to version control.

Q136. What is the purpose of JWT secret key?

Answer: Used to sign and verify JWTs. Ensures tokens cannot be tampered with.

Q137. What is Flask secret key?

Answer: Used by Flask for session signing, CSRF protection, and other security features.

Q138. What is the difference between JWT secret key and Flask secret key?

Answer: JWT secret signs tokens; Flask secret signs sessions/cookies. They can be different.

Q139. How is JWT built?

Answer: Header (algorithm, type) + Payload (claims) + Signature (signed with secret). Encoded as Base64URL.

Q140. How does JWT work?

Answer: Server issues a signed token on login. Client stores it and sends it in the `Authorization` header. Server verifies the signature and extracts claims.

Q141. What is JWT?

Answer: JSON Web Token — a compact, URL-safe token for securely transmitting claims between parties.

Q142. What are the 3 parts of JWT?

Answer: Header, Payload, Signature.

Q143. Where is JWT stored?

Answer: Client-side: localStorage, sessionStorage, or cookies.

Q144. Where is token stored — client side or server side?

Answer: Client side (localStorage/sessionStorage/cookies). The server does not store tokens (stateless).

Q145. What is the default expiration time of authentication token?

Answer: Depends on config. Flask-JWT-Extended default is 15 minutes for access tokens.

Q146. How to set token expiration time?

Answer: `app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(hours=1) .`

Q147. What happens when token expires?

Answer: The server rejects the request with 401 Unauthorized. The client must re-authenticate.

Q148. How to handle token expiry?

Answer: Use refresh tokens, redirect to login, or prompt re-authentication.

Q149. What is `fs_uniquifier` ?

Answer: A unique identifier for each user in Flask-Security, used for session/token invalidation.

Q150. What is the difference between `fs_uniquifier` and `fs_uniquifier` tokens?

Answer: `fs_uniquifier` is a column; tokens are generated using it. When password changes, `fs_uniquifier` changes, invalidating old tokens.

Q151. What happens to `fs_uniquifier` token when a user changes password?

Answer: It is regenerated, invalidating all existing sessions/tokens.

Q152. What is Flask-Security?

Answer: An extension providing authentication, authorization, password hashing, roles, and session/token management.

Q153. What is `user_datastore` ?

Answer: An object in Flask-Security that manages users and roles (create, find, update).

Q154. What is `UserMixin` ?

Answer: A Flask-Login mixin providing default user methods (`is_authenticated` , `is_active` , `get_id`).

Q155. What is `RoleMixin` ?

Answer: A Flask-Security mixin for role models, providing `name` , `description` , and permissions.

Q156. What is RBAC?

Answer: Role-Based Access Control — permissions are assigned to roles, and users are assigned roles. Access is granted based on roles.

Q157. How is RBAC implemented?

Answer: Define roles (admin, user), assign roles to users, and check roles in decorators (`@roles_required('admin')`) before allowing access.

Q158. How have you implemented RBAC for admin?

Answer: Admin role is assigned to admin users. Routes are protected with `@roles_required('admin')` or a custom `@admin_required` decorator.

Q159. What is authentication vs authorization?

Answer:

- **Authentication:** Verifying identity (login).
- **Authorization:** Verifying permissions (what you can do).

Q160. What is the status code for unauthorized access?

Answer: 401 Unauthorized (not authenticated) or 403 Forbidden (authenticated but not allowed).

Q161. What is 400 vs 401 vs 403 vs 404?

Answer:

- **400:** Bad Request
- **401:** Unauthorized (not logged in)
- **403:** Forbidden (logged in but no permission)
- **404:** Not Found

Q162. What is 400, 401, 403, 404, 500?

Answer: Same as above; 500 is Internal Server Error.

Q163. What are different HTTP status codes?

Answer: 1xx (Info), 2xx (Success), 3xx (Redirection), 4xx (Client Error), 5xx (Server Error).

Q164. What is 201?

Answer: Created — resource successfully created.

Q165. What is 301?

Answer: Moved Permanently — permanent redirect.

Q166. What is 302?

Answer: Found — temporary redirect.

Q167. What is 500?

Answer: Internal Server Error.

Q168. What is 501?

Answer: Not Implemented — server doesn't support the functionality.

Q169. What is 410?

Answer: Gone — resource permanently deleted.

Q170. What is 303?

Answer: See Other — redirect with GET.

Q171. What is 405?

Answer: Method Not Allowed.

Q172. What is 505?

Answer: HTTP Version Not Supported.

Q173. What is 422?

Answer: Unprocessable Entity — validation error.

Q174. What is HTTP?

Answer: HyperText Transfer Protocol — application-layer protocol for transmitting web content.

Q175. What is HTTP vs HTTPS?

Answer: HTTPS is HTTP over TLS/SSL, encrypting data in transit.

Q176. What is HTTP/1.1?

Answer: Version 1.1 of HTTP, supporting persistent connections, pipelining, and chunked transfer.

Q177. What are HTTP methods?

Answer: GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS, TRACE, CONNECT.

Q178. What is GET vs POST?

Answer:

- **GET:** Retrieves data; parameters in URL; idempotent; cacheable.
- **POST:** Submits data; parameters in body; not idempotent; not cacheable.

Q179. What is PUT vs PATCH?

Answer:

- **PUT:** Replaces the entire resource.
- **PATCH:** Partially updates the resource.

Q180. What is POST vs PUT?

Answer:

- **POST:** Creates a new resource; not idempotent.
- **PUT:** Creates or replaces a resource at a known URI; idempotent.

Q181. Can we use POST instead of PUT?

Answer: Yes, but PUT is idempotent and semantically better for updates/replacements. POST is for creation.

Q182. Why is POST idempotent?

Answer: POST is **not** idempotent. PUT and DELETE are idempotent. (If asked, clarify: POST is non-idempotent.)

Q183. What is idempotency?

Answer: An operation is idempotent if performing it multiple times has the same effect as once. GET, PUT, DELETE are idempotent; POST is not.

Q184. What is REST?

Answer: Representational State Transfer — an architectural style for APIs using HTTP methods, statelessness, and resource-based URLs.

Q185. What is RESTful API?

Answer: An API that follows REST principles: stateless, client-server, cacheable, uniform interface, resource-based.

Q186. What are the principles of REST API?

Answer: Statelessness, client-server, cacheability, uniform interface, layered system, code-on-demand (optional).

Q187. What is an API?

Answer: Application Programming Interface — a set of rules allowing different software to communicate.

Q188. What is the difference between API and webhook?

Answer:

- **API:** Pull-based; client requests data.
- **Webhook:** Push-based; server sends data to a URL when an event occurs.

Q189. What is a webhook?

Answer: A user-defined HTTP callback triggered by an event. The server sends a POST request to a configured URL.

Q190. What is a webhook vs websocket?

Answer:

- **Webhook:** One-way, event-driven HTTP POST.
- **WebSocket:** Two-way, persistent connection for real-time communication.

Q191. What is a websocket?

Answer: A protocol providing full-duplex communication over a single TCP connection. Used for real-time apps (chat, live updates).

Q192. How does websocket work?

Answer: Client sends an HTTP upgrade request; server responds with 101 Switching Protocols. Then both can send messages freely.

Q193. What is Server-Sent Events (SSE)?

Answer: A one-way communication from server to client over HTTP. Used for live updates (stock prices, notifications).

Q194. What is SMTP?

Answer: Simple Mail Transfer Protocol — used for sending emails.

Q195. What is SMTP port?

Answer: 25 (default), 587 (TLS), 465 (SSL).

Q196. What is the default SMTP port?

Answer: 25.

Q197. What is the difference between SMTP and HTTP?

Answer: SMTP is for email; HTTP is for web. SMTP uses push; HTTP uses request-response.

Q198. What is MailHog?

Answer: A local email testing tool that captures outgoing emails without sending them. Used in development.

Q199. What is the configuration for MailHog?

Answer: SMTP host localhost , port 1025 , web UI at localhost:8025 .

Q200. What is the port for MailHog?

Answer: SMTP: 1025; Web UI: 8025.

Q201. What is the purpose of result backend in Redis config?

Answer: Stores the results of Celery tasks so they can be retrieved later.

Q202. What is the difference between broker URL and result backend?

Answer:

- **Broker URL:** Where tasks are sent (queue).
- **Result backend:** Where task results are stored.

Q203. What is `broker_url` ?

Answer: The URL of the message broker (Redis/RabbitMQ) used by Celery.

Q204. What is `result_backend` ?

Answer: The URL where Celery stores task results.

Q205. What is the difference between Redis and SQLite?

Answer:

- **Redis:** In-memory key-value store; fast; used for caching/broker.
- **SQLite:** File-based relational database; persistent; used for structured data.

Q206. What type of database is Redis?

Answer: NoSQL, key-value, in-memory database.

Q207. What type of database is SQLite?

Answer: Relational (RDBMS), file-based, serverless.

Q208. What is a relational database?

Answer: A database organized into tables with rows and columns, with relationships via keys.

Q209. What is a non-relational database?

Answer: A database not using tables/rows (e.g., document, key-value, graph).

Q210. What is NoSQL?

Answer: "Not only SQL" — databases that don't use the relational model. Examples: MongoDB, Redis, Cassandra.

Q211. Is NoSQL a relational database?

Answer: No.

Q212. What is the difference between relational and non-relational database?

Answer:

- **Relational:** Tables, rows, columns, ACID, SQL, fixed schema.
- **Non-relational:** Flexible schema, eventual consistency, horizontal scaling, various data models.

Q213. Give examples of non-relational databases.

Answer: MongoDB (document), Redis (key-value), Cassandra (column), Neo4j (graph).

Q214. What is the advantage of using relational databases?

Answer: ACID compliance, data integrity, powerful querying (SQL), relationships, maturity.

Q215. What is DBMS?

Answer: Database Management System — software to create, manage, and query databases (e.g., MySQL, PostgreSQL).

Q216. What is the difference between database and DBMS?

Answer: Database is the data collection; DBMS is the software managing it.

Q217. What is normalization?

Answer: Organizing data to reduce redundancy and improve integrity. Normal forms: 1NF, 2NF, 3NF, BCNF.

Q218. What is 1NF, 2NF, 3NF, BCNF?

Answer:

- **1NF:** Atomic values, no repeating groups.
- **2NF:** 1NF + no partial dependency on composite key.
- **3NF:** 2NF + no transitive dependency.
- **BCNF:** Every determinant is a candidate key.

Q219. What is denormalization?

Answer: Intentionally adding redundancy to improve read performance.

Q220. What is an ER diagram?

Answer: Entity-Relationship diagram — visual representation of entities, attributes, and relationships.

Q221. What is a JOIN?

Answer: Combines rows from two or more tables based on a related column.

Q222. What is LEFT JOIN?

Answer: Returns all rows from the left table and matching rows from the right; NULL if no match.

Q223. What is INNER JOIN?

Answer: Returns only rows with matches in both tables.

Q224. What is a subquery?

Answer: A query nested inside another query.

Q225. What are aggregate functions?

Answer: Functions like COUNT, SUM, AVG, MIN, MAX that operate on multiple rows.

Q226. What is GROUP BY?

Answer: Groups rows with the same values in specified columns, often used with aggregates.

Q227. What is HAVING?

Answer: Filters groups after GROUP BY (WHERE filters rows before grouping).

Q228. What is the difference between WHERE and HAVING?

Answer: WHERE filters rows before grouping; HAVING filters groups after grouping.

Q229. What is DELETE vs TRUNCATE?

Answer:

- **DELETE:** Removes rows; can use WHERE; logged; rollback possible.
- **TRUNCATE:** Removes all rows; faster; no WHERE; not logged per row; rollback depends on DB.

Q230. What is a trigger in DB?

Answer: A stored procedure automatically executed on INSERT/UPDATE/DELETE events.

Q231. What is an index?

Answer: A data structure (B-tree) that speeds up data retrieval.

Q232. What is a composite index?

Answer: An index on two or more columns.

Q233. What is a foreign key constraint?

Answer: Ensures a column value exists in the referenced table's primary key.

Q234. What is a unique constraint?

Answer: Ensures all values in a column are distinct.

Q235. What is a check constraint?

Answer: Ensures values meet a condition (e.g., $\text{age} > 0$).

Q236. What is a not null constraint?

Answer: Ensures a column cannot have NULL values.

Q237. What is a default constraint?

Answer: Sets a default value if none is provided.

Q238. What is a primary key?

Answer: A column (or set) uniquely identifying each row; NOT NULL; one per table.

Q239. What is a unique key?

Answer: A column (or set) with unique values; allows NULL; multiple per table.

Q240. What is a candidate key?

Answer: Any column that can uniquely identify a row.

Q241. What is a composite key?

Answer: A primary key made of multiple columns.

Q242. What is a surrogate key?

Answer: An artificial key (e.g., auto-increment ID) with no business meaning.

Q243. What is a natural key?

Answer: A key with business meaning (e.g., email, SSN).

Q244. What is a foreign key?

Answer: A column referencing another table's primary key.

Q245. What is referential integrity?

Answer: Ensures foreign key values match existing primary keys.

Q246. What is cascading?

Answer: Automatically propagating changes (delete/update) from parent to child rows.

Q247. What is ON DELETE CASCADE?

Answer: Deletes child rows when parent is deleted.

Q248. What is ON UPDATE CASCADE?

Answer: Updates child foreign keys when parent key changes.

Q249. What is the difference between ON DELETE and ON UPDATE?

Answer: ON DELETE triggers on deletion; ON UPDATE on update.

Q250. What happens if you don't use cascade?

Answer: Deletion may fail or leave orphan records.

Q251. What is the impact of deleting a record?

Answer: Depends on constraints — cascade, set null, restrict, or error.

Q252. What is lazy loading?

Answer: Loading related data only when accessed.

Q253. What is eager loading?

Answer: Loading related data immediately with the main query.

Q254. Difference between lazy loading and eager loading.

Answer: Lazy loads on demand; eager loads upfront. Eager avoids N+1 but may load unnecessary data.

Q255. When is eager loading better?

Answer: When you know you'll need related data for all fetched records, avoiding N+1 queries.

Q256. What is SPA?

Answer: Single Page Application — loads once and updates dynamically without full page reloads.

Q257. What is MPA?

Answer: Multi-Page Application — each navigation loads a new page from the server.

Q258. Difference between SPA and MPA.

Answer: SPA: faster navigation, client-side rendering, complex setup. MPA: simpler, server-rendered, full reloads.

Q259. What is PWA?

Answer: Progressive Web App — web app with native-like features (offline, push notifications, installable).

Q260. What is MVC?

Answer: Model-View-Controller — architectural pattern separating data (Model), UI (View), and logic (Controller).

Q261. What is the flow of MVC?

Answer: User → Controller → Model → Controller → View → User.

Q262. What architecture did you use?

Answer: [e.g., MVC with Flask (Model: SQLAlchemy, View: Vue, Controller: Flask routes).]

Q263. What is client-server architecture?

Answer: Clients request services; servers provide them. Centralized.

Q264. What is distributed architecture?

Answer: Multiple servers/nodes work together, often decentralized.

Q265. Difference between client-server and distributed architecture.

Answer: Client-server: centralized; distributed: multiple nodes, scalable, fault-tolerant.

Q266. What is scaling?

Answer: Increasing capacity to handle more load.

Q267. What is vertical scaling?

Answer: Adding more resources (CPU, RAM) to a single server.

Q268. What is horizontal scaling?

Answer: Adding more servers to distribute load.

Q269. Difference between vertical and horizontal scaling.

Answer: Vertical: scale up (limited); Horizontal: scale out (more flexible, fault-tolerant).

Q270. How would you scale your app for 10,000 users?

Answer: Load balancer, multiple app servers, database replication/sharding, caching (Redis), CDN, message queue (Celery), horizontal scaling.

Q271. How would you scale your app for 1 million users?

Answer: Microservices, distributed databases, sharding, CDN, auto-scaling, Kubernetes, monitoring, caching layers.

Q272. Is your application scalable?

Answer: [Yes/No — with reasons: e.g., "Partially. With Redis caching and Celery, it can handle moderate load, but for millions, I'd need load balancing and DB sharding."]

Q273. What is VPC?

Answer: Virtual Private Cloud — an isolated network within a cloud provider.

Q274. What is a man-in-the-middle attack?

Answer: An attacker intercepts communication between two parties. Prevented by HTTPS/TLS.

Q275. What is DDoS?

Answer: Distributed Denial of Service — overwhelming a server with traffic from many sources.

Q276. What is DoS?

Answer: Denial of Service — overwhelming a server from a single source.

Q277. What is SQL injection?

Answer: Injecting malicious SQL via user input. Prevented by parameterized queries/ORM.

Q278. What is XSS?

Answer: Cross-Site Scripting — injecting malicious scripts into web pages. Prevented by escaping output.

Q279. What is CSRF?

Answer: Cross-Site Request Forgery — tricking a user into unwanted actions. Prevented by CSRF tokens.

Q280. What is CORS?

Answer: Cross-Origin Resource Sharing — browser security for cross-origin requests. Configured via headers.

Q281. What is the difference between security and privacy?

Answer:

- **Security:** Protecting data from unauthorized access.
- **Privacy:** Controlling who can access personal data.

Q282. What is hashing?

Answer: Converting data into a fixed-length string using a one-way function. Cannot be reversed.

Q283. Why is password hashing required?

Answer: To store passwords securely. Even if the database leaks, passwords remain protected.

Q284. What was the default hashing algorithm in MAD1?

Answer: [e.g., SHA-256 or bcrypt — depends on your MAD1 project.]

Q285. What is SHA-256?

Answer: A cryptographic hash function producing a 256-bit hash. One-way.

Q286. Can SHA-256 be decrypted back?

Answer: No. Hashing is one-way. You can only compare hashes.

Q287. How many hashing algorithms do you know?

Answer: MD5, SHA-1, SHA-256, SHA-512, bcrypt, scrypt, Argon2.

Q288. What is a rainbow table?

Answer: A precomputed table of hashes for common passwords, used to crack hashes. Prevented by salting.

Q289. What is salting?

Answer: Adding random data to a password before hashing to prevent rainbow table attacks.

Q290. What is memoization?

Answer: Caching the result of a function based on its arguments. Avoids recomputation.

Q291. What is the difference between memoization and caching?

Answer: Memoization is caching function results based on arguments; caching is broader (HTTP responses, DB queries).

Q292. What is caching?

Answer: Storing frequently accessed data in fast storage (RAM) to reduce latency.

Q293. Why use caching?

Answer: Faster response, reduced database load, better scalability.

Q294. Where did you implement caching?

Answer: [e.g., On the search route, admin dashboard API, etc., using Flask-Caching with Redis.]

Q295. What is cached?

Answer: [e.g., Search results, user lists, dashboard statistics.]

Q296. When do we need to clear the cache?

Answer: When data changes (CRUD operations) to avoid stale data.

Q297. How to clear cache?

Answer: `cache.clear()` or `cache.delete(key)` .

Q298. How to clear cache for only one specific route?

Answer: Use `cache.delete(key)` with the specific key, or `@cache.cached` with a custom key prefix.

Q299. What is cache invalidation?

Answer: Removing or updating cached data when the source changes.

Q300. What is cache hit?

Answer: When requested data is found in the cache.

Q301. What is cache miss?

Answer: When requested data is not in the cache; must be fetched from source.

Q302. What is TTL?

Answer: Time To Live — how long cached data remains valid.

Q303. What is timeout in `@cache.cached(timeout)`?

Answer: The number of seconds the cached response is valid. After that, it's refreshed.

Q304. What is the difference between memcached and Redis?

Answer:

- **Memcached:** Simple key-value, no persistence, no data structures.
- **Redis:** Rich data structures, persistence, pub/sub, replication.

Q305. What is Redis?

Answer: An in-memory key-value store used for caching, message brokering, and real-time data.

Q306. Why is Redis faster?

Answer: Data is stored in RAM, not disk. No disk I/O for reads/writes.

Q307. Where does Redis store its cache?

Answer: In RAM. Optionally persisted to disk (RDB/AOF).

Q308. What data structure does Redis use?

Answer: Key-value, with values being strings, lists, sets, hashes, sorted sets, etc.

Q309. What is in-memory?

Answer: Data stored in RAM for fast access.

Q310. What is key-value pair?

Answer: A data model where each key maps to a value.

Q311. What is key prefix in cache vs memoize?

Answer: `cached` uses the request path as key; `memoize` uses function name + arguments.

Q312. What is the difference between cache and memoization?

Answer: Cache is general-purpose; memoization is caching function results based on arguments.

Q313. What is Celery?

Answer: A distributed task queue for running asynchronous and scheduled tasks in Python.

Q314. What is the need for Celery?

Answer: To offload long-running tasks (emails, reports) from the request-response cycle.

Q315. What are the components of Celery?

Answer: Producer (Flask), Broker (Redis), Worker (executes tasks), Result Backend (stores results), Beat (scheduler).

Q316. What is Celery Beat?

Answer: A scheduler that triggers periodic tasks.

Q317. What is Celery Worker?

Answer: A process that executes tasks from the queue.

Q318. Difference between Celery Beat and Celery Worker.

Answer: Beat schedules tasks; Worker executes them.

Q319. What is the difference between Celery worker and Celery beat?

Answer: Same as above — Beat schedules, Worker executes.

Q320. What is the purpose of beat?

Answer: To schedule periodic tasks (e.g., daily reminders).

Q321. What is the purpose of worker?

Answer: To execute tasks asynchronously.

Q322. What is a message broker?

Answer: Middleware that translates messages between services. Celery uses Redis/RabbitMQ.

Q323. What is a task queue?

Answer: A queue of tasks to be executed by workers.

Q324. What is a message queue?

Answer: A queue for asynchronous communication between services.

Q325. What is the difference between task queue and message queue?

Answer: Task queue is for executing tasks; message queue is for passing messages.

Q326. What is publish/subscribe?

Answer: A messaging pattern where publishers send messages to channels, and subscribers receive them.

Q327. What is point-to-point messaging?

Answer: A messaging pattern where one producer sends to one consumer via a queue.

Q328. Difference between pub/sub and point-to-point.

Answer: Pub/sub: one-to-many; point-to-point: one-to-one.

Q329. What is the difference between Redis and Celery?

Answer: Redis is a data store/broker; Celery is a task queue that uses Redis as a broker.

Q330. Why use Redis with Celery?

Answer: Redis is fast, supports pub/sub, and works well as a broker and result backend.

Q331. Why not use normal database instead of Redis?

Answer: Databases are slower for queue operations; Redis is in-memory and optimized for this.

Q332. How does Redis work with Celery?

Answer: Celery pushes tasks to Redis (broker); workers pull and execute them; results go to Redis (backend).

Q333. What is `.delay()` ?

Answer: A shortcut to send a task to the queue for asynchronous execution.

Q334. What is `.apply_async()` ?

Answer: A more flexible way to send tasks with options (countdown, eta, priority).

Q335. How to prioritize a task in Celery?

Answer: Use `priority` in `apply_async()` or define task queues with priorities.

Q336. How to schedule a task in Celery?

Answer: Use `beat_schedule` in Celery config with crontab or timedelta.

Q337. How to change crontab schedule?

Answer: Edit `beat_schedule` in `celeryconfig.py` OR `app.config`.

Q338. What is crontab?

Answer: A time-based job scheduler in Unix. Celery Beat uses crontab syntax.

Q339. Does crontab take seconds?

Answer: Standard crontab has minute granularity; Celery supports seconds via `timedelta`.

Q340. What is the minimum schedule in Celery?

Answer: 1 second (using `timedelta(seconds=1)`).

Q341. What is the default time for Celery Beat?

Answer: No default; you define schedules.

Q342. How to shutdown Celery task without terminal?

Answer: Use `celery control shutdown` or revoke tasks via `app.control.revoke(task_id)`.

Q343. What are the Celery jobs you implemented?

Answer: [e.g., Daily reminder email, monthly report generation, CSV export.]

Q344. Show Celery tasks.

Answer: [Open `tasks.py` and show `@celery.task` functions.]

Q345. Show monthly report emails.

Answer: [Show the task and the email sent via MailHog or inbox.]

Q346. Show daily reminder emails.

Answer: [Show the task and the email.]

Q347. How to send email using Celery?

Answer: Define a task with `@celery.task`, use `mail.send()` inside, and trigger via `.delay()`.

Q348. What is the flow of sending email via Celery?

Answer: Flask triggers task → Redis broker → Worker picks task → Sends email → Stores result.

Q349. What is the difference between Celery and cron job?

Answer: Celery is distributed, Python-based, supports async and scheduled tasks. Cron is OS-level, simple, no distributed support.

Q350. What is a cron job?

Answer: A scheduled task in Unix-like systems.

Q351. What is the difference between Celery async and Celery beat?

Answer: Async tasks run immediately on demand; Beat tasks run on a schedule.

Q352. What is the difference between scheduled and async tasks?

Answer: Scheduled run at specific times; async run immediately when triggered.

Q353. What happens if Celery worker crashes?

Answer: Tasks remain in the queue; when a worker restarts, it picks them up (if acknowledged).

Q354. How to handle long-running Celery tasks when user logs out?

Answer: Tasks run independently of the user session. Use task IDs to track and revoke if needed.

Q355. How to validate Celery task when token expires?

Answer: Tasks don't use tokens; they run server-side. Validate within the task if needed.

Q356. What is the default time of authentication token?

Answer: Depends on config; Flask-JWT-Extended default is 15 minutes.

3. DATABASE & MODELS

Q1. Explain your database schema.

Answer: [Describe tables: User, Role, Booking, Show, Venue, etc., and their relationships.]

Q2. Explain all models in your project.

Answer: [List each model, its columns, and relationships.]

Q3. How many tables do you have?

Answer: [e.g., 6 tables: User, Role, UserRoles, Booking, Show, Venue.]

Q4. Explain the relationships between tables.

Answer: [e.g., User has many Bookings; Booking belongs to Show; Show belongs to Venue.]

Q5. What is the relationship between User and Cart?

Answer: One-to-many (one user has one cart, cart has many items).

Q6. What is the relationship between Cart and Product?

Answer: Many-to-many via CartItems.

Q7. Is there any one-to-one relationship in your model?

Answer: [e.g., User and Profile — one-to-one.]

Q8. Explain the relationship between Songs, Albums, Playlists, Users.

Answer: User has many Playlists; Playlist has many Songs (many-to-many); Song belongs to Album; Album has many Songs.

Q9. What is the relationship between Book and Category?

Answer: Many-to-one (many books belong to one category).

Q10. What is the relationship between Trek and Staff?

Answer: Many-to-one (many treks assigned to one staff).

Q11. What is the relationship between Student, Company, User?

Answer: User is base; Student and Company are roles. Student applies to Company drives (many-to-many via Application).

Q12. How are company details stored in the user table?

Answer: [e.g., As a separate Company model linked to User via `user_id` .]

Q13. What is the difference between User and Student tables?

Answer: User stores authentication data; Student stores student-specific data (CGPA, branch).

Q14. Why did you use a separate table for RoleUsers?

Answer: For many-to-many relationship between User and Role.

Q15. Why did you use a separate table for category_request?

Answer: To track requests for new categories separately from approved categories.

Q16. Why did you use a separate table for favourites?

Answer: To model many-to-many between User and Item (e.g., songs, theatres).

Q17. How would you implement a favourite feature?

Answer: Create a `Favourite` table with `user_id` and `item_id` (many-to-many).

Q18. How would you implement a like/flag a song feature?

Answer: Add a `Like` table with `user_id` and `song_id`; toggle on click.

Q19. How would you implement a chat feature?

Answer: Use WebSockets (Flask-SocketIO) and a `Message` table with `sender_id`, `receiver_id`, `content`, `timestamp`.

Q20. How would you implement search suggestions?

Answer: Use `ilike` queries with `%` and return top matches; optionally use a search engine like Elasticsearch.

Q21. How would you implement a feature to send email whenever a new show is made?

Answer: Trigger a Celery task after show creation that sends an email to subscribers.

Q22. How would you implement a feature to prevent duplicate entries?

Answer: Add a unique constraint on the relevant column(s) and handle `IntegrityError`.

Q23. How would you implement a feature to prevent a student from applying twice?

Answer: Add a unique constraint on `(student_id, drive_id)` and check before insert.

Q24. How would you implement a feature to prevent a user from booking same slot?

Answer: Check for existing booking with same `user_id`, `show_id`, `time`; or add unique constraint.

Q25. How would you implement a feature to prevent overbooking?

Answer: Check available seats before booking; use transactions/locking to prevent race conditions.

Q26. How would you implement a feature to prevent negative values?

Answer: Add a check constraint (`CheckConstraint('price >= 0')`) and validate in backend/frontend.

Q27. How would you implement a feature to prevent past dates?

Answer: Validate in backend (if date < today: raise ValueError) and frontend (min date attribute).

Q28. How would you implement a feature to prevent invalid data?

Answer: Use validators, check constraints, and frontend/backend validation.

Q29. What is the impact of deleting a venue that has bookings?

Answer: Bookings become orphaned or deleted (if cascade). Better to restrict deletion or refund.

Q30. What happens to bookings if a movie is deleted?

Answer: Depends on cascade; either bookings are deleted or deletion is restricted.

Q31. What happens if you change seat = 1?

Answer: Existing bookings may exceed capacity; need validation or cancellation.

Q32. What happens to child records if parent is deleted?

Answer: Depends on cascade: deleted, set null, or restricted.

Q33. What if ON DELETE CASCADE is not used?

Answer: Deletion may fail or leave orphans.

Q34. What is referential integrity?

Answer: Foreign keys always reference valid primary keys.

Q35. How to maintain referential integrity?

Answer: Use foreign keys, constraints, and cascade rules.

Q36. What is a foreign key?

Answer: A column referencing another table's primary key.

Q37. How are foreign keys used in your models?

Answer: [e.g., `user_id = db.Column(db.Integer, db.ForeignKey('user.id'))` .]

Q38. What is a primary key?

Answer: Unique identifier for a row; NOT NULL.

Q39. What is a unique key?

Answer: Ensures uniqueness; allows NULL.

Q40. Difference between primary key and unique key.

Answer: Primary: NOT NULL, one per table. Unique: allows NULL, multiple per table.

Q41. Can unique key be used for making connections?

Answer: Yes, if it's NOT NULL and unique, it can be referenced by a foreign key.

Q42. What is a composite key?

Answer: Primary key made of multiple columns.

Q43. What is a candidate key?

Answer: Any column that can uniquely identify a row.

Q44. Difference between candidate key and primary key.

Answer: Candidate is any unique identifier; primary is the chosen one.

Q45. Can we use auto-increment instead of primary key?

Answer: Auto-increment alone doesn't guarantee uniqueness across all scenarios; primary key provides uniqueness and NOT NULL.

Q46. What is the last index value if autoincrement is on?

Answer: The last inserted ID.

Q47. If a record is deleted while autoincrement is on, what will be the next index value?

Answer: The next auto-increment value continues from the last used, not reused.

Q48. What is `__tablename__` ?

Answer: Explicitly sets the table name.

Q49. What is `__repr__` ?

Answer: String representation for debugging.

Q50. What is `__repr__` vs `__str__` ?

Answer: `__repr__` for developers; `__str__` for users.

Q51. What is `serialize` ?

Answer: Converting a model to a dict/JSON.

Q52. What is `to_dict()` ?

Answer: Custom method returning model fields as a dict.

Q53. What are dunder methods?

Answer: Special methods like `__init__` , `__repr__` .

Q54. What is `db.Model` ?

Answer: Base class for models.

Q55. How do you initialize a table?

Answer: Define a class inheriting `db.Model` with columns.

Q56. Where is the database initialized?

Answer: In `app.py` With `db = SQLAlchemy(app)` .

Q57. What is `db.create_all()` ?

Answer: Creates all tables.

Q58. Where is `db.create_all()` called?

Answer: In `app.py` inside app context.

Q59. If you delete the SQLite database file and restart, will it regenerate?

Answer: Only if `db.create_all()` is called.

Q60. Will all tables get recreated?

Answer: Yes, if `db.create_all()` runs.

Q61. What is `db.session.add()` vs `db.session.commit()` ?

Answer: `add()` stages; `commit()` persists.

Q62. What is `.all()` vs `.first()` ?

Answer: `.all()` returns list; `.first()` returns first or None.

Q63. What is `.one()` ?

Answer: Returns exactly one; errors otherwise.

Q64. What is `query.filter()` vs `filter_by()` ?

Answer: `filter()` uses expressions; `filter_by()` uses kwargs.

Q65. What is `like` vs `ilike` ?

Answer: `like` case-sensitive; `ilike` case-insensitive.

Q66. What is `%` in search?

Answer: Wildcard for pattern matching.

Q67. What is `joinedload` ?

Answer: Eager-loads related data in one query.

Q68. Why use `joinedload`?

Answer: Avoids N+1 queries.

Q69. What is N+1 query problem?

Answer: One query for parent + N queries for children.

Q70. What is `lazy=True` ?

Answer: Lazy loading (default).

Q71. What is `lazy='dynamic'` ?

Answer: Returns a query object for further filtering.

Q72. What is `backref` ?

Answer: Auto-creates reverse relationship.

Q73. What is `back_populates` ?

Answer: Explicitly defines both sides.

Q74. Difference between `backref` and `back_populates`.

Answer: `backref` one-sided; `back_populates` two-sided.

Q75. What is `cascade='all, delete-orphan'` ?

Answer: Cascade all operations; delete orphans.

Q76. What is `onDelete='CASCADE'` ?

Answer: DB-level cascade delete.

Q77. What is `uselist=False` ?

Answer: Makes relationship one-to-one.

Q78. What is `secondary` in relationship?

Answer: Association table for many-to-many.

Q79. What is `secondary="user_roles"` ?

Answer: Uses `user_roles` as junction table.

Q80. What is a self-referential relationship?

Answer: Model references itself (e.g., manager).

Q81. What is indexing?

Answer: Speeds up queries.

Q82. How to index in SQLAlchemy?

Answer: `index=True` on column.

Q83. What is index on two columns?

Answer: Composite index.

Q84. What is the purpose of index?

Answer: Faster SELECT.

Q85. What are SQL constraints?

Answer: NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, DEFAULT.

Q86. What is a check constraint?

Answer: Validates column values.

Q87. What is a not null constraint?

Answer: Disallows NULL.

Q88. What is a default constraint?

Answer: Sets default value.

Q89. What is the difference between String and Text?

Answer: String: limited length; Text: unlimited.

Q90. What is `db.String(128)` ? What does 128 mean?

Answer: VARCHAR(128); max 128 chars.

Q91. How is 128 stored in memory?

Answer: As a length prefix + variable-length string.

Q92. What is SQL injection?

Answer: Malicious SQL via input.

Q93. What is the difference between DELETE and TRUNCATE?

Answer: DELETE: row-by-row, WHERE, rollback. TRUNCATE: all rows, faster, no WHERE.

Q94. What is a JOIN?

Answer: Combines rows from tables.

Q95. What is LEFT JOIN?

Answer: All left rows + matching right.

Q96. What is INNER JOIN?

Answer: Only matching rows.

Q97. What is a subquery?

Answer: Nested query.

Q98. What are aggregate functions?

Answer: COUNT, SUM, AVG, MIN, MAX.

Q99. What is GROUP BY?

Answer: Groups rows for aggregation.

Q100. What is HAVING?

Answer: Filters groups.

Q101. Difference between WHERE and HAVING.

Answer: WHERE before grouping; HAVING after.

Q102. What is normalization?

Answer: Reducing redundancy.

Q103. What is 1NF, 2NF, 3NF, BCNF?

Answer: Progressive normalization forms.

Q104. What is denormalization?

Answer: Adding redundancy for performance.

Q105. What is an ER diagram?

Answer: Entity-Relationship diagram.

Q106. What is a relational database?

Answer: Tables with relationships.

Q107. What is a non-relational database?

Answer: No tables (document, key-value).

Q108. What is NoSQL?

Answer: Non-relational databases.

Q109. Is NoSQL a relational database?

Answer: No.

Q110. Difference between relational and non-relational database.

Answer: Relational: tables, ACID, SQL. Non-relational: flexible schema, eventual consistency.

Q111. Give examples of non-relational databases.

Answer: MongoDB, Redis, Cassandra, Neo4j.

Q112. What is DBMS?

Answer: Database Management System.

Q113. Difference between database and DBMS.

Answer: Database: data; DBMS: software.

Q114. What is the difference between SQLite and Redis?

Answer: SQLite: file-based RDBMS; Redis: in-memory key-value.

Q115. What type of database is SQLite?

Answer: Relational.

Q116. What type of database is Redis?

Answer: NoSQL, key-value, in-memory.

Q117. What is the difference between SQL and NoSQL?

Answer: SQL: tables, fixed schema, ACID. NoSQL: flexible, scalable, eventual consistency.

Q118. What is a trigger in DB?

Answer: Auto-executed procedure on events.

Q119. What is a view in DB?

Answer: Virtual table based on a query.

Q120. What is a stored procedure?

Answer: Precompiled SQL code stored in DB.

Q121. What is a transaction?

Answer: A unit of work that is atomic.

Q122. What is ACID?

Answer: Atomicity, Consistency, Isolation, Durability.

Q123. What is atomicity?

Answer: All or nothing.

Q124. What is consistency?

Answer: Data remains valid.

Q125. What is isolation?

Answer: Transactions don't interfere.

Q126. What is durability?

Answer: Committed data persists.

Q127. What is a deadlock?

Answer: Two transactions waiting for each other's locks.

Q128. What is a lock?

Answer: Mechanism to control concurrent access.

Q129. What is optimistic locking?

Answer: Check version before update.

Q130. What is pessimistic locking?

Answer: Lock rows before update.

Q131. What is sharding?

Answer: Partitioning data across servers.

Q132. What is replication?

Answer: Copying data to multiple servers.

Q133. What is partitioning?

Answer: Dividing a table into smaller parts.

Q134. What is a composite index?

Answer: Index on multiple columns.

Q135. What is a covering index?

Answer: Index containing all columns needed by a query.

Q136. What is a clustered index?

Answer: Determines physical order of rows.

Q137. What is a non-clustered index?

Answer: Separate structure pointing to rows.

Q138. Difference between clustered and non-clustered index.

Answer: Clustered: one per table, physical order. Non-clustered: multiple, separate structure.

Q139. What is a full-text index?

Answer: Index for text search.

Q140. What is a hash index?

Answer: Index using hash table.

Q141. What is a B-tree index?

Answer: Balanced tree index (default).

Q142. What is a bitmap index?

Answer: Index using bitmaps (low cardinality).

Q143. What is a spatial index?

Answer: Index for geometric data.

Q144. What is a unique index?

Answer: Ensures uniqueness.

Q145. What is a partial index?

Answer: Index on a subset of rows.

Q146. What is a functional index?

Answer: Index on an expression.

Q147. What is an expression index?

Answer: Same as functional index.

Q148. What is a covering index?

Answer: Index that covers all query columns.

Q149. What is index fragmentation?

Answer: Index becomes inefficient due to updates.

Q150. What is index rebuild?

Answer: Recreating an index to remove fragmentation.

Q151. What is index reorganization?

Answer: Defragmenting an index.

Q152. What is index statistics?

Answer: Metadata about index distribution.

Q153. What is query optimization?

Answer: Improving query performance.

Q154. What is query plan?

Answer: Steps the DB will take to execute a query.

Q155. What is EXPLAIN?

Answer: Command to show query plan.

Q156. What is a slow query log?

Answer: Log of queries exceeding a time threshold.

Q157. What is a database deadlock?

Answer: Transactions waiting for each other.

Q158. How to resolve deadlock?

Answer: Rollback one transaction, retry.

Q159. What is a long-running query?

Answer: Query taking too long.

Q160. How to kill a long-running query?

Answer: KILL QUERY OR pg_cancel_backend .

Q161. What is connection pooling?

Answer: Reusing DB connections.

Q162. What is a database connection?

Answer: Session between app and DB.

Q163. What is connection leak?

Answer: Not closing connections.

Q164. What is a transaction log?

Answer: Record of all changes.

Q165. What is WAL?

Answer: Write-Ahead Logging.

Q166. What is journaling?

Answer: Logging changes before applying.

Q167. What is checkpoint?

Answer: Flushing data to disk.

Q168. What is vacuum?

Answer: Reclaiming storage in PostgreSQL.

Q169. What is analyze?

Answer: Updating statistics.

Q170. What is reindex?

Answer: Rebuilding indexes.

Q171. What is cluster?

Answer: Grouping rows by index.

Q172. What is a materialized view?

Answer: Physically stored view.

Q173. What is a temporary table?

Answer: Table existing for a session.

Q174. What is a CTE?

Answer: Common Table Expression (WITH clause).

Q175. What is a window function?

Answer: Function over a window of rows.

Q176. What is a recursive query?

Answer: Query referencing itself.

Q177. What is a pivot table?

Answer: Reorienting data.

Q178. What is a cross join?

Answer: Cartesian product.

Q179. What is a self join?

Answer: Joining a table to itself.

Q180. What is a natural join?

Answer: Join on common columns.

Q181. What is a full outer join?

Answer: All rows from both tables.

Q182. What is a right join?

Answer: All right rows + matching left.

Q183. What is a left outer join?

Answer: Same as LEFT JOIN.

Q184. What is a union?

Answer: Combines results, removes duplicates.

Q185. What is union all?

Answer: Combines results, keeps duplicates.

Q186. What is intersect?

Answer: Common rows.

Q187. What is except?

Answer: Rows in first but not second.

Q188. What is a correlated subquery?

Answer: Subquery referencing outer query.

Q189. What is a scalar subquery?

Answer: Returns a single value.

Q190. What is a derived table?

Answer: Subquery in FROM clause.

Q191. What is a common table expression?

Answer: Named temporary result set.

Q192. What is a recursive CTE?

Answer: CTE referencing itself.

Q193. What is a lateral join?

Answer: Subquery referencing preceding tables.

Q194. What is a cross apply?

Answer: SQL Server lateral join.

Q195. What is an outer apply?

Answer: Like LEFT JOIN LATERAL.

Q196. What is a pivot?

Answer: Rows to columns.

Q197. What is an unpivot?

Answer: Columns to rows.

Q198. What is a rollup?

Answer: Subtotals and grand total.

Q199. What is a cube?

Answer: All combinations of subtotals.

Q200. What is grouping sets?

Answer: Multiple GROUP BY sets.

4. CACHING, REDIS & CELERY

Q1. What is caching?

Answer: Storing frequently accessed data in fast storage (RAM) to reduce latency.

Q2. Why do we use caching?

Answer: Faster response, reduced DB load, better scalability.

Q3. Where have you implemented caching?

Answer: [e.g., On search route, dashboard API, using Flask-Caching with Redis.]

Q4. Show where you implemented caching.

Answer: [Open the file and show `@cache.cached(timeout=50)` decorator.]

Q5. What did you cache?

Answer: [e.g., Search results, user lists, statistics.]

Q6. How did you implement caching?

Answer: Using Flask-Caching with Redis as backend.

Q7. What is the benefit of caching?

Answer: Speed, reduced load, scalability.

Q8. What is the timeout you set for caching?

Answer: [e.g., 50 seconds.]

Q9. How to change the caching timer?

Answer: Change the `timeout` parameter in `@cache.cached(timeout=...)` .

Q10. How to prove caching is working?

Answer: Add a print statement in the cached function; it prints only on first call.

Q11. How to clear the cache?

Answer: `cache.clear()` OR `cache.delete(key)` .

Q12. How to clear cache for only one route?

Answer: `cache.delete(key)` with the specific key.

Q13. What is cache invalidation?

Answer: Removing/updating cached data when source changes.

Q14. What is cache hit?

Answer: Data found in cache.

Q15. What is cache miss?

Answer: Data not in cache.

Q16. What is TTL?

Answer: Time To Live.

Q17. What is the difference between `cache.cache` and `cache.memoize`?

Answer: `cached` uses request path; `memoize` uses function arguments.

Q18. What is the difference between memoization and caching?

Answer: Memoization is caching function results; caching is broader.

Q19. What is memoization?

Answer: Caching function results based on arguments.

Q20. What is a key prefix in `cache` vs `memoize`?

Answer: `cached` : request path; `memoize` : function name + args.

Q21. What is Redis?

Answer: In-memory key-value store.

Q22. Why is Redis faster?

Answer: In-memory (RAM).

Q23. Where does Redis store its cache?

Answer: RAM (optionally persisted to disk).

Q24. What data structure does Redis use?

Answer: Key-value with strings, lists, sets, hashes, sorted sets.

Q25. What is in-memory?

Answer: Data in RAM.

Q26. What is key-value pair?

Answer: Key maps to value.

Q27. What is the difference between Redis and memcached?

Answer: Redis: rich data structures, persistence. Memcached: simple key-value.

Q28. What is the difference between Redis and SQLite?

Answer: Redis: in-memory NoSQL; SQLite: file-based RDBMS.

Q29. What is the difference between Redis and a normal database?

Answer: Redis: in-memory, fast, NoSQL. Normal DB: disk-based, persistent, relational.

Q30. Why use Redis instead of a normal database?

Answer: Speed, caching, message brokering.

Q31. Why use Redis for caching?

Answer: In-memory, fast, supports TTL.

Q32. Where is Redis hosted?

Answer: Locally on `localhost:6379` or a cloud service.

Q33. How is Redis running?

Answer: As a background service (`redis-server`).

Q34. If we stop Redis, will it affect Celery tasks?

Answer: Yes, Celery can't queue or execute tasks without the broker.

Q35. If we stop Flask app, will Redis still be running?

Answer: Yes, Redis runs independently.

Q36. How did you set up Celery and Redis?

Answer: Installed Redis, configured Celery with `broker_url='redis://localhost:6379/0'` , started worker and beat.

Q37. What specific commands did you run to start Redis?

Answer: `redis-server` .

Q38. What is the configuration for MailHog?

Answer: SMTP: `localhost:1025` , Web UI: `localhost:8025` .

Q39. What is the purpose of result backend in Redis config?

Answer: Stores Celery task results.

Q40. What is the difference between broker URL and result backend?

Answer: Broker: queue for tasks; Backend: stores results.

Q41. What is `broker_url` ?

Answer: URL of the message broker.

Q42. What is `result_backend` ?

Answer: URL for storing task results.

Q43. What is Celery?

Answer: Distributed task queue.

Q44. What is the need for Celery?

Answer: Offload long-running tasks.

Q45. What are the components of Celery?

Answer: Producer, Broker, Worker, Result Backend, Beat.

Q46. What is Celery Beat?

Answer: Scheduler for periodic tasks.

Q47. What is Celery Worker?

Answer: Executes tasks.

Q48. Difference between Celery Beat and Celery Worker.

Answer: Beat schedules; Worker executes.

Q49. What is the difference between Celery worker and Celery beat?

Answer: Same as above.

Q50. What is the purpose of beat?

Answer: Schedule periodic tasks.

Q51. What is the purpose of worker?

Answer: Execute tasks.

Q52. What is a message broker?

Answer: Middleware for message passing.

Q53. What is a task queue?

Answer: Queue of tasks.

Q54. What is a message queue?

Answer: Queue for messages.

Q55. What is the difference between task queue and message queue?

Answer: Task queue executes tasks; message queue passes messages.

Q56. What is publish/subscribe?

Answer: One-to-many messaging.

Q57. What is point-to-point messaging?

Answer: One-to-one messaging.

Q58. Difference between pub/sub and point-to-point.

Answer: Pub/sub: many subscribers; P2P: one consumer.

Q59. What is the difference between Redis and Celery?

Answer: Redis: data store/broker; Celery: task queue.

Q60. Why use Redis with Celery?

Answer: Fast, reliable broker.

Q61. Why not use normal database instead of Redis?

Answer: DB slower for queues.

Q62. How does Redis work with Celery?

Answer: Redis as broker and result backend.

Q63. What is `.delay()` ?

Answer: Sends task to queue.

Q64. What is `.apply_async()` ?

Answer: Flexible task sending.

Q65. How to prioritize a task in Celery?

Answer: `apply_async(priority=...)` .

Q66. How to schedule a task in Celery?

Answer: `beat_schedule` with crontab.

Q67. How to change crontab schedule?

Answer: Edit `beat_schedule` .

Q68. What is crontab?

Answer: Time-based scheduler.

Q69. Does crontab take seconds?

Answer: No, minute granularity.

Q70. What is the minimum schedule in Celery?

Answer: 1 second.

Q71. What is the default time for Celery Beat?

Answer: None; you define.

Q72. How to shutdown Celery task without terminal?

Answer: `celery control shutdown` or `revoke`.

Q73. What are the Celery jobs you implemented?

Answer: [Daily reminder, monthly report, CSV export.]

Q74. Show Celery tasks.

Answer: [Open `tasks.py`.]

Q75. Show monthly report emails.

Answer: [Show email.]

Q76. Show daily reminder emails.

Answer: [Show email.]

Q77. How to send email using Celery?

Answer: Task with `mail.send()` .

Q78. What is the flow of sending email via Celery?

Answer: Flask → Redis → Worker → Email.

Q79. What is the difference between Celery and cron job?

Answer: Celery: distributed, Python. Cron: OS-level.

Q80. What is a cron job?

Answer: Scheduled Unix task.

Q81. What is the difference between Celery async and Celery beat?

Answer: Async: on-demand; Beat: scheduled.

Q82. What is the difference between scheduled and async tasks?

Answer: Scheduled: at specific times; Async: immediately.

Q83. What happens if Celery worker crashes?

Answer: Tasks remain in queue; restart picks them up.

Q84. How to handle long-running Celery tasks when user logs out?

Answer: Tasks run independently; revoke if needed.

Q85. How to validate Celery task when token expires?

Answer: Tasks don't use tokens; validate internally.

Q86. What is the default time of authentication token?

Answer: 15 minutes (Flask-JWT-Extended default).

Q87. What is priority mail?

Answer: High-priority email task.

Q88. How to prioritize a task in Celery?

Answer: `priority` param in `apply_async` .

Q89. What is the priority of tasks?

Answer: Numeric priority (0-9).

Q90. How to set task priority?

Answer: `task.apply_async(priority=9)` .

Q91. What is task routing?

Answer: Sending tasks to specific queues.

Q92. What is task retry?

Answer: Retrying failed tasks.

Q93. What is task timeout?

Answer: Max time for task execution.

Q94. What is task result?

Answer: Output of a task.

Q95. What is task state?

Answer: PENDING, STARTED, SUCCESS, FAILURE.

Q96. What is task chain?

Answer: Sequential tasks.

Q97. What is task group?

Answer: Parallel tasks.

Q98. What is task chord?

Answer: Group + callback.

Q99. What is task signature?

Answer: Task invocation with args.

Q100. What is task canvas?

Answer: Workflow of tasks.

5. AUTHENTICATION & SECURITY

Q1. What is authentication?

Answer: Verifying identity.

Q2. What is authorization?

Answer: Verifying permissions.

Q3. Difference between authentication and authorization.

Answer: AuthN: who you are. AuthZ: what you can do.

Q4. Is Admin authorized or authenticated?

Answer: Both — authenticated (logged in) and authorized (has admin role).

Q5. How is authentication implemented?

Answer: JWT/Flask-Security.

Q6. How is authorization implemented?

Answer: RBAC with roles and decorators.

Q7. What is RBAC?

Answer: Role-Based Access Control.

Q8. How is RBAC implemented?

Answer: Roles assigned to users; decorators check roles.

Q9. How have you implemented RBAC for admin?

Answer: `@roles_required('admin')` or custom `@admin_required`.

Q10. How does `@admin_required` work?

Answer: Checks JWT and admin role.

Q11. How does `@role_required` work?

Answer: Checks if user has required role.

Q12. What is `@jwt_required`?

Answer: Ensures valid JWT.

Q13. Can we swap `@jwt_required` and `@role_required`?

Answer: No; JWT must be verified first.

Q14. What is JWT?

Answer: JSON Web Token.

Q15. How is JWT built?

Answer: Header + Payload + Signature.

Q16. What are the 3 parts of JWT?

Answer: Header, Payload, Signature.

Q17. Where is JWT stored?

Answer: localStorage/sessionStorage/cookies.

Q18. Where is token stored — client side or server side?

Answer: Client side.

Q19. What is the default expiration time of authentication token?

Answer: 15 minutes (Flask-JWT-Extended).

Q20. How to set token expiration time?

Answer: `JWT_ACCESS_TOKEN_EXPIRES` .

Q21. What happens when token expires?

Answer: 401 Unauthorized.

Q22. How to handle token expiry?

Answer: Refresh token or re-login.

Q23. What is fs_uniquifier?

Answer: Unique user identifier for token invalidation.

Q24. What is the difference between fs_uniquifier and fs_uniquifier tokens?

Answer: fs_uniquifier is a column; tokens are generated using it.

Q25. What happens to fs_uniquifier token when a user changes password?

Answer: Regenerated, invalidating old tokens.

Q26. What is Flask-Security?

Answer: Extension for auth, roles, password hashing.

Q27. What is user_datastore?

Answer: Manages users and roles.

Q28. What is UserMixin?

Answer: Provides default user methods.

Q29. What is RoleMixin?

Answer: Provides role methods.

Q30. What is Flask-Login?

Answer: Session-based authentication extension.

Q31. Is Flask-Login session-based or token-based?

Answer: Session-based.

Q32. What is session-based authentication?

Answer: Server stores session; client stores session ID cookie.

Q33. What is token-based authentication?

Answer: Server issues token; client sends it with each request.

Q34. Difference between session-based and token-based authentication.

Answer: Session: server-side state; Token: stateless.

Q35. Why token-based over session-based?

Answer: Stateless, scalable, works across domains.

Q36. What is a session?

Answer: Server-side storage of user state.

Q37. What is a cookie?

Answer: Client-side storage sent with requests.

Q38. Difference between session and cookie.

Answer: Session: server-side; Cookie: client-side.

Q39. Difference between local storage and session storage.

Answer: Local: persists; Session: cleared on tab close.

Q40. Difference between local storage and cookies.

Answer: Local: larger, not sent to server; Cookie: smaller, sent to server.

Q41. What is local storage?

Answer: Browser storage that persists.

Q42. What is session storage?

Answer: Browser storage for the session.

Q43. How many types of storage are there in browser?

Answer: localStorage, sessionStorage, cookies, IndexedDB, Cache API.

Q44. What is the advantage of local storage?

Answer: Persistence, larger size.

Q45. How do you clear local storage?

Answer: `localStorage.clear()` .

Q46. Where is local storage present?

Answer: In the browser (per origin).

Q47. What is the difference between local storage and Vuex store?

Answer: LocalStorage: persistent; Vuex: in-memory, reactive.

Q48. Why use Vuex store over local storage?

Answer: Reactivity, centralized state, devtools.

Q49. What happens when you hard reload with Vuex store?

Answer: State resets (unless persisted).

Q50. What is the status code for unauthorized access?

Answer: 401.

Q51. What is 401 vs 403?

Answer: 401: not authenticated; 403: authenticated but forbidden.

Q52. What is 400 vs 401?

Answer: 400: bad request; 401: unauthorized.

Q53. What is CSRF?

Answer: Cross-Site Request Forgery.

Q54. What is CORS?

Answer: Cross-Origin Resource Sharing.

Q55. What is XSS?

Answer: Cross-Site Scripting.

Q56. What is SQL injection?

Answer: Malicious SQL via input.

Q57. What is man-in-the-middle attack?

Answer: Intercepting communication.

Q58. What is DDoS?

Answer: Distributed Denial of Service.

Q59. What is DoS?

Answer: Denial of Service.

Q60. What is the difference between security and privacy?

Answer: Security: protection; Privacy: control over personal data.

Q61. What is hashing?

Answer: One-way conversion.

Q62. Why is password hashing required?

Answer: Secure storage.

Q63. What was the default hashing algorithm in MAD1?

Answer: [e.g., SHA-256/bcrypt.]

Q64. What is SHA-256?

Answer: 256-bit hash function.

Q65. Can SHA-256 be decrypted back?

Answer: No.

Q66. How many hashing algorithms do you know?

Answer: MD5, SHA-1, SHA-256, bcrypt, scrypt, Argon2.

Q67. What is a rainbow table?

Answer: Precomputed hash table.

Q68. What is salting?

Answer: Adding random data before hashing.

Q69. What is bcrypt?

Answer: Adaptive password hashing.

Q70. What is AES?

Answer: Symmetric encryption.

Q71. What is RSA?

Answer: Asymmetric encryption.

Q72. What is symmetric encryption?

Answer: Same key for encrypt/decrypt.

Q73. What is asymmetric encryption?

Answer: Public/private key pair.

Q74. What is SSL/TLS?

Answer: Encryption protocols for secure communication.

Q75. What is HTTPS?

Answer: HTTP over TLS.

Q76. What is a digital certificate?

Answer: Verifies server identity.

Q77. What is OAuth?

Answer: Authorization framework.

Q78. What is OpenID?

Answer: Authentication protocol.

Q79. What is SAML?

Answer: XML-based SSO.

Q80. What is SSO?

Answer: Single Sign-On.

Q81. What is MFA?

Answer: Multi-Factor Authentication.

Q82. What is 2FA?

Answer: Two-Factor Authentication.

Q83. What is a firewall?

Answer: Network security system.

Q84. What is a VPN?

Answer: Virtual Private Network.

Q85. What is a proxy?

Answer: Intermediary server.

Q86. What is a reverse proxy?

Answer: Proxy for servers.

Q87. What is a load balancer?

Answer: Distributes traffic.

Q88. What is rate limiting?

Answer: Limiting requests.

Q89. What is throttling?

Answer: Slowing requests.

Q90. What is input validation?

Answer: Checking user input.

Q91. What is output encoding?

Answer: Encoding output to prevent XSS.

Q92. What is sanitization?

Answer: Cleaning input.

Q93. What is escaping?

Answer: Escaping special characters.

Q94. What is a vulnerability?

Answer: Weakness.

Q95. What is a threat?

Answer: Potential attack.

Q96. What is a risk?

Answer: Probability × impact.

Q97. What is an exploit?

Answer: Attack using vulnerability.

Q98. What is a patch?

Answer: Fix for vulnerability.

Q99. What is a CVE?

Answer: Common Vulnerabilities and Exposures.

Q100. What is OWASP?

Answer: Open Web Application Security Project.

6. PROJECT IMPLEMENTATION & CODE CHANGES

Q1. Demonstrate your project.

Answer: [Show all features: login, CRUD, bookings, admin dashboard, etc.]

Q2. Show all features of the project.

Answer: [List and demo each feature.]

Q3. Explain the codebase.

Answer: [Walk through models, routes, tasks, Vue components.]

Q4. Explain the file structure.

Answer: [Describe folders and files.]

Q5. Explain the directory structure.

Answer: [Same as above.]

Q6. Explain models.py.

Answer: [Show each model and relationships.]

Q7. Explain app.py.

Answer: [Show app creation, extensions, blueprints.]

Q8. Explain routes.py.

Answer: [Show API endpoints.]

Q9. Explain api.py.

Answer: [Show API logic.]

Q10. Explain tasks.py.

Answer: [Show Celery tasks.]

Q11. Explain config.py.

Answer: [Show configuration.]

Q12. Explain main.py.

Answer: [Show entry point.]

Q13. Explain index.html.

Answer: [Show root div and script.]

Q14. Explain package.json.

Answer: [Show dependencies and scripts.]

Q15. Explain router.js.

Answer: [Show routes.]

Q16. Explain Vue files.

Answer: [Show components.]

Q17. Why do you have so many Vue files?

Answer: Modularity and reusability.

Q18. Difference between components and pages.

Answer: Components: reusable; Pages: route-level.

Q19. What did you like the most — frontend or backend?

Answer: [Your preference.]

Q20. What improvements did you make from MAD1 to MAD2?

Answer: [e.g., Vue SPA, Celery, Redis, RBAC, better UI.]

Q21. What did you learn from MAD1 and MAD2?

Answer: [List learnings.]

Q22. What was the most challenging part?

Answer: [e.g., Celery/Redis integration.]

Q23. What problems did you face while making the project?

Answer: [e.g., CORS, async tasks, DB relationships.]

Q24. How did you test your project?

Answer: [Manual testing, Postman, browser dev tools.]

Q25. How did you debug?

Answer: [Console logs, breakpoints, print statements.]

Q26. How can you improve the security of the application?

Answer: HTTPS, input validation, rate limiting, CSRF protection, secure headers.

Q27. What other features can you add?

Answer: [e.g., Payment gateway, notifications, chat.]

Q28. How would you rate your app out of 5?

Answer: [e.g., 4/5 with reasons.]

Q29. How would you rate your app out of 10?

Answer: [e.g., 8/10 with reasons.]

Q30. How much AI% did you use?

Answer: [Be honest — e.g., 20% for boilerplate/debugging.]

Q31. Where did you use AI?

Answer: [e.g., Debugging, boilerplate, CSS.]

Q32. Did you mention AI% in the report?

Answer: [Yes/No.]

Q33. Did you use ChatGPT?

Answer: [Yes/No.]

Q34. What are you doing besides this degree?

Answer: [Your answer.]

Q35. Why are you doing this degree?

Answer: [Your answer.]

Q36. Introduce yourself.

Answer: [Your introduction.]

Q37. What are your future plans?

Answer: [Your plans.]

Q38. Where do you see yourself in 5 years?

Answer: [Your answer.]

Q39. What grade did you get in MAD1?

Answer: [Your grade.]

Q40. What did you score in theory?

Answer: [Your score.]

Q41. Show your GitHub repository.

Answer: [Open GitHub.]

Q42. Show your collaborators.

Answer: [Show settings.]

Q43. Show your commit history.

Answer: [Show commits.]

Q44. How many commits did you make?

Answer: [Number.]

Q45. How long did it take to create this app?

Answer: [e.g., 2 weeks.]

Q46. Show your checksum.

Answer: [Show checksum file.]

Q47. Download from the portal.

Answer: [Download and run.]

Q48. Run the checksum.

Answer: [Run and verify.]

Q49. Show your ID card.

Answer: [Show ID.]

Q50. Share your screen.

Answer: [Share screen.]

Q51. Turn on your camera.

Answer: [Turn on camera.]

7. CODE CHANGES / LIVE CODING

Q1. Change the background color of the page.

Answer: In Vue: `<div :style="{ backgroundColor: 'red' }">` or CSS
`body { background-color: red; } .`

Q2. Change the background color to red.

Answer: `background-color: red;` in CSS or `:style="{ backgroundColor: 'red' }"` .

Q3. Change the text color to blue.

Answer: `color: blue;` OR `:style="{ color: 'blue' }"` .

Q4. Change the text color to purple.

Answer: `color: purple;` .

Q5. Change the font color.

Answer: `color: <value>;` .

Q6. Change the font size.

Answer: `font-size: 20px;` .

Q7. Change the button color.

Answer: `background-color: green;` on the button.

Q8. Change the button color to green.

Answer: `background-color: green; .`

Q9. Change the button color to red.

Answer: `background-color: red; .`

Q10. Change the button color to yellow.

Answer: `background-color: yellow; .`

Q11. Change the navbar color.

Answer: `background-color: blue; on navbar.`

Q12. Change the navbar position.

Answer: Use CSS `position: fixed; bottom: 0; .`

Q13. Move the navbar to the bottom.

Answer: `position: fixed; bottom: 0; width: 100%; .`

Q14. Move the navbar to the center.

Answer: `display: flex; justify-content: center; .`

Q15. Move the navbar to the right side.

Answer: `display: flex; justify-content: flex-end; .`

Q16. Move the navbar to the middle of the screen.

Answer: `position: fixed; top: 50%; transform: translateY(-50%);` .

Q17. Interchange navbar and footer.

Answer: Move navbar code below footer in template or use CSS `order` .

Q18. Move the search bar to the bottom.

Answer: `position: fixed; bottom: 0;` .

Q19. Move the search bar to the right side.

Answer: `float: right;` OR `margin-left: auto;` .

Q20. Move the login form to the center.

Answer: `display: flex; justify-content: center; align-items: center; height: 100vh;` .

Q21. Center the login form vertically and horizontally.

Answer: Same as above.

Q22. Shift the username and password fields to the center.

Answer: Wrap in a flex container with `justify-content: center;` .

Q23. Move the register button to the top-right corner.

Answer: `position: absolute; top: 0; right: 0;` .

Q24. Move the register button below the navbar.

Answer: Place it after navbar in template.

Q25. Move the sidebar to the right side.

Answer: `float: right;` or `flex order: 2;`

Q26. Remove the Actions column from the Registered Companies table.

Answer: Delete the `<th>Actions</th>` and corresponding `<td>`.

Q27. Show only 2 doctors in the list.

Answer: `v-for="doctor in doctors.slice(0, 2)"`.

Q28. Show only 2 appointments.

Answer: `appointments.slice(0, 2)`.

Q29. Show only 1 appointment.

Answer: `appointments.slice(0, 1)`.

Q30. Show only cancelled appointments.

Answer: `appointments.filter(a => a.status === 'cancelled')`.

Q31. Show only confirmed bookings.

Answer: `bookings.filter(b => b.status === 'confirmed') .`

Q32. Show only placed students.

Answer: `students.filter(s => s.status === 'placed') .`

Q33. Show only students with CGPA > 7.

Answer: `students.filter(s => s.cgpa > 7) .`

Q34. Show only active staff.

Answer: `staff.filter(s => s.active) .`

Q35. Show only inactive students.

Answer: `students.filter(s => !s.active) .`

Q36. Show only blacklisted users.

Answer: `users.filter(u => u.blacklisted) .`

Q37. Show only deleted items.

Answer: `items.filter(i => i.deleted) .`

Q38. Add a checkbox named "deleted".

Answer: `<input type="checkbox" v-model="showDeleted"> .`

Q39. Add a column to the table.

Answer: Add `<th>` and `<td>` in the table.

Q40. Add a new column of timestamp.

Answer: `timestamp = db.Column(db.DateTime, default=datetime.utcnow) .`

Q41. Add a new column of amount_received.

Answer: `amount_received = db.Column(db.Float) .`

Q42. Add a new column of email.

Answer: `email = db.Column(db.String, unique=True, nullable=False) .`

Q43. Add a new column of roll number.

Answer: `roll_number = db.Column(db.String) .`

Q44. Add a new column of emergency contact.

Answer: `emergency_contact = db.Column(db.String) .`

Q45. Add a new dropdown field.

Answer: `<select v-model="selected"><option>...</option></select> .`

Q46. Add a new dropdown for age.

Answer: Same as above with age options.

Q47. Add a new button.

Answer: `<button @click="handler">Click</button> .`

Q48. Add a delete button.

Answer: `<button @click="deleteItem(item.id)">Delete</button> .`

Q49. Add a cancel appointment button.

Answer: `<button @click="cancelAppointment(id)">Cancel</button> .`

Q50. Add a download CSV button.

Answer: `<button @click="downloadCSV">Download</button> .`

Q51. Add a button to show roll number.

Answer: `<button @click="showRoll">{{ rollNumber }}</button> .`

Q52. Add a button to redirect to student history.

Answer: `<button @click="$router.push('/history')">History</button> .`

Q53. Add a button to fetch a message from API.

Answer: `<button @click="fetchMessage">Fetch</button> with
fetch('/api/hello').then(...)` .

Q54. Add a button to fetch the message again.

Answer: Same button can re-fetch.

Q55. Add a button to hide the login button.

Answer: `<button @click="showLogin = false">Hide</button> .`

Q56. Add a button to show/hide paragraph.

Answer: `<button @click="show = !show">Toggle</button><p v-if="show">Text</p> .`

Q57. Add a button to toggle between Hello and GoodBye.

Answer:

`<button @click="msg = msg === 'Hello' ? 'GoodBye' : 'Hello'">{{ msg }}</button> .`

Q58. Add a button to increment count.

Answer: `<button @click="count++">{{ count }}</button> .`

Q59. Add a button that says "Hi" when clicked.

Answer: `<button @click="alert('Hi')">Click</button> .`

Q60. Add a button to submit.

Answer: `<button type="submit">Submit</button> .`

Q61. Add a button to toggle color.

Answer: `<button @click="color = color === 'red' ? 'blue' : 'red'">Toggle</button> .`

Q62. Add a button to select color (red, blue, green).

Answer: Radio buttons or checkboxes with `v-model` .

Q63. Add a submit button that changes color.

Answer: `<button :style="{ backgroundColor: selectedColor }">Submit</button>` .

Q64. Add a login button.

Answer: `<button @click="login">Login</button>` .

Q65. Add a signup button.

Answer: `<button @click="signup">Signup</button>` .

Q66. Add a logout button.

Answer: `<button @click="logout">Logout</button>` .

Q67. Add a search bar.

Answer: `<input v-model="query" @input="search">` .

Q68. Add a filter option.

Answer: `<select v-model="filter"><option>...</option></select>` .

Q69. Add a search functionality.

Answer: Backend: `Model.query.filter(Model.name.ilike(f'%{q}%')).all()` .

Q70. Add a search by username.

Answer: Same as above on username column.

Q71. Add a search by email.

Answer: Same on email column.

Q72. Add a search by name.

Answer: Same on name column.

Q73. Add a welcome message.

Answer: `<h1>Welcome, {{ username }}</h1> .`

Q74. Add a welcome message when user logs in.

Answer: Show after login using `v-if="loggedIn" .`

Q75. Add a welcome message under the search box.

Answer: `<p v-if="query === 'IITM'">Welcome!</p> .`

Q76. Add an alert message.

Answer: `alert('Message')` or a custom modal.

Q77. Add a confirmation dialog.

Answer: `if (confirm('Are you sure?')) { ... } .`

Q78. Add a loading state.

Answer: `v-if="loading"` with a spinner.

Q79. Add a shimmer effect.

Answer: CSS animation with gradient.

Q80. Add a table instead of card view.

Answer: Replace card markup with `<table>` .

Q81. Add a new tab.

Answer: Add a new route and nav link.

Q82. Add a new page.

Answer: Create a new `.vue` file and route.

Q83. Add a new view.

Answer: Same as page.

Q84. Add a new component.

Answer: Create a `.vue` file and import it.

Q85. Add a new route.

Answer: Add to `router.js` and create the view.

Q86. Add a new API endpoint.

Answer: `@app.route('/new', methods=['GET'])` .

Q87. Add a new model.

Answer: Define a class inheriting `db.Model` .

Q88. Add a new table.

Answer: Same as model; run `db.create_all()` .

Q89. Add a new role.

Answer: Insert into Role table and assign to users.

Q90. Add a new user.

Answer: Insert into User table.

Q91. Add a new venue.

Answer: Insert into Venue table.

Q92. Add a new show.

Answer: Insert into Show table.

Q93. Add a new product.

Answer: Insert into Product table.

Q94. Add a new category.

Answer: Insert into Category table.

Q95. Add a new campaign.

Answer: Insert into Campaign table.

Q96. Add a new drive.

Answer: Insert into Drive table.

Q97. Add a new trek.

Answer: Insert into Trek table.

Q98. Add a new staff.

Answer: Insert into Staff table.

Q99. Add a new doctor.

Answer: Insert into Doctor table.

Q100. Add a new patient.

Answer: Insert into Patient table.

Q101. Register a new company.

Answer: Use the registration form/API.

Q102. Register a new student.

Answer: Same.

Q103. Register a new user.

Answer: Same.

Q104. Create a new drive.

Answer: Use the create drive form/API.

Q105. Create a new show.

Answer: Use the create show form/API.

Q106. Create a new venue.

Answer: Use the create venue form/API.

Q107. Create a new movie.

Answer: Use the create movie form/API.

Q108. Create a new product.

Answer: Use the create product form/API.

Q109. Create a new booking.

Answer: Use the booking form/API.

Q110. Create a new appointment.

Answer: Use the appointment form/API.

Q111. Book a ticket.

Answer: Select show, seats, confirm.

Q112. Book a show.

Answer: Same.

Q113. Book a trek.

Answer: Select trek, confirm.

Q114. Book a slot.

Answer: Select slot, confirm.

Q115. Cancel a booking.

Answer: Call cancel API.

Q116. Cancel an appointment.

Answer: Call cancel API.

Q117. Delete a venue.

Answer: Call delete API.

Q118. Delete a movie.

Answer: Call delete API.

Q119. Delete a show.

Answer: Call delete API.

Q120. Delete a product.

Answer: Call delete API.

Q121. Delete a user.

Answer: Call delete API.

Q122. Delete a doctor.

Answer: Call delete API.

Q123. Delete a staff.

Answer: Call delete API.

Q124. Delete a lot.

Answer: Call delete API.

Q125. Blacklist a user.

Answer: Set `blacklisted = True` .

Q126. Blacklist a doctor.

Answer: Same.

Q127. Blacklist a company.

Answer: Same.

Q128. Blacklist a staff.

Answer: Same.

Q129. Approve a company.

Answer: Set `approved = True` .

Q130. Approve a drive.

Answer: Set `approved = True` .

Q131. Approve a booking.

Answer: Set `status = 'approved'` .

Q132. Approve a student.

Answer: Set `approved = True` .

Q133. Deactivate a company.

Answer: Set `active = False` .

Q134. Deactivate a staff.

Answer: Same.

Q135. Deactivate a user.

Answer: Same.

Q136. Export CSV.

Answer: Use `csv` module or `pandas` to generate and send.

Q137. Export CSV for all venues.

Answer: Loop through venues and write to one CSV.

Q138. Export CSV for all theatres.

Answer: Same.

Q139. Export report.

Answer: Generate report and send via email.

Q140. Generate monthly report.

Answer: Celery task that compiles data and emails it.

Q141. Generate daily reminder.

Answer: Celery task that sends reminders.

Q142. Send email.

Answer: `mail.send(Message(...))` .

Q143. Schedule email.

Answer: Celery Beat with crontab.

Q144. Change the schedule of email.

Answer: Edit `beat_schedule` .

Q145. Change crontab timing.

Answer: `crontab(minute='*/5')` .

Q146. Change celery beat timing.

Answer: Same.

Q147. Change the timing to 1 minute.

Answer: `crontab(minute='*/1')` Or `timedelta(minutes=1)` .

Q148. Change the timing to 30 seconds.

Answer: `timedelta(seconds=30)` .

Q149. Change the timing to 10 minutes.

Answer: `timedelta(minutes=10)` .

Q150. Change the timing to 5:20 PM.

Answer: `crontab(hour=17, minute=20)` .

Q151. Change the timing to 6:52.

Answer: `crontab(hour=18, minute=52)` .

Q152. Change the priority of tasks.

Answer: `apply_async(priority=...)` .

Q153. Change the database.

Answer: Update `SQLALCHEMY_DATABASE_URI` and run migrations.

Q154. Change the port from 5000 to 8000.

Answer: `app.run(port=8000) .`

Q155. Change the background image.

Answer: `background-image: url('...'); .`

Q156. Change the background image to blue color.

Answer: `background-color: blue; .`

Q157. Change the background to an image.

Answer: `background-image: url('image.jpg'); .`

Q158. Change the background to red.

Answer: `background-color: red; .`

Q159. Change the font to blue.

Answer: `color: blue; .`

Q160. Change the font to purple.

Answer: `color: purple; .`

Q161. Change the font to yellow.

Answer: `color: yellow; .`

Q162. Change the font size.

Answer: `font-size: 20px; .`

Q163. Change the heading.

Answer: Edit the `<h1>` text.

Q164. Change the heading from 'List of Songs' to 'Songs'.

Answer: Edit the text.

Q165. Change the title.

Answer: Edit `<title>` or heading.

Q166. Change the app name.

Answer: Edit `name` in `package.json` or app title.

Q167. Change the Vue app name to app2.

Answer: Edit in `package.json` OR `main.js` .

Q168. Change the message.

Answer: Edit the string in the component.

Q169. Change the button text.

Answer: Edit the text inside `<button>` .

Q170. Change the button color to pink.

Answer: `background-color: pink;` .

Q171. Change the logout button to pink.

Answer: Same.

Q172. Change the navbar background color to red.

Answer: `background-color: red;` .

Q173. Change the sidebar color.

Answer: `background-color: ...;` on sidebar.

Q174. Change the table row color.

Answer: `tr { background-color: ...; }` .

Q175. Change the status color.

Answer: Conditional class based on status.

Q176. Change the chart color.

Answer: Update `chart.js` options.

Q177. Change the bar chart to stacked bar chart.

Answer: Set `stacked: true` in chart options.

Q178. Change the stacked bar chart to simple bar chart.

Answer: Set `stacked: false` .

Q179. Change the chart to take only 2 colors.

Answer: Limit `backgroundColor` array to 2 colors.

Q180. Change the color of the bars.

Answer: Update `backgroundColor` .

Q181. Change the color of the login box.

Answer: `background-color: ...; .`

Q182. Change the color of the login button.

Answer: `background-color: ...; .`

Q183. Change the color of the form.

Answer: `background-color: ...; .`

Q184. Change the color of the box.

Answer: Same.

Q185. Change the color of the text.

Answer: `color: ...; .`

Q186. Change the color of the button.

Answer: `background-color: ...; .`

Q187. Change the color of the navbar.

Answer: Same.

Q188. Change the color of the sidebar.

Answer: Same.

Q189. Change the color of the font.

Answer: `color: ...; .`

Q190. Change the color of the background.

Answer: `background-color: ...; .`

Q191. Change the color of the page.

Answer: Same.

Q192. Change the color of the div.

Answer: Same.

Q193. Change the color of the table.

Answer: Same.

Q194. Change the color of the card.

Answer: Same.

Q195. Change the color of the modal.

Answer: Same.

Q196. Change the color of the header.

Answer: Same.

Q197. Change the color of the footer.

Answer: Same.

Q198. Change the color of the search bar.

Answer: Same.

Q199. Change the color of the button to green.

Answer: background-color: green; .

Q200. Change the color of the button to red.

Answer: background-color: red; .

8. CODING QUESTIONS (WRITE CODE)

Q1. Write a hello world route.

Answer:

python

```
@app.route('/hello')
def hello():
    return 'Hello, World!'
```

Q2. Write a route to add two numbers.

Answer:

python

```
@app.route('/add/<int:a>/<int:b>')
def add(a, b):
    return jsonify({'sum': a + b})
```

Q3. Write a route to add two integers from URL.

Answer: Same as above.

Q4. Write a route to return sum of two numbers.

Answer: Same.

Q5. Write a route to return square of a number.

Answer:

python

```
@app.route('/square/<int:n>')
def square(n):
    return jsonify({'square': n * n})
```

Q6. Write a route to return cube of a number.

Answer:

python

```
@app.route('/cube/<int:n>')
def cube(n):
    return jsonify({'cube': n ** 3})
```

Q7. Write a route to check if a number is prime.

Answer:

python

```
@app.route('/prime/<int:n>')
def prime(n):
    if n < 2:
        return jsonify({'prime': False})
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return jsonify({'prime': False})
    return jsonify({'prime': True})
```

Q8. Write a route to check if a number is even or odd.

Answer:

python

```
@app.route('/evenodd/<int:n>')
def evenodd(n):
    return jsonify({'result': 'even' if n % 2 == 0 else 'odd'})
```

Q9. Write a route for Fibonacci series.

Answer:

python

```
@app.route('/fib/<int:a>/<int:b>')
def fib(a, b):
    seq = [a, b]
    for _ in range(8):
        seq.append(seq[-1] + seq[-2])
    return jsonify(seq)
```

Q10. Write a route to return current timestamp.

Answer:

python

```
from datetime import datetime
@app.route('/time')
def time():
    return jsonify({'time': datetime.now().isoformat()})
```

Q11. Write a route to return today's date + 5 days.

Answer:

python

```
from datetime import datetime, timedelta
@app.route('/date')
```

```
def date():  
    return jsonify({'date': (datetime.now() + timedelta(days=5)).isoformat()})
```

Q12. Write a route to return "hello world".

Answer: Same as Q1.

Q13. Write a route to return success.

Answer:

python

```
@app.route('/success')  
def success():  
    return jsonify({'status': 'success'})
```

Q14. Write a login route.

Answer:

python

```
@app.route('/login', methods=['POST'])  
def login():  
    data = request.get_json()  
    user = User.query.filter_by(email=data['email']).first()  
    if user and check_password_hash(user.password, data['password']):  
        token = create_access_token(identity=user.id)  
        return jsonify({'token': token}), 200  
    return jsonify({'error': 'Invalid credentials'}), 401
```

Q15. Write a signup route.

Answer:

python

```
@app.route('/signup', methods=['POST'])
def signup():
    data = request.get_json()
    hashed = generate_password_hash(data['password'])
    user = User(email=data['email'], password=hashed)
    db.session.add(user)
    db.session.commit()
    return jsonify({'message': 'User created'}), 201
```

Q16. Write a registration route.

Answer: Same as signup.

Q17. Write a route to fetch all users.

Answer:

python

```
@app.route('/users')
def users():
    return jsonify([u.to_dict() for u in User.query.all()])
```

Q18. Write a route to fetch all students.

Answer:

python

```
@app.route('/students')
def students():
    return jsonify([s.to_dict() for s in Student.query.all()])
```

Q19. Write a route to fetch all doctors.

Answer:

python

```
@app.route('/doctors')
def doctors():
    return jsonify([d.to_dict() for d in Doctor.query.all()])
```

Q20. Write a route to fetch all companies.**Answer:**

python

```
@app.route('/companies')
def companies():
    return jsonify([c.to_dict() for c in Company.query.all()])
```

Q21. Write a route to fetch all drives.**Answer:**

python

```
@app.route('/drives')
def drives():
    return jsonify([d.to_dict() for d in Drive.query.all()])
```

Q22. Write a route to fetch all treks.**Answer:**

python

```
@app.route('/treks')
def treks():
    return jsonify([t.to_dict() for t in Trek.query.all()])
```

Q23. Write a route to fetch all blacklisted users.

Answer:

python

```
@app.route('/blacklisted')
def blacklisted():
    return jsonify([u.to_dict() for u in User.query.filter_by(blacklisted=True).all()])
```

Q24. Write a route to fetch all blacklisted companies.

Answer:

python

```
@app.route('/blacklisted-companies')
def blacklisted_companies():
    return jsonify([c.to_dict() for c in Company.query.filter_by(blacklisted=True).all()])
```

Q25. Write a route to fetch all blacklisted staff.

Answer:

python

```
@app.route('/blacklisted-staff')
def blacklisted_staff():
    return jsonify([s.to_dict() for s in Staff.query.filter_by(blacklisted=True).all()])
```

Q26. Write a route to fetch all shortlisted students.

Answer:

python

```
@app.route('/shortlisted')
def shortlisted():
    return jsonify([s.to_dict() for s in Student.query.filter_by(status='shortlisted').all()])
```

Q27. Write a route to fetch all approved students.

Answer:

python

```
@app.route('/approved')
def approved():
    return jsonify([s.to_dict() for s in Student.query.filter_by(approved=True).all()])
```

Q28. Write a route to fetch all unapproved drives.

Answer:

python

```
@app.route('/unapproved-drives')
def unapproved_drives():
    return jsonify([d.to_dict() for d in Drive.query.filter_by(approved=False).all()])
```

Q29. Write a route to fetch all rejected drives.

Answer:

python

```
@app.route('/rejected-drives')
def rejected_drives():
    return jsonify([d.to_dict() for d in Drive.query.filter_by(status='rejected').all()])
```

Q30. Write a route to fetch all completed treks.

Answer:

python

```
@app.route('/completed-treks')
def completed_treks():
    return jsonify([t.to_dict() for t in Trek.query.filter_by(status='completed').all()])
```

Q31. Write a route to fetch all fully booked treks.

Answer:

python

```
@app.route('/full-treks')
def full_treks():
    return jsonify([t.to_dict() for t in Trek.query.filter_by(available_slots=0).all()])
```

Q32. Write a route to fetch all easy treks.

Answer:

python

```
@app.route('/easy-treks')
def easy_treks():
    return jsonify([t.to_dict() for t in Trek.query.filter_by(difficulty='easy').all()])
```

Q33. Write a route to fetch all difficult treks.

Answer:

python

```
@app.route('/difficult-treks')
def difficult_treks():
```

```
return jsonify([t.to_dict() for t in Trek.query.filter_by(difficulty='difficult').all()])
```

Q34. Write a route to fetch all treks assigned to a staff.

Answer:

python

```
@app.route('/staff/<name>/treks')
def staff_treks(name):
    staff = Staff.query.filter_by(name=name).first()
    return jsonify([t.to_dict() for t in staff.treks])
```

Q35. Write a route to fetch all drives for a company.

Answer:

python

```
@app.route('/company/<name>/drives')
def company_drives(name):
    company = Company.query.filter_by(name=name).first()
    return jsonify([d.to_dict() for d in company.drives])
```

Q36. Write a route to fetch all users with role admin.

Answer:

python

```
@app.route('/admins')
def admins():
    return jsonify([u.to_dict() for u in User.query.filter(User.roles.any(name='admin')).all()])
```

Q37. Write a route to fetch all users who logged in past 15 minutes.

Answer:

python

```
from datetime import datetime, timedelta
@app.route('/recent-users')
def recent_users():
    cutoff = datetime.now() - timedelta(minutes=15)
    return jsonify([u.to_dict() for u in User.query.filter(User.last_login >= cutoff).all()])
```

Q38. Write a route to fetch all appointments in past 5 days.

Answer:

python

```
@app.route('/recent-appointments')
def recent_appointments():
    cutoff = datetime.now() - timedelta(days=5)
    return jsonify([a.to_dict() for a in Appointment.query.filter(Appointment.date >= cutoff).all()])
```

Q39. Write a route to fetch all reservations in past x days.

Answer:

python

```
@app.route('/reservations/<int:x>')
def reservations(x):
    cutoff = datetime.now() - timedelta(days=x)
    return jsonify([r.to_dict() for r in Reservation.query.filter(Reservation.date >= cutoff).all()])
```

Q40. Write a route to fetch all doctors of cardiology.

Answer:

python

```
@app.route('/cardiology-doctors')
def cardiology_doctors():
    return jsonify([d.to_dict() for d in Doctor.query.filter_by(department='cardiology').all()])
```

Q41. Write a route to fetch all drives with deadline.**Answer:**

python

```
@app.route('/drives-deadline')
def drives_deadline():
    return jsonify([d.to_dict() for d in Drive.query.filter(Drive.deadline >= datetime.now()).all()])
```

Q42. Write a route to fetch all drives where student can apply.**Answer:**

python

```
@app.route('/eligible-drives')
def eligible_drives():
    return jsonify([d.to_dict() for d in Drive.query.filter_by(approved=True).all()])
```

Q43. Write a route to fetch all drives where student is placed.**Answer:**

python

```
@app.route('/placed-drives')
def placed_drives():
```

```
return jsonify([d.to_dict() for d in Drive.query.filter_by(status='placed').all()])
```

Q44. Write a route to fetch all students with CGPA > 7.

Answer:

python

```
@app.route('/high-cgpa')
def high_cgpa():
    return jsonify([s.to_dict() for s in Student.query.filter(Student.cgpa > 7).all()])
```

Q45. Write a route to fetch all students with status placed.

Answer:

python

```
@app.route('/placed-students')
def placed_students():
    return jsonify([s.to_dict() for s in Student.query.filter_by(status='placed').all()])
```

Q46. Write a route to fetch all students with status shortlisted.

Answer: Same pattern.

Q47. Write a route to fetch all students with status applied.

Answer: Same pattern.

Q48. Write a route to fetch all students with status selected.

Answer: Same pattern.

Q49. Write a route to fetch all students with status rejected.

Answer: Same pattern.

Q50. Write a route to fetch all students with status pending.

Answer: Same pattern.

Q51. Write a route to fetch all students with status approved.

Answer: Same pattern.

Q52. Write a route to fetch all students with status blacklisted.

Answer: Same pattern.

Q53. Write a route to fetch all students with status active.

Answer: Same pattern.

Q54. Write a route to fetch all students with status inactive.

Answer: Same pattern.

Q55. Write a route to fetch all students with status deleted.

Answer: Same pattern.

Q56. Write a route to fetch all students with status placed.

Answer: Same pattern.

Q57. Write a route to fetch all students with status unplaced.

Answer: Same pattern.

Q58. Write a route to fetch all students with status offered.

Answer: Same pattern.

Q59. Write a route to fetch all students with status accepted.

Answer: Same pattern.

Q60. Write a route to fetch all students with status declined.

Answer: Same pattern.

Q61. Write a Vue component with a button.

Answer:

```
vue

<template>
  <button @click="handleClick">Click Me</button>
</template>
<script>
export default {
  methods: {
    handleClick() { alert('Clicked!'); }
  }
}
</script>
```

Q62. Write a Vue component with an input and a button.

Answer:

```
vue

<template>
  <input v-model="text" />
  <button @click="submit">Submit</button>
</template>
<script>
export default {
  data() { return { text: '' }; },
  methods: { submit() { console.log(this.text); } }
}
</script>
```

Q63. Write a Vue component with 2 inputs and a button.

Answer: Similar with two `v-model` bindings.

Q64. Write a Vue component with a toggle button.

Answer:

```
vue

<template>
  <button @click="show = !show">Toggle</button>
  <p v-if="show">Visible</p>
</template>
<script>
export default { data() { return { show: false }; } }
</script>
```

Q65. Write a Vue component with a submit button.

Answer: Same as Q62.

Q66. Write a Vue component with a checkbox.**Answer:**

```
vue

<template>
  <input type="checkbox" v-model="checked" />
</template>
<script>
export default { data() { return { checked: false }; } }
</script>
```

Q67. Write a Vue component with a dropdown.**Answer:**

```
vue

<template>
  <select v-model="selected">
    <option value="a">A</option>
    <option value="b">B</option>
  </select>
</template>
<script>
export default { data() { return { selected: 'a' }; } }
</script>
```

Q68. Write a Vue component with a date input.**Answer:**

```
vue

<template>
  <input type="date" v-model="date" />
</template>
<script>
export default { data() { return { date: '' }; } }
</script>
```

Q69. Write a Vue component with a form.**Answer:**

```
vue

<template>
  <form @submit.prevent="submit">
    <input v-model="name" />
    <button type="submit">Submit</button>
  </form>
</template>
<script>
export default {
  data() { return { name: '' }; },
  methods: { submit() { console.log(this.name); } }
}
</script>
```

Q70. Write a Vue component with v-for.**Answer:**

```
vue

<template>
  <ul>
    <li v-for="item in items" :key="item.id">{{ item.name }}</li>
  </ul>
</template>
<script>
export default { data() { return { items: [{id:1,name:'A'}, {id:2,name:'B'}] }; } }
</script>
```

Q71. Write a Vue component with v-if.**Answer:**

```
vue
```

```
<template>
  <p v-if="show">Hello</p>
</template>
<script>
export default { data() { return { show: true }; } }
</script>
```

Q72. Write a Vue component with v-model.

Answer: Same as Q62.

Q73. Write a Vue component with v-bind.

Answer:

vue

```
<template>
  
</template>
<script>
export default { data() { return { imageUrl: 'https://...' }; } }
</script>
```

Q74. Write a Vue component with v-show.

Answer:

vue

```
<template>
  <p v-show="show">Hello</p>
</template>
<script>
export default { data() { return { show: true }; } }
</script>
```

Q75. Write a Vue component with props.**Answer:**

```
vue

<!-- Child -->
<template><p>{{ message }}</p></template>
<script>export default { props: ['message'] }</script>

<!-- Parent -->
<Child message="Hello" />
```

Q76. Write a Vue component with emit.**Answer:**

```
vue

<!-- Child -->
<template><button @click="$emit('custom', 'data')">Click</button></template>

<!-- Parent -->
<Child @custom="handle" />
```

Q77. Write a Vue component with slots.**Answer:**

```
vue

<!-- Child -->
<template><slot></slot></template>

<!-- Parent -->
<Child><p>Content</p></Child>
```

Q78. Write a Vue component with computed.

Answer:

vue

```
<template><p>{{ fullName }}</p></template>
<script>
export default {
  data() { return { first: 'A', last: 'B' }; },
  computed: { fullName() { return this.first + ' ' + this.last; } }
}
</script>
```

Q79. Write a Vue component with watch.**Answer:**

vue

```
<script>
export default {
  data() { return { query: '' }; },
  watch: { query(newVal) { console.log(newVal); } }
}
</script>
```

Q80. Write a Vue component with mounted.**Answer:**

vue

```
<script>
export default {
  mounted() { console.log('Mounted'); }
}
</script>
```

Q81. Write a Vue component that fetches from API.

Answer:

vue

```
<script>
export default {
  data() { return { data: null }; },
  async mounted() {
    const res = await fetch('/api/data');
    this.data = await res.json();
  }
}
</script>
```

Q82. Write a Vue component that sends data to API.**Answer:**

vue

```
<script>
export default {
  methods: {
    async send() {
      await fetch('/api/data', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ name: 'A' })
      });
    }
  }
}
</script>
```

Q83. Write a Vue component that displays a list.

Answer: Same as Q70.

Q84. Write a Vue component that displays a table.

Answer:

vue

```
<template>
  <table>
    <tr v-for="item in items" :key="item.id">
      <td>{{ item.name }}</td>
    </tr>
  </table>
</template>
```

Q85. Write a Vue component that displays a card.**Answer:**

vue

```
<template>
  <div class="card">
    <h3>{{ title }}</h3>
    <p>{{ description }}</p>
  </div>
</template>
```

Q86. Write a Vue component that displays a modal.**Answer:**

vue

```
<template>
  <div v-if="show" class="modal">
    <p>Modal content</p>
    <button @click="show = false">Close</button>
  </div>
</template>
```

Q87. Write a Vue component that displays a form.

Answer: Same as Q69.

Q88. Write a Vue component that displays a chart.

Answer: Use Chart.js in `mounted()` .

Q89. Write a Vue component that displays a message.

Answer:

vue

```
<template><p>{{ message }}</p></template>
<script>export default { data() { return { message: 'Hello' }; } }</script>
```

Q90. Write a Vue component that displays a welcome message.

Answer:

vue

```
<template><h1>Welcome, {{ username }}</h1></template>
```

Q91. Write a Vue component that displays current time.

Answer:

vue

```
<template><p>{{ time }}</p></template>
<script>
export default {
  data() { return { time: new Date().toLocaleTimeString() }; },
  mounted() { setInterval(() => this.time = new Date().toLocaleTimeString(), 1000); }
```

```
}  
</script>
```

Q92. Write a Vue component that displays a clock.

Answer: Same as above.

Q93. Write a Vue component that toggles visibility.

Answer: Same as Q64.

Q94. Write a Vue component that increments count.

Answer:

vue

```
<template><button @click="count++">{{ count }}</button></template>  
<script>export default { data() { return { count: 0 }; } }</script>
```

Q95. Write a Vue component that says "Hi".

Answer:

vue

```
<template><button @click="sayHi">Click</button></template>  
<script>export default { methods: { sayHi() { alert('Hi'); } } }</script>
```

Q96. Write a Vue component that says "Hello".

Answer:

vue

```
<template><p>Hello</p></template>
```

Q97. Write a Vue component that says "GoodBye".

Answer:

vue

```
<template><p>GoodBye</p></template>
```

Q98. Write a Vue component that prints "Hello".

Answer: Same as Q96.

Q99. Write a Vue component that returns sum.

Answer:

vue

```
<template><p>{{ sum }}</p></template>
```

```
<script>export default { data() { return { a: 2, b: 3 }; }, computed: { sum() { return th  
is.a + this.b; } } }</script>
```

Q100. Write a Vue component that returns square.

Answer: Similar with $n * n$.

Q101. Write a JS function that prints your name.

Answer:

js

```
function printName() { console.log('Your Name'); }
```

Q102. Write a JS function that returns sum.

Answer:

js

```
function sum(a, b) { return a + b; }
```

Q103. Write a JS function that returns square.

Answer:

js

```
function square(n) { return n * n; }
```

Q104. Write a JS function that returns cube.

Answer:

js

```
function cube(n) { return n ** 3; }
```

Q105. Write a JS function that returns 4 times.

Answer:

js

```
function times4(n) { return n * 4; }
```

Q106. Write a JS function that returns full name.

Answer:

js

```
function fullName(first, last) { return first + ' ' + last; }
```

Q107. Write a JS function that returns square of 16.

Answer:

js

```
function square16() { console.log(16 * 16); }
```

Q108. Write a JS function that returns square of 2.

Answer:

js

```
function square2() { console.log(2 * 2); }
```

Q109. Write a JS function that returns sum of 3 numbers.

Answer:

js

```
function sum3(a, b, c) { return a + b + c; }
```

Q110. Write a JS function that returns cube root.

Answer:

js

```
function cubeRoot(n) { return Math.cbrt(n); }
```

Q111. Write a JS function that returns month.

Answer:

js

```
function getMonth(date) { return new Date(date).getMonth() + 1; }
```

Q112. Write a JS function that returns age.

Answer:

js

```
function age(dob) {  
  const diff = Date.now() - new Date(dob).getTime();  
  return Math.floor(diff / (1000 * 60 * 60 * 24 * 365.25));  
}
```

Q113. Write a JS function that returns index.

Answer:

js

```
function findIndex(str, sub) { return str.indexOf(sub); }
```

Q114. Write a JS function that returns boolean.

Answer:


```
js
```

```
function isEven(n) { return n % 2 === 0; }
```

Q115. Write a JS function that returns string.

Answer:

```
js
```

```
function greet() { return 'Hello'; }
```

Q116. Write a JS function that returns array.

Answer:

```
js
```

```
function getArray() { return [1, 2, 3]; }
```

Q117. Write a JS function that returns object.

Answer:

```
js
```

```
function getObj() { return { name: 'A', age: 20 }; }
```

Q118. Write a JS function that returns promise.

Answer:

```
js
```

```
function getPromise() { return Promise.resolve('Done'); }
```

Q119. Write a JS function that returns async.

Answer:

js

```
async function getData() { return 'Data'; }
```

Q120. Write a JS function that returns await.

Answer:

js

```
async function getData() { const res = await fetch('/api'); return res.json(); }
```

Q121. Write a JS for loop.

Answer:

js

```
for (let i = 0; i < 5; i++) { console.log(i); }
```

Q122. Write a JS while loop.

Answer:

js

```
let i = 0;  
while (i < 5) { console.log(i); i++; }
```

Q123. Write a JS if-else.

Answer:

js

```
if (x > 0) { console.log('positive'); } else { console.log('non-positive'); }
```

Q124. Write a JS switch case.**Answer:**

js

```
switch (x) {  
  case 1: console.log('one'); break;  
  case 2: console.log('two'); break;  
  default: console.log('other');  
}
```

Q125. Write a JS try-catch.**Answer:**

js

```
try { risky(); } catch (e) { console.error(e); }
```

Q126. Write a JS arrow function.**Answer:**

js

```
const add = (a, b) => a + b;
```

Q127. Write a JS anonymous function.

Answer:

js

```
setTimeout(function() { console.log('Hi'); }, 1000);
```

Q128. Write a JS higher order function.

Answer:

js

```
function apply(fn, x) { return fn(x); }
```

Q129. Write a JS map function.

Answer:

js

```
const doubled = [1,2,3].map(n => n * 2);
```

Q130. Write a JS filter function.

Answer:

js

```
const evens = [1,2,3,4].filter(n => n % 2 === 0);
```

Q131. Write a JS reduce function.

Answer:

js

```
const sum = [1,2,3].reduce((acc, n) => acc + n, 0);
```

Q132. Write a JS forEach function.

Answer:

js

```
[1,2,3].forEach(n => console.log(n));
```

Q133. Write a JS find function.

Answer:

js

```
const found = [1,2,3].find(n => n > 1);
```

Q134. Write a JS some function.

Answer:

js

```
const hasEven = [1,2,3].some(n => n % 2 === 0);
```

Q135. Write a JS every function.

Answer:

js

```
const allEven = [2,4,6].every(n => n % 2 === 0);
```

Q136. Write a JS sort function.

Answer:

js

```
[3,1,2].sort((a, b) => a - b);
```

Q137. Write a JS reverse function.

Answer:

js

```
[1,2,3].reverse();
```

Q138. Write a JS splice function.

Answer:

js

```
arr.splice(1, 2); // remove 2 elements from index 1
```

Q139. Write a JS slice function.

Answer:

js

```
arr.slice(1, 3);
```

Q140. Write a JS concat function.

Answer:

js

```
[1,2].concat([3,4]);
```

Q141. Write a JS join function.

Answer:

js

```
['a','b'].join('-');
```

Q142. Write a JS split function.

Answer:

js

```
'a-b-c'.split('-');
```

Q143. Write a JS replace function.

Answer:

js

```
'hello'.replace('h', 'H');
```

Q144. Write a JS trim function.

Answer:

js

```
' hi '.trim();
```

Q145. Write a JS toUpperCase function.

Answer:

js

```
'hi'.toUpperCase();
```

Q146. Write a JS toLowerCase function.

Answer:

js

```
'HI'.toLowerCase();
```

Q147. Write a JS charAt function.

Answer:

js

```
'hello'.charAt(1);
```

Q148. Write a JS indexOf function.

Answer:

js

```
'hello'.indexOf('e');
```

Q149. Write a JS includes function.

Answer:

js

```
'hello'.includes('ell');
```

Q150. Write a JS startsWith function.

Answer:

js

```
'hello'.startsWith('he');
```

Q151. Write a JS endsWith function.

Answer:

js

```
'hello'.endsWith('lo');
```

Q152. Write a JS repeat function.

Answer:

js

```
'ha'.repeat(3);
```

Q153. Write a JS padStart function.

Answer:

js

```
'5'.padStart(2, '0');
```

Q154. Write a JS padEnd function.

Answer:

js

```
'5'.padEnd(2, '0');
```

Q155. Write a JS match function.

Answer:

js

```
'hello'.match(/l/g);
```

Q156. Write a JS search function.

Answer:

js

```
'hello'.search('l');
```

Q157. Write a JS test function.

Answer:

js

```
/l/.test('hello');
```

Q158. Write a JS exec function.

Answer:

js

```
/l/.exec('hello');
```

Q159. Write a JS compile function.

Answer:

js

```
new RegExp('l', 'g').compile('l', 'g');
```

Q160. Write a JS toString function.

Answer:

js

```
(123).toString();
```

Q161. Write a JS valueOf function.

Answer:

js

```
(123).valueOf();
```

Q162. Write a JS toFixed function.

Answer:

js

```
(3.14159).toFixed(2);
```

Q163. Write a JS toPrecision function.

Answer:

js

```
(3.14159).toPrecision(3);
```

Q164. Write a JS toExponential function.

Answer:

js

```
(12345).toExponential(2);
```

Q165. Write a JS Number function.

Answer:

js

```
Number('123');
```

Q166. Write a JS parseInt function.

Answer:

js

```
parseInt('123px');
```

Q167. Write a JS parseFloat function.

Answer:

js

```
parseFloat('3.14abc');
```

Q168. Write a JS isNaN function.

Answer:

js

```
isNaN('abc');
```

Q169. Write a JS isFinite function.

Answer:

js

```
isFinite(123);
```

Q170. Write a JS isInteger function.

Answer:

js

```
Number.isInteger(123);
```

Q171. Write a JS isSafeInteger function.

Answer:

js

```
Number.isSafeInteger(123);
```

Q172. Write a JS MAX_VALUE.

Answer:

js

```
Number.MAX_VALUE;
```

Q173. Write a JS MIN_VALUE.

Answer:

js

```
Number.MIN_VALUE;
```

Q174. Write a JS POSITIVE_INFINITY.

Answer:

js

```
Number.POSITIVE_INFINITY;
```

Q175. Write a JS NEGATIVE_INFINITY.

Answer:

js

```
Number.NEGATIVE_INFINITY;
```

Q176. Write a JS NaN.

Answer:

```
js
```

```
NaN;
```

Q177. Write a JS undefined.

Answer:

```
js
```

```
undefined;
```

Q178. Write a JS null.

Answer:

```
js
```

```
null;
```

Q179. Write a JS boolean.

Answer:

```
js
```

```
true;
```

Q180. Write a JS symbol.

Answer:

js

```
Symbol('id');
```

Q181. Write a JS bigint.

Answer:

js

```
123n;
```

Q182. Write a JS typeof.

Answer:

js

```
typeof 123;
```

Q183. Write a JS instanceof.

Answer:

js

```
[] instanceof Array;
```

Q184. Write a JS delete.

Answer:

js

```
delete obj.prop;
```

Q185. Write a JS in.

Answer:

js

```
'prop' in obj;
```

Q186. Write a JS void.

Answer:

js

```
void 0;
```

Q187. Write a JS yield.

Answer:

js

```
function* gen() { yield 1; }
```

Q188. Write a JS await.

Answer:

js

```
await fetch('/api');
```

Q189. Write a JS async.

Answer:

js

```
async function f() {}
```

Q190. Write a JS class.

Answer:

js

```
class Person { constructor(name) { this.name = name; } }
```

Q191. Write a JS constructor.

Answer:

js

```
constructor(name) { this.name = name; }
```

Q192. Write a JS extends.

Answer:

js

```
class Student extends Person {}
```

Q193. Write a JS super.

Answer:

```
js  
  
super(name);
```

Q194. Write a JS static.

Answer:

```
js  
  
static greet() { return 'Hi'; }
```

Q195. Write a JS get.

Answer:

```
js  
  
get name() { return this._name; }
```

Q196. Write a JS set.

Answer:

```
js  
  
set name(v) { this._name = v; }
```

Q197. Write a JS new.

Answer:

```
js
```

```
new Person('A');
```

Q198. Write a JS this.

Answer:

```
js
```

```
this.name;
```

Q199. Write a JS prototype.

Answer:

```
js
```

```
Person.prototype.greet = function() { return 'Hi'; };
```

Q200. Write a JS proto.

Answer:

```
js
```

```
obj.__proto__;
```

9. THEORY — GENERAL / MISCELLANEOUS

Q1. What is a web application?

Answer: A software application that runs on a web server and is accessed via a browser over the internet.

Q2. What are the main aspects of web application development?

Answer: Frontend (UI), backend (logic), database, API, authentication, deployment, security, scalability.

Q3. What is frontend?

Answer: The client-side part users see and interact with (HTML, CSS, JS, Vue).

Q4. What is backend?

Answer: The server-side part handling logic, database, and APIs (Flask, Python).

Q5. Difference between frontend and backend.

Answer: Frontend: UI/UX, runs in browser. Backend: logic/data, runs on server.

Q6. Which is more important — frontend or backend?

Answer: Both are important. Frontend for user experience; backend for functionality, security, and data integrity.

Q7. What is frontend validation vs backend validation?

Answer: Frontend: in browser (JS); fast but bypassable. Backend: on server; secure but slower.

Q8. Which validation is more important?

Answer: Backend validation, because frontend can be bypassed.

Q9. Can frontend validation be bypassed?

Answer: Yes, by disabling JS or using tools like Postman.

Q10. What is the difference between security and privacy?

Answer: Security: protecting data from threats. Privacy: controlling who accesses personal data.

Q11. What is a framework?

Answer: A pre-built structure/library that provides reusable code and conventions (e.g., Flask, Vue).

Q12. What is a library?

Answer: A collection of functions/classes you call (e.g., jQuery, Axios).

Q13. What is the difference between framework and library?

Answer: Framework controls the flow (inversion of control); library is called by your code.

Q14. What is JavaScript?

Answer: A dynamic, interpreted programming language for the web.

Q15. What is JavaScript used for?

Answer: Client-side interactivity, server-side (Node.js), mobile apps, games.

Q16. How does JavaScript work?

Answer: Parsed and executed by the JS engine (V8, SpiderMonkey) in the browser or Node.js.

Q17. How is JavaScript initiated?

Answer: Via `<script>` tags, event handlers, or module imports.

Q18. What is an event loop?

Answer: A mechanism that handles async callbacks by moving them from the task queue to the call stack when it's empty.

Q19. What is a callback function?

Answer: A function passed as an argument to another function, called later.

Q20. What is a promise?

Answer: An object representing eventual completion/failure of an async operation.

Q21. What are the stages of a promise?

Answer: Pending, Fulfilled, Rejected.

Q22. What is async/await?

Answer: Syntactic sugar over promises for writing asynchronous code synchronously.

Q23. What is the difference between synchronous and asynchronous?

Answer: Sync: blocking, sequential. Async: non-blocking, concurrent.

Q24. What is an asynchronous function?

Answer: A function declared with `async` that returns a promise.

Q25. What does an async function return?

Answer: A promise.

Q26. What is await?

Answer: Pauses execution until the promise resolves.

Q27. What is a higher-order function?

Answer: A function that takes/returns another function.

Q28. What is a closure?

Answer: A function that remembers its lexical scope even when called outside it.

Q29. What is hoisting?

Answer: Variables/functions are moved to the top of their scope before execution.

Q30. What is scope?

Answer: The context in which variables are accessible.

Q31. What is lexical scope?

Answer: Scope determined by where variables are declared in the code.

Q32. What is a variable?

Answer: A named storage for data.

Q33. What is const vs let vs var?

Answer: const: immutable; let: block-scoped; var: function-scoped.

Q34. Why use const instead of let?

Answer: Prevents reassignment, safer, clearer intent.

Q35. What is a function?

Answer: A reusable block of code.

Q36. What is an arrow function?

Answer: Shorter syntax, no own `this`.

Q37. What is an anonymous function?

Answer: A function without a name.

Q38. What is a named function?

Answer: A function with a name.

Q39. What is a callback?

Answer: A function passed to another function.

Q40. What is a promise chain?

Answer: `.then().then()` for sequential async operations.

Q41. What is Promise.all?

Answer: Waits for all promises to resolve.

Q42. What is Promise.race?

Answer: Resolves when the first promise settles.

Q43. What is Promise.allSettled?

Answer: Waits for all promises to settle (resolve or reject).

Q44. What is Promise.any?

Answer: Resolves when the first promise fulfills.

Q45. What is async/await?

Answer: See Q22.

Q46. What is try/catch?

Answer: Error handling.

Q47. What is throw?

Answer: Throws an error.

Q48. What is finally?

Answer: Block that runs after try/catch regardless.

Q49. What is an error?

Answer: An object representing a problem.

Q50. What is an exception?

Answer: An error that disrupts normal flow.

Q51. What is a syntax error?

Answer: Invalid code.

Q52. What is a runtime error?

Answer: Error during execution.

Q53. What is a logical error?

Answer: Code runs but produces wrong results.

Q54. What is a type error?

Answer: Wrong type used.

Q55. What is a reference error?

Answer: Using an undefined variable.

Q56. What is a range error?

Answer: Number out of range.

Q57. What is a URI error?

Answer: Invalid URI.

Q58. What is an eval error?

Answer: Error in eval().

Q59. What is a DOM?

Answer: Document Object Model — tree representation of HTML.

Q60. What is Virtual DOM?

Answer: JS representation of real DOM.

Q61. What is Shadow DOM?

Answer: Encapsulated DOM for web components.

Q62. What is a DOM node?

Answer: An element in the DOM tree.

Q63. What is a DOM element?

Answer: An HTML tag in the DOM.

Q64. What is DOM manipulation?

Answer: Changing DOM via JS.

Q65. What is an event?

Answer: User/system action (click, load).

Q66. What is event bubbling?

Answer: Event propagates from child to parent.

Q67. What is event capturing?

Answer: Event propagates from parent to child.

Q68. What is event delegation?

Answer: Attaching a listener to a parent for multiple children.

Q69. What is event propagation?

Answer: Bubbling + capturing.

Q70. What is preventDefault?

Answer: Prevents default action.

Q71. What is stopPropagation?

Answer: Stops event propagation.

Q72. What is addEventListener?

Answer: Attaches an event handler.

Q73. What is removeEventListener?

Answer: Removes an event handler.

Q74. What is querySelector?

Answer: Selects the first matching element.

Q75. What is getElementById?

Answer: Selects by ID.

Q76. What is getElementsByClassName?

Answer: Selects by class.

Q77. What is `getElementsByTagName`?

Answer: Selects by tag.

Q78. What is `createElement`?

Answer: Creates a new element.

Q79. What is `appendChild`?

Answer: Adds a child element.

Q80. What is `removeChild`?

Answer: Removes a child element.

Q81. What is `innerHTML`?

Answer: HTML content of an element.

Q82. What is `textContent`?

Answer: Text content of an element.

Q83. What is `innerText`?

Answer: Visible text of an element.

Q84. What is `setAttribute`?

Answer: Sets an attribute.

Q85. What is `getAttribute`?

Answer: Gets an attribute.

Q86. What is `classList`?

Answer: List of classes on an element.

Q87. What is `style`?

Answer: Inline style object.

Q88. What is `dataset`?

Answer: Custom data attributes.

Q89. What is `localStorage`?

Answer: Persistent browser storage.

Q90. What is `sessionStorage`?

Answer: Browser storage for the session.

Q91. What is `cookie`?

Answer: Small data stored by the browser, sent with requests.

Q92. What is `IndexedDB`?

Answer: Browser database for large data.

Q93. What is Web SQL?

Answer: Deprecated browser SQL database.

Q94. What is Cache API?

Answer: Stores HTTP responses.

Q95. What is Service Worker?

Answer: Background script for offline/push.

Q96. What is PWA?

Answer: Progressive Web App.

Q97. What is SPA?

Answer: Single Page Application.

Q98. What is MPA?

Answer: Multi-Page Application.

Q99. What is SSR?

Answer: Server-Side Rendering.

Q100. What is CSR?

Answer: Client-Side Rendering.

Q101. What is SSG?

Answer: Static Site Generation.

Q102. What is ISR?

Answer: Incremental Static Regeneration.

Q103. What is hydration?

Answer: Attaching JS to server-rendered HTML.

Q104. What is lazy loading?

Answer: Loading on demand.

Q105. What is code splitting?

Answer: Splitting bundles into smaller chunks.

Q106. What is tree shaking?

Answer: Removing unused code.

Q107. What is bundling?

Answer: Combining files into one.

Q108. What is minification?

Answer: Removing whitespace/comments.

Q109. What is uglification?

Answer: Renaming variables to shorten code.

Q110. What is transpilation?

Answer: Converting modern JS to older JS.

Q111. What is polyfill?

Answer: Code that adds missing features.

Q112. What is Babel?

Answer: JS transpiler.

Q113. What is Webpack?

Answer: Module bundler.

Q114. What is Vite?

Answer: Fast build tool.

Q115. What is Rollup?

Answer: Module bundler for libraries.

Q116. What is Parcel?

Answer: Zero-config bundler.

Q117. What is npm?

Answer: Node Package Manager.

Q118. What is yarn?

Answer: Alternative package manager.

Q119. What is pnpm?

Answer: Fast, disk-efficient package manager.

Q120. What is package.json?

Answer: Project manifest.

Q121. What is package-lock.json?

Answer: Locks dependency versions.

Q122. What is node_modules?

Answer: Folder with installed packages.

Q123. What is .gitignore?

Answer: Files Git should ignore.

Q124. What is README.md?

Answer: Project documentation.

Q125. What is a license?

Answer: Legal terms for using code.

Q126. What is open source?

Answer: Code available for public use/modification.

Q127. What is MIT license?

Answer: Permissive open-source license.

Q128. What is Apache license?

Answer: Permissive license with patent grant.

Q129. What is GPL?

Answer: Copyleft license.

Q130. What is a repository?

Answer: Storage for code and history.

Q131. What is Git?

Answer: Distributed version control system.

Q132. What is GitHub?

Answer: Cloud platform for Git repos.

Q133. What is GitLab?

Answer: DevOps platform with Git repos.

Q134. What is Bitbucket?

Answer: Git repo hosting by Atlassian.

Q135. What is a commit?

Answer: Snapshot of changes.

Q136. What is a branch?

Answer: Independent line of development.

Q137. What is a merge?

Answer: Combining branches.

Q138. What is a pull request?

Answer: Request to merge changes.

Q139. What is a fork?

Answer: Copy of a repo.

Q140. What is a clone?

Answer: Local copy of a repo.

Q141. What is a push?

Answer: Upload commits.

Q142. What is a pull?

Answer: Download commits.

Q143. What is a fetch?

Answer: Download without merging.

Q144. What is a remote?

Answer: Remote repo URL.

Q145. What is origin?

Answer: Default remote name.

Q146. What is upstream?

Answer: Original repo for a fork.

Q147. What is HEAD?

Answer: Current commit pointer.

Q148. What is a tag?

Answer: Named reference to a commit.

Q149. What is a release?

Answer: Tagged version with notes.

Q150. What is a version?

Answer: Identifier for a release.

Q151. What is semantic versioning?

Answer: MAJOR.MINOR.PATCH.

Q152. What is a changelog?

Answer: List of changes per version.

Q153. What is a contribution?

Answer: Code/docs added to a project.

Q154. What is a collaborator?

Answer: User with write access.

Q155. What is a maintainer?

Answer: Person managing a project.

Q156. What is an issue?

Answer: Bug/feature request.

Q157. What is a milestone?

Answer: Group of issues for a goal.

Q158. What is a project board?

Answer: Kanban board for tasks.

Q159. What is a wiki?

Answer: Documentation pages.

Q160. What is a gist?

Answer: Snippet sharing.

Q161. What is a GitHub Action?

Answer: CI/CD workflow.

Q162. What is CI/CD?

Answer: Continuous Integration/Continuous Deployment.

Q163. What is continuous integration?

Answer: Merging code frequently with automated tests.

Q164. What is continuous deployment?

Answer: Auto-deploy to production.

Q165. What is continuous delivery?

Answer: Auto-deploy to staging, manual to production.

Q166. What is a pipeline?

Answer: Automated build/test/deploy process.

Q167. What is a build?

Answer: Compiling/bundling code.

Q168. What is a test?

Answer: Verifying code correctness.

Q169. What is a deployment?

Answer: Releasing to a server.

Q170. What is a staging environment?

Answer: Pre-production environment.

Q171. What is a production environment?

Answer: Live environment.

Q172. What is a development environment?

Answer: Local/dev setup.

Q173. What is a local environment?

Answer: Your machine.

Q174. What is a virtual environment?

Answer: Isolated Python environment.

Q175. What is a container?

Answer: Lightweight, isolated runtime.

Q176. What is Docker?

Answer: Containerization platform.

Q177. What is Kubernetes?

Answer: Container orchestration.

Q178. What is a pod?

Answer: Smallest deployable unit in K8s.

Q179. What is a service?

Answer: Network endpoint for pods.

Q180. What is a deployment?

Answer: Declarative pod management.

Q181. What is a replica set?

Answer: Ensures a number of pod replicas.

Q182. What is a stateful set?

Answer: For stateful apps.

Q183. What is a daemon set?

Answer: Runs a pod on every node.

Q184. What is a job?

Answer: Runs a task to completion.

Q185. What is a cron job?

Answer: Scheduled job.

Q186. What is a config map?

Answer: Configuration data.

Q187. What is a secret?

Answer: Sensitive data.

Q188. What is a volume?

Answer: Storage for pods.

Q189. What is a persistent volume?

Answer: Durable storage.

Q190. What is a namespace?

Answer: Virtual cluster.

Q191. What is a cluster?

Answer: Set of nodes.

Q192. What is a node?

Answer: Worker machine.

Q193. What is a master node?

Answer: Control plane node.

Q194. What is a worker node?

Answer: Runs pods.

Q195. What is a control plane?

Answer: Manages the cluster.

Q196. What is a kubelet?

Answer: Agent on each node.

Q197. What is a kube-proxy?

Answer: Network proxy.

Q198. What is an API server?

Answer: K8s API endpoint.

Q199. What is etcd?

Answer: Key-value store for K8s.

Q200. What is a scheduler?

Answer: Assigns pods to nodes.

10. PROJECT-SPECIFIC QUESTIONS

Q1. Why did you choose this project?

Answer: [Your reason — e.g., interest in the domain, relevance, learning goals.]

Q2. Why did you choose a music app instead of a grocery store?

Answer: [Your reason.]

Q3. What is your project statement?

Answer: [One-line summary of your project.]

Q4. What problem does your project solve?

Answer: [Describe the problem and your solution.]

Q5. Who are the users of your project?

Answer: [e.g., Admin, User, Staff.]

Q6. What are the roles in your project?

Answer: [List roles.]

Q7. What can an admin do?

Answer: [List admin capabilities.]

Q8. What can a user do?

Answer: [List user capabilities.]

Q9. What can a staff do?

Answer: [List staff capabilities.]

Q10. What can a doctor do?

Answer: [List doctor capabilities.]

Q11. What can a patient do?

Answer: [List patient capabilities.]

Q12. What can a company do?

Answer: [List company capabilities.]

Q13. What can a student do?

Answer: [List student capabilities.]

Q14. What can a trekker do?

Answer: [List trekker capabilities.]

Q15. What are the core features?

Answer: [List core features.]

Q16. What are the additional features?

Answer: [List additional features.]

Q17. What features did you not implement?

Answer: [List missing features and reasons.]

Q18. Why didn't you implement ticket cancellation?

Answer: [Reason — e.g., not in core requirements.]

Q19. Why didn't you implement vehicle information?

Answer: [Reason.]

Q20. Why didn't you implement eligibility criteria?

Answer: [Reason.]

Q21. Why didn't you implement password validation?

Answer: [Reason.]

Q22. Why didn't you implement email notifications?

Answer: [Reason.]

Q23. Why didn't you implement the export feature?

Answer: [Reason.]

Q24. Why didn't you implement the favourite feature?

Answer: [Reason.]

Q25. Why didn't you implement the chat feature?

Answer: [Reason.]

Q26. Why didn't you implement the review feature?

Answer: [Reason.]

Q27. Why didn't you implement the search feature?

Answer: [Reason.]

Q28. Why didn't you implement the filter feature?

Answer: [Reason.]

Q29. Why didn't you implement the sort feature?

Answer: [Reason.]

Q30. Why didn't you implement pagination?

Answer: [Reason.]

Q31. Why didn't you implement the CSV download location?

Answer: [Reason — e.g., browser default is fine.]

Q32. Why didn't you implement the seat selection?

Answer: [Reason.]

Q33. Why didn't you implement the payment gateway?

Answer: [Reason.]

Q34. Why didn't you implement the notification system?

Answer: [Reason.]

Q35. Why didn't you implement the rating system?

Answer: [Reason.]

Q36. Why didn't you implement the recommendation system?

Answer: [Reason.]

Q37. Why didn't you implement the analytics dashboard?

Answer: [Reason.]

Q38. Why didn't you implement the reporting system?

Answer: [Reason.]

Q39. Why didn't you implement the audit log?

Answer: [Reason.]

Q40. Why didn't you implement the backup system?

Answer: [Reason.]

Q41. Why didn't you implement the logging system?

Answer: [Reason.]

Q42. Why didn't you implement the monitoring system?

Answer: [Reason.]

Q43. Why didn't you implement the error tracking?

Answer: [Reason.]

Q44. Why didn't you implement the performance monitoring?

Answer: [Reason.]

Q45. Why didn't you implement the security audit?

Answer: [Reason.]

Q46. Why didn't you implement the penetration testing?

Answer: [Reason.]

Q47. Why didn't you implement the load testing?

Answer: [Reason.]

Q48. Why didn't you implement the unit testing?

Answer: [Reason.]

Q49. Why didn't you implement the integration testing?

Answer: [Reason.]

Q50. Why didn't you implement the end-to-end testing?

Answer: [Reason.]

11. VIVA EXPERIENCE & PROCTOR FEEDBACK (Meta)

Q1. How was your viva experience?

Answer: [Your experience.]

Q2. What suggestions do you have for fellow students?

Answer: [Your suggestions.]

Q3. What tips would you give for this examiner?

Answer: [Your tips.]

Q4. Was the proctor rude or chill?

Answer: [Your observation.]

Q5. Did the proctor help you?

Answer: [Yes/No.]

Q6. Did the proctor give you time?

Answer: [Yes/No.]

Q7. Did the proctor interrupt you?

Answer: [Yes/No.]

Q8. Did the proctor ask questions from the sheet?

Answer: [Yes/No.]

Q9. Did the proctor ask questions outside the sheet?

Answer: [Yes/No.]

Q10. Did the proctor ask you to code?

Answer: [Yes/No.]

Q11. Did the proctor ask you to change code?

Answer: [Yes/No.]

Q12. Did the proctor ask you to debug?

Answer: [Yes/No.]

Q13. Did the proctor ask you to explain code?

Answer: [Yes/No.]

Q14. Did the proctor ask you to demonstrate?

Answer: [Yes/No.]

Q15. Did the proctor ask you to show GitHub?

Answer: [Yes/No.]

Q16. Did the proctor ask you to show ID?

Answer: [Yes/No.]

Q17. Did the proctor ask you to share screen?

Answer: [Yes/No.]

Q18. Did the proctor ask you to turn on camera?

Answer: [Yes/No.]

Q19. Did the proctor join on time?

Answer: [Yes/No.]

Q20. Did the proctor join late?

Answer: [Yes/No.]

Q21. How long was the viva?

Answer: [Duration.]

Q22. Did the proctor give feedback at the end?

Answer: [Yes/No.]

Q23. Did the proctor wish you luck?

Answer: [Yes/No.]

Q24. Did the proctor say "you did well"?

Answer: [Yes/No.]

Q25. Did the proctor say "you need to improve"?

Answer: [Yes/No.]

Q26. Did the proctor say "study more"?

Answer: [Yes/No.]

Q27. Did the proctor say "focus on basics"?

Answer: [Yes/No.]

Q28. Did the proctor say "focus on backend"?

Answer: [Yes/No.]

Q29. Did the proctor say "focus on frontend"?

Answer: [Yes/No.]

Q30. Did the proctor say "focus on theory"?

Answer: [Yes/No.]

Q31. Did the proctor say "focus on coding"?

Answer: [Yes/No.]

Q32. Did the proctor say "practice coding"?

Answer: [Yes/No.]

Q33. Did the proctor say "read the sheet"?

Answer: [Yes/No.]

Q34. Did the proctor say "don't rely on the sheet"?

Answer: [Yes/No.]

Q35. Did the proctor say "the sheet is outdated"?

Answer: [Yes/No.]

Q36. Did the proctor say "I know about the sheet"?

Answer: [Yes/No.]

Q37. Did the proctor ask for the sheet link?

Answer: [Yes/No.]

Q38. Did the proctor ask about AI usage?

Answer: [Yes/No.]

Q39. Did the proctor ask about AI percentage?

Answer: [Yes/No.]

Q40. Did the proctor ask to show AI history?

Answer: [Yes/No.]

Q41. Did the proctor accuse you of malpractice?

Answer: [Yes/No.]

Q42. Did the proctor report you?

Answer: [Yes/No.]

Q43. Did the proctor pass you?

Answer: [Yes/No.]

Q44. Did the proctor fail you?

Answer: [Yes/No.]

Q45. Did the proctor give you a grade?

Answer: [Yes/No.]

Q46. Did the proctor tell you your grade?

Answer: [Yes/No.]

Q47. Did the proctor say "you will know after results"?

Answer: [Yes/No.]

Q48. Did the proctor say "all the best for level 2"?

Answer: [Yes/No.]

Q49. Did the proctor say "prepare for level 3"?

Answer: [Yes/No.]

Q50. Did the proctor say "you are eligible for MAD2"?

Answer: [Yes/No.]

12. FREQUENTLY ASKED QUESTIONS — TOP 50 WITH ANSWERS

Q1. What is Vue.js?

Vue.js is a progressive JavaScript framework for building UIs and SPAs, featuring reactive data binding, components, and a virtual DOM.

Q2. What is CLI vs CDN? Which did you use?

CLI: full build tooling, modular, scalable. CDN: simple script tag, no build. [Your choice.]

Q3. What are lifecycle hooks?

Methods that run at specific stages: beforeCreate, created, beforeMount, mounted, beforeUpdate, updated, beforeUnmount, unmounted.

Q4. What is v-if vs v-show?

v-if: conditionally renders (removes from DOM). v-show: toggles CSS display.

Q5. What is caching? Where did you implement it?

Storing frequently accessed data in fast storage. Implemented on [route] using Flask-Caching + Redis.

Q6. What is Celery?

Distributed task queue for async and scheduled tasks.

Q7. What is Redis?

In-memory key-value store used for caching and as a Celery broker.

Q8. What is ORM?

Object-Relational Mapping — maps tables to classes and rows to objects.

Q9. What is SQLAlchemy?

Python SQL toolkit and ORM.

Q10. What is authentication vs authorization?

AuthN: identity. AuthZ: permissions.

Q11. What is JWT? Where is it stored?

JSON Web Token. Stored in localStorage/sessionStorage/cookies.

Q12. What is RBAC? How is it implemented?

Role-Based Access Control. Implemented via roles and decorators like `@roles_required`.

Q13. What are props? How to pass data child→parent?

Props: parent→child data. Child→parent: `$emit` .

Q14. What is v-model vs v-bind?

v-model: two-way binding. v-bind: one-way binding.

Q15. What are webhooks?

Event-driven HTTP callbacks.

Q16. What is celery beat vs celery worker?

Beat: scheduler. Worker: executor.

Q17. What is primary key vs unique key?

Primary: NOT NULL, one. Unique: allows NULL, multiple.

Q18. What is backref vs back_populates?

backref: one-sided. back_populates: two-sided.

Q19. What is cascade?

Propagating operations (delete/update) to related rows.

Q20. What is lazy loading?

Loading related data on demand.

Q21. What is SPA?

Single Page Application — no full reloads.

Q22. What is PUT vs PATCH vs POST?

PUT: full replace. PATCH: partial update. POST: create.

Q23. What is SQL injection?

Injecting malicious SQL via input. Prevented by ORM/parameterized queries.

Q24. What is CORS?

Cross-Origin Resource Sharing — browser security for cross-origin requests.

Q25. What is CSRF?

Cross-Site Request Forgery — tricking users into unwanted actions.

Q26. What is session vs token based auth?

Session: server-side state. Token: stateless.

Q27. What is local storage vs session storage?

Local: persists. Session: cleared on tab close.

Q28. What is memoization vs caching?

Memoization: caching function results. Caching: broader.

Q29. What is Vuex vs Pinia?

Vuex: older, mutations/actions. Pinia: newer, simpler, Composition API.

Q30. What is computed vs watch?

Computed: derives/caches values. Watch: side effects on change.

Q31. What is v-if vs v-for?

v-if: conditional. v-for: list rendering.

Q32. What is GET vs POST?

GET: retrieve, URL params. POST: submit, body params.

Q33. What is PUT vs POST?

PUT: idempotent, replace. POST: non-idempotent, create.

Q34. What is PUT vs PATCH?

PUT: full replace. PATCH: partial update.

Q35. What is 400, 401, 403, 404?

Bad Request, Unauthorized, Forbidden, Not Found.

Q36. What is Redis vs SQLite?

Redis: in-memory NoSQL. SQLite: file-based RDBMS.

Q37. What is relational vs non-relational DB?

Relational: tables, ACID. Non-relational: flexible schema, eventual consistency.

Q38. What is client-server vs distributed?

Client-server: centralized. Distributed: multiple nodes.

Q39. What is vertical vs horizontal scaling?

Vertical: bigger server. Horizontal: more servers.

Q40. What is frontend vs backend?

Frontend: UI. Backend: logic/data.

Q41. What is authentication vs authorization?

AuthN: identity. AuthZ: permissions.

Q42. What is security vs privacy?

Security: protection. Privacy: control over personal data.

Q43. What is Git vs VCS?

Git is a VCS (distributed). VCS is the general concept.

Q44. What is hashing vs encryption?

Hashing: one-way. Encryption: reversible.

Q45. What is API vs webhook?

API: pull. Webhook: push.

Q46. What is webhook vs websocket?

Webhook: one-way HTTP POST. WebSocket: two-way persistent.

Q47. What is async vs sync?

Async: non-blocking. Sync: blocking.

Q48. What is promise vs async/await?

Promise: object. async/await: syntactic sugar over promises.

Q49. What is a callback?

A function passed to another function, called later.

Q50. What is an event loop?

Mechanism that handles async callbacks.

This guide is now fully self-contained. Every question includes its answer. Good luck for your viva!

This response is AI-generated, for reference only.